

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное
образовательное учреждение высшего образования
«Череповецкий государственный университет»

Институт (факультет)	информационных технологий
Направление подготовки (специальность)	09.04.04 Программная инженерия (Искусственный интеллект)
Выпускающая кафедра	Математическое и программное обеспечение ЭВМ

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Название работы	Программное обеспечение генератора HTML-шаблонов по рукописным рисункам-макетам
-----------------	---

Студента	Квашенникова Михаила Сергеевича Ф.И.О.
----------	---

ДОПУСТИТЬ К ЗАЩИТЕ

Директор института (декан факультета)	(Ершов Е.В.)
Заведующий выпускающей кафедрой	(Ершов Е.В.)
Руководитель выпускной квалификационной работы	(Юдина О.В.)
Консультант по технико-экономическому обоснованию разработки	(Виноградова Л.Н.)
Нормоконтролер	(Виноградова Л.Н.)
Выпускник	(Квашенников М.С.)

Череповец
2024 г.

Оглавление

Введение.....	4
1. Характеристика проблемы генерирования HTML-шаблонов по рукописным рисункам-макетам.....	8
1.1. Анализ существующих моделей, методов и средств генерирования HTML-шаблонов по рукописным рисункам-макетам.....	8
1.2. Описание предметной области генерирования HTML-шаблонов по рукописным рисункам-макетам.....	16
1.3. Постановка задачи, разработка требований к информационному и аппаратно-программному обеспечению генерирования HTML-шаблонов по рукописным рисункам-макетам.....	21
1.4. Выводы по первому разделу.....	23
2. Математическое и алгоритмическое обеспечение генерирования HTML- шаблонов по рукописным-макетам.....	24
2.1. Описание математической модели детектирования объектов с рукописных рисунков-макетов для генерирования HTML-шаблонов.....	24
2.2. Адаптация и тестирование математической модели	36
2.3. Результаты моделирования детектора объектов с рукописных рисунков-макетов для генерирования HTML-шаблонов.....	45
2.4. Разработка метода генерирования HTML-шаблонов по рукописным рисункам-макетам	50
2.5. Разработка алгоритмов генерирования HTML-шаблонов по рукописным рисункам-макетам.....	51
2.6. Выводы по разделу 2	58
3. Проектирование информационного и программного обеспечения генерирования HTML-шаблонов по рукописным рисункам-макетам	59
3.1. Концептуальное моделирование проектируемой системы.....	59
3.1.1. Модель «черного ящика»	59
3.1.2. Модель состава	59

3.1.3. Модель структуры.....	61
3.1.4. Модель «белого ящика»	62
3.2. Проектирование программного обеспечения	63
3.2.1. Выбор технологии проектирования, среды и языка программирования	63
3.2.2. Разработка логической модели	69
3.2.3. Разработка физической модели	81
3.2.4. Разработка интерфейсов	85
3.3. Выводы по разделу 3	94
4. Результаты экспериментальной проверки программного обеспечения генерирования HTML-шаблонов по рукописным рисункам-макетам	95
4.1. Описание эксперимента и экспериментальных данных.....	95
4.2. Результаты экспериментальной проверки алгоритма генерирования HTML-шаблона	99
4.3. Результаты экспериментальной проверки модулей и классов программного обеспечения генератора HTML-шаблонов по рукописным рисункам-макетам	115
4.5. Выводы по результатам экспериментального тестирования	120
Заключение	121
Список литературы	124
Приложение 1. Устав проекта.....	129
Приложение 2. Текст программы	146
Приложение 3. Спецификация.....	155
Приложение 4. Руководство пользователя	157
Приложение 5. Акт о внедрении	160

Введение

Актуальность темы. В современном мире разрабатывается и продвигается большое количество интернет-сайтов и разрабатываются кроссплатформенные приложения. В большинстве случаев, обладателями сайтов являются компании малых, средних, крупных бизнесов, а в некоторых случаях - и частные лица. Создание веб-сайтов для компаний имеет ряд преимуществ:

- помогает бизнесу достичь большего количества клиентов и увеличить своё присутствие на рынке;
- позволяют компаниям продемонстрировать свои продукты и услуги, что может способствовать большему количеству продаж и повышению доходов;
- онлайн-присутствие позволяет компаниям получать отзывы от клиентов и разрабатывать лучшие стратегии для достижения потребностей покупателей;
- присутствие в Сети помогает бизнесу стать более надежным и уважаемым в глазах клиентов, что приводит к повышению лояльности и доверия.

Новые веб-сайты и кроссплатформенные приложения создаются с использованием различных языков и технологий программирования, включая HTML, CSS, JavaScript и другие. HTML (язык гипертекстовой разметки) используется для создания структуры и содержимого веб-сайта, а CSS (каскадные таблицы стилей) — для оформления внешнего вида. JavaScript используется для добавления интерактивности на сайт. Другие технологии могут включать системы баз данных, такие как MySQL, серверные языки сценариев, такие как PHP и ASP.NET, среды веб-разработки, такие как Ruby on Rails и Node.js, и интерфейсные среды, такие как React и Angular. В конце концов, современные веб-сайты невероятно сложны и требуют глубокого понимания компьютерных наук и технических навыков для их создания.

Также стоит отметить, что HTML может быть использован не только для построения веб-сайтов, но также и для разработки интерфейса для компьютерных и мобильных приложений.

С каждым годом процесс создания сайтов и HTML-интерфейсов автоматизируется: создаются инструменты для быстрого написания HTML-кода, разрабатываются конструкторы сайтов (включающие уже готовые наборы HTML-шаблонов). Популярными становятся методы генерации HTML-шаблонов без использования какого-либо кода и знаний программирования, в данном случае полезность данного программного обеспечения играет большую роль для людей, которые не знают синтаксиса HTML.

На текущий момент, существует мало решений для генерации HTML-шаблонов по рукописным рисункам-макетам, и данные решения обладают определёнными недостатками:

- малое количество классов для распознавания;
- низкая точность расположения объектов;
- недостаточное количество выходных данных при распознавании объектов, из которых состоит макет;
- нет инструментов для редактирования стилей и контента без использования кода для сгенерированных HTML-шаблонов.

Целью работы является сокращение затрат времени на написание HTML-кода при разработке кроссплатформенных приложений (Android, Web, iOS, Desktop).

Для достижения поставленной цели решены следующие задачи:

- выполнен анализ предметной области в рамках проводимых исследовательских работ;
- выполнено теоретическое обоснование решаемой задачи, описание методов и модели, реализован алгоритм для генерации HTML-кода по рукописным рисункам-макетам;

- выполнено проектирование программного обеспечения;
- проведена экспериментальная проверка алгоритма генерации HTML-шаблонов по рукописным рисункам-макетам.

Объект исследования: система распознавания рукописных рисунков-макетов и генерации HTML-кода.

Предмет исследования: алгоритмическое обеспечение распознавания рукописных рисунков-макетов и генерации HTML-кода.

Методы исследования. Для решения поставленных задач в работе использовались методы математического моделирования, машинного обучения, алгоритмы компьютерной обработки изображений, анализ источников по проблеме исследования.

Научная новизна заключается в разработке алгоритма генерирования HTML-шаблона на основе рукописного рисунка-макета, отличающегося от известных возможностью адаптировать шаблон под разные разрешения устройств, имеющий повышенную точность расположения объектов на шаблоне.

Практическая значимость работы заключается в возможности применения полученного в ходе исследования программного продукта, который может использоваться для автоматизации рутинных задач по написанию HTML-шаблонов, полученные шаблоны можно использовать при проектировании веб-приложений, мобильных приложений, а также для прототипирования интерфейсов.

Публикации. По теме диссертации опубликована одна печатная работа на тему «Применение сверточной нейронной сети в задаче генерирования HTML-шаблонов по рукописным рисункам-макетам» в рецензируемом научном журнале «Вестник Череповецкого государственного университета».

Апробация результатов. Результаты магистерской диссертационной работы докладывались на: студенческой научной конференции: «VI Всероссийская научно-практическая конференция «Современные

информационные технологии. Теория и практика»»; на ЧГУ СНК-2023; ЧГУ СНК-2024.

Реализация результатов. Созданный в рамках диссертационной работы программный продукт развернут на предприятии ООО «АСТРОСОФТ ПРО», активно тестируется и используется сотрудниками компании для прототипирования пользовательского интерфейса в мобильных приложениях.

Структура и объем работы. Диссертационная работа состоит из введения, четырех разделов, заключения, списка литературы, включающего 41 наименование, и 5-ти приложений. Объем диссертационной работы – 160 страниц. В тексте диссертации содержатся 101 рисунок и 30 таблиц.

1. Характеристика проблемы генерирования HTML-шаблонов по рукописным рисункам-макетам

1.1. Анализ существующих моделей, методов и средств генерирования HTML-шаблонов по рукописным рисункам-макетам

Макет – это визуальная концепция основных элементов будущего веб-приложения. Как правило, процесс разработки макета осуществляется с использованием какой-либо программы для графического дизайна (например, Figma). На этом этапе формируются все важные компоненты дизайна веб-страницы в соответствии с выбранным стилем представления информации. Основной задачей на данном этапе является способность создать графические элементы таким образом, чтобы они быстро загружались и хорошо выглядели на экранах различных устройств, независимо от используемого браузера. Для увеличения эффективности процесса разработки веб-приложения, разработчики часто обращаются к использованию готовых дизайн-макетов, которых в настоящее время представлено в изобилии в сети Интернет и встроены в различные графические инструменты редактирования [1].

HTML (от английского HyperText Markup Language) — это язык гипертекстовой разметки текста. Он нужен, чтобы размещать на веб-странице элементы: текст, картинки, таблицы и видео.

HTML состоит из тегов — команд, которые указывают браузеру, как отображать помещённый в них текст. Это и есть элементы веб-страницы. У каждого тега есть имя, которое заключается в угловые скобки < и >. Теги бывают парные и непарные. Парные состоят из двух тегов — открывающего и закрывающего, а непарные — из одного. При этом непарный тег тоже можно закрывать, но это необязательно. У каждого тега есть атрибуты. С их помощью можно передавать элементам веб-страницы дополнительные данные: размеры, уникальный идентификатор элемента, ссылки на изображения и так далее [2].

Минимальный набор HTML тегов для разработки веб-страниц:

- тег для создания контейнера для размещения текста или другого контента: `<div>`;
- для отображения и форматирования текста на странице: `<p>`, `<i>`, `<b>`, `<font>`, `<br>`;
- для вставки изображений, видео и аудио: `<img>`, `<video>`, `<audio>`;
- для создания гиперссылок и навигации по сайту: `<a>` [3].

Поскольку HTML задает только структуру шаблона, то CSS предоставляет возможность стилизовать все структурные элементы, чтобы шаблон выглядел согласно задуманному дизайну. CSS – это технология каскадных таблиц стилей, она дает возможность точно определить характеристики практически всех элементов веб-страницы, что позволяет придать странице определенный внешний вид. С ее помощью можно настроить параметры шрифта, цвет текста и фона, выравнивание и многое другое [4].

HTML и CSS являются ключевыми инструментами в процессе создания веб-страниц или графического интерфейса мобильных приложений. Этот дуэт составляет базу знаний, необходимую для начинающего специалиста в среднем звене веб-технологий [5].

Для обеспечения правильного отображения и корректной интерпретации веб-страницы браузерами, важно поддерживать валидность HTML-кода. Валидаторы HTML могут использоваться для проверки синтаксиса и соответствия кода стандартам HTML. На рисунках ниже можно увидеть пример простейшего исходного кода HTML-шаблона, а также его структуру виде дерева, и как этот код в конечном итоге интерпретируется в браузере (рис.1-3).

```

1  <!DOCTYPE html>
2  <head>
3    <style>
4      body {
5        background-color: rgb(255, 196, 140);
6      }
7
8      .container {
9        text-align: center;
10     }
11   </style>
12 </head>
13 <body>
14   <div class="container">
15     <h1>Заголовок шаблона</h1>
16     <p>Параграф шаблона</p>
17     
18   </div>
19 </body>
20 </html>

```

Рис. 1. Пример простейшего исходного кода HTML-макета

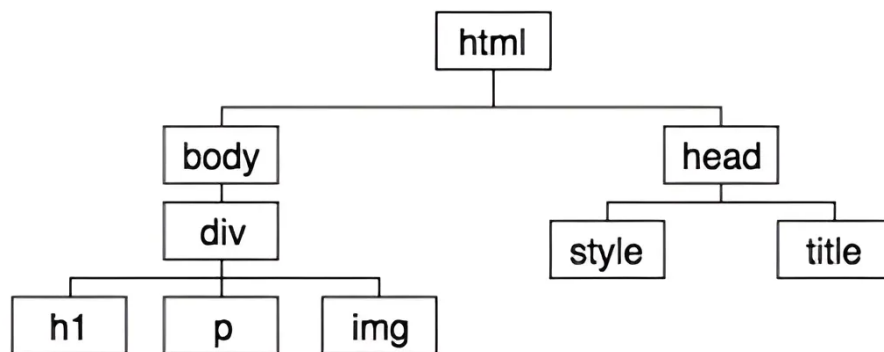


Рис. 2. Простейшая структура HTML-шаблона



Рис. 3. Интерпретация простейшего HTML-кода в браузере

HTML-код интерпретируется браузером и выдает пользователю графический интерфейс. Браузер – это приложение, созданное для просмотра веб-страниц. Каждый браузер обладает своими уникальными особенностями, поэтому для обеспечения универсальности кода необходимо тестировать и исправлять его с учетом всех этих особенностей. На сегодняшний день наибольшей популярностью пользуются браузеры: Chrome, Firefox, Yandex Browser [6].

Несмотря на кажущуюся простоту HTML-структуры, даже опытные разработчики часто совершают ошибки при верстке HTML-шаблонов, одной из таких является ошибка, связанная с фиксированной высотой блоков. Это касается блоков с текстовым содержимым. Разработчик устанавливает фиксированную высоту блока (`height`), и при размещении текста внутри блока может показаться, что все выполнено корректно. Однако, не осознавая этого, разработчик уже допустил ошибку, которая пока остается незамеченной. Однако, если добавить немного больше текста, то он начнет выходить за пределы блока. Чтобы избежать такого сценария, следует использовать свойство `min-height` в CSS [7]. Подобных подводных камней при верстке множество, поэтому важно использовать только современные подходы и проверять свой код в различных браузерах и на различных устройствах. Тем более, что некоторые теги в современных браузерах являются устаревшими, и в более новых версиях их поддержку начинают удалять. Примеры устаревших HTML-тегов:

- `<font></font>` — для форматирования текста;
- `<frameset><frame></frameset>` - теги для создания фреймов;
- `<big></big>` — для увеличения размера шрифта на единицу.

До внедрения каскадных таблиц стилей CSS тег `<font>` был единственным способом задания цвета, размера и шрифта текста. Удаление этого тега из спецификации вполне оправдано. Это упрощает модификацию внешнего вида веб-страницы и делает код более чистым и понятным. Также оправдан отказ от использования тегов фреймов, так как это мешало правильной индексации

страниц поисковыми системами. Однако некоторые теги все еще остаются очень полезными, а некоторые могут нести семантический смысл [8]. Чтобы не использовать устаревшие теги в своем коде, необходимо знакомиться с современной спецификацией HTML.

На текущий момент, задача генерирования HTML-шаблонов по рукописным рисункам-макетам решается с использованием нейросетей для распознавания объектов на макете, после чего применяется алгоритм, который по распознанным объектам формирует HTML-сетку.

Одной из основных причин сложности данной задачи является несовершенство технологий распознавания объектов на изображении. Нейросети могут иметь трудности с точным определением границ и геометрии объектов на рукописном макете, особенно если изображение не четкое или содержит сложные элементы. Это может привести к ошибкам в определении расположения и размеров объектов, что затрудняет формирование корректной HTML-сетки. Другой трудностью является разнообразие стилей рукописных макетов. Дизайнеры могут использовать различные способы обозначения элементов интерфейса, а это усложняет процесс распознавания и классификации объектов. Это требует разработки более сложных и адаптивных алгоритмов, способных обрабатывать различные стили и подходы к созданию рукописных макетов. Кроме того, важную роль играет эффективность и оптимизация создаваемых HTML-шаблонов. Генерация сетки должна учитывать не только распознанные объекты, но и оптимальное их расположение, адаптивность к разным разрешениям экранов и другие аспекты, что также усложняет процесс разработки алгоритмов.

На данный момент существует два зарубежных сервиса, решающих подобную задачу: Sketch2Code и ChatGPT-4.

1. Sketch2Code (Microsoft) (рис.4-7) – это веб-приложение, которое использует искусственный интеллект для преобразования рукописного дизайна пользовательского интерфейса в рабочий HTML-код. Для этого сначала пользователь указывает шаблоны проектирования. Далее приложение

анализирует рукописный чертеж или текст, определяет его структуру, а затем генерирует HTML-код в соответствии с обнаруженным макетом, включая обнаруженные элементы дизайна [9].

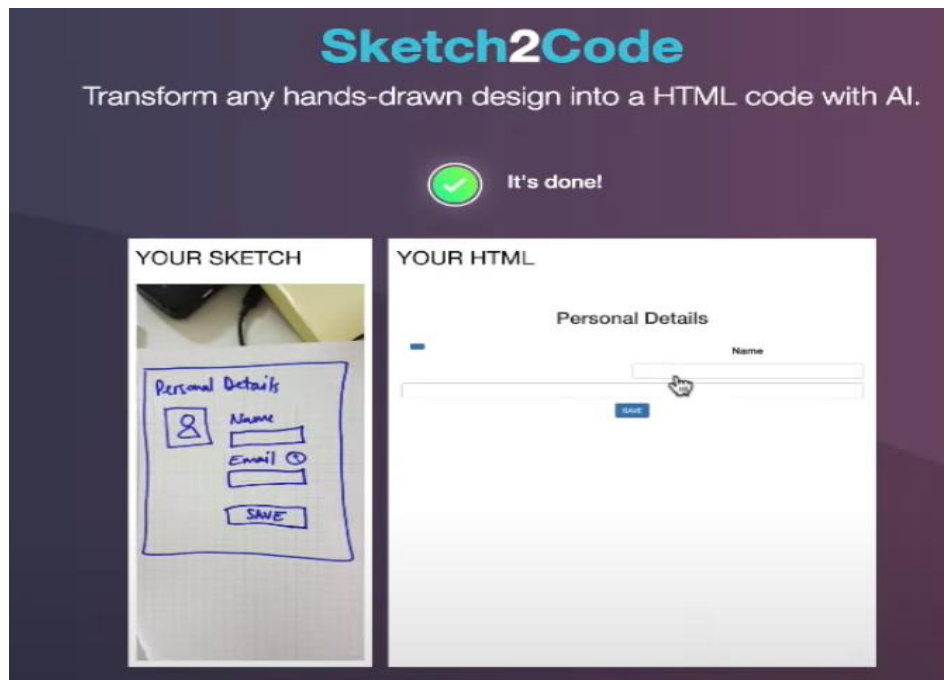


Рис. 4. Sketch2Code пример №1

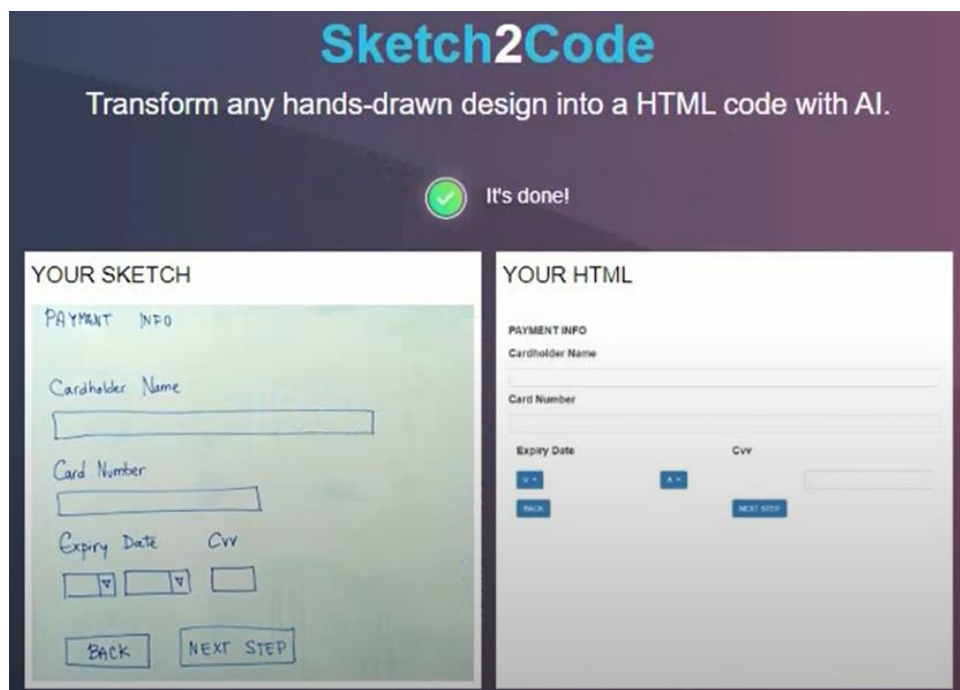


Рис. 5. Sketch2Code пример №2

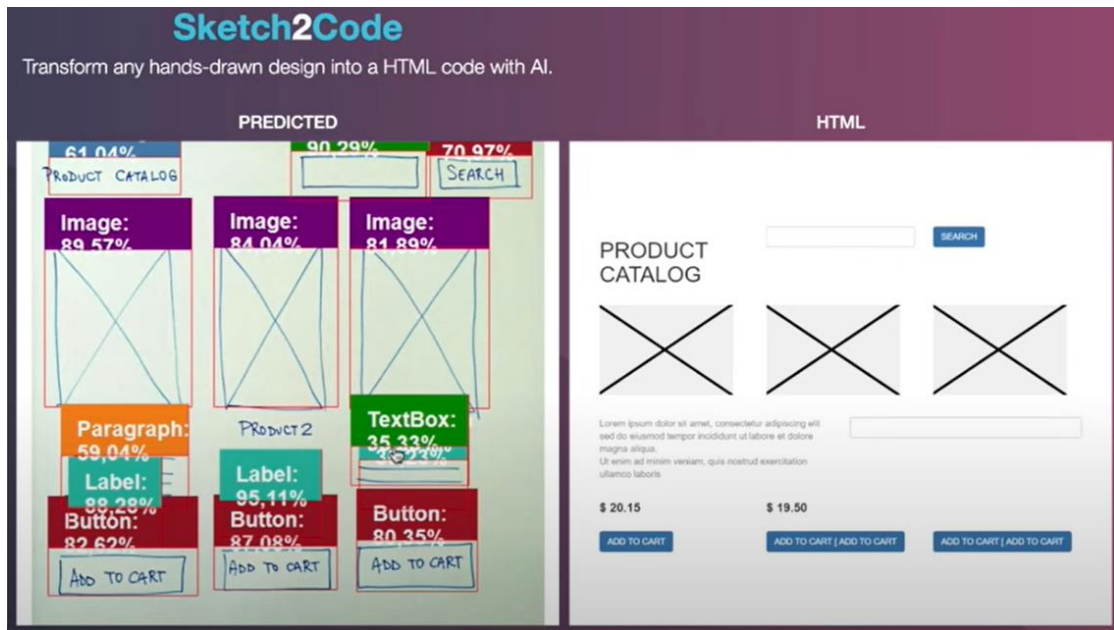


Рис. 6. Sketch2Code пример №3

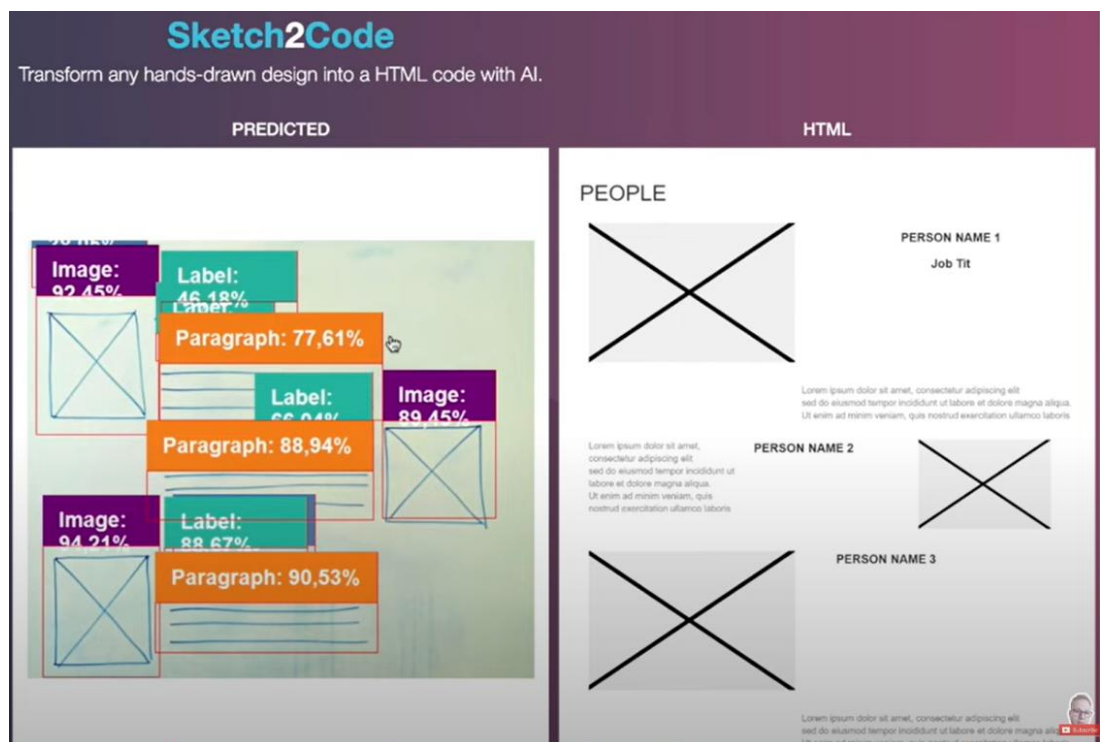


Рис. 7. Sketch2Code пример №4

Недостатки:

- поскольку нет четких правил для рисования макета – каждый пользователь программы может интерпретировать элементы HTML-

страницы по-разному, поэтому макет начинает «ломаться», что видно на рис.4;

- на момент написания ВКР данный сервис не работает в открытом доступе;
- нет возможности редактирования элементов макета в режиме реального времени без знания HTML;
- полученный HTML-код не является адаптивным, то есть при изменении ширины окна элементы не будут изменять размер и положение.

2. ChatGPT-4 (OpenAI) (рис.8) - мультимодальная большая языковая модель, созданная OpenAI, четвёртая в серии GPT. Она была выпущена 14 марта 2023 года и доступна для пользователей ChatGPT Plus [10].

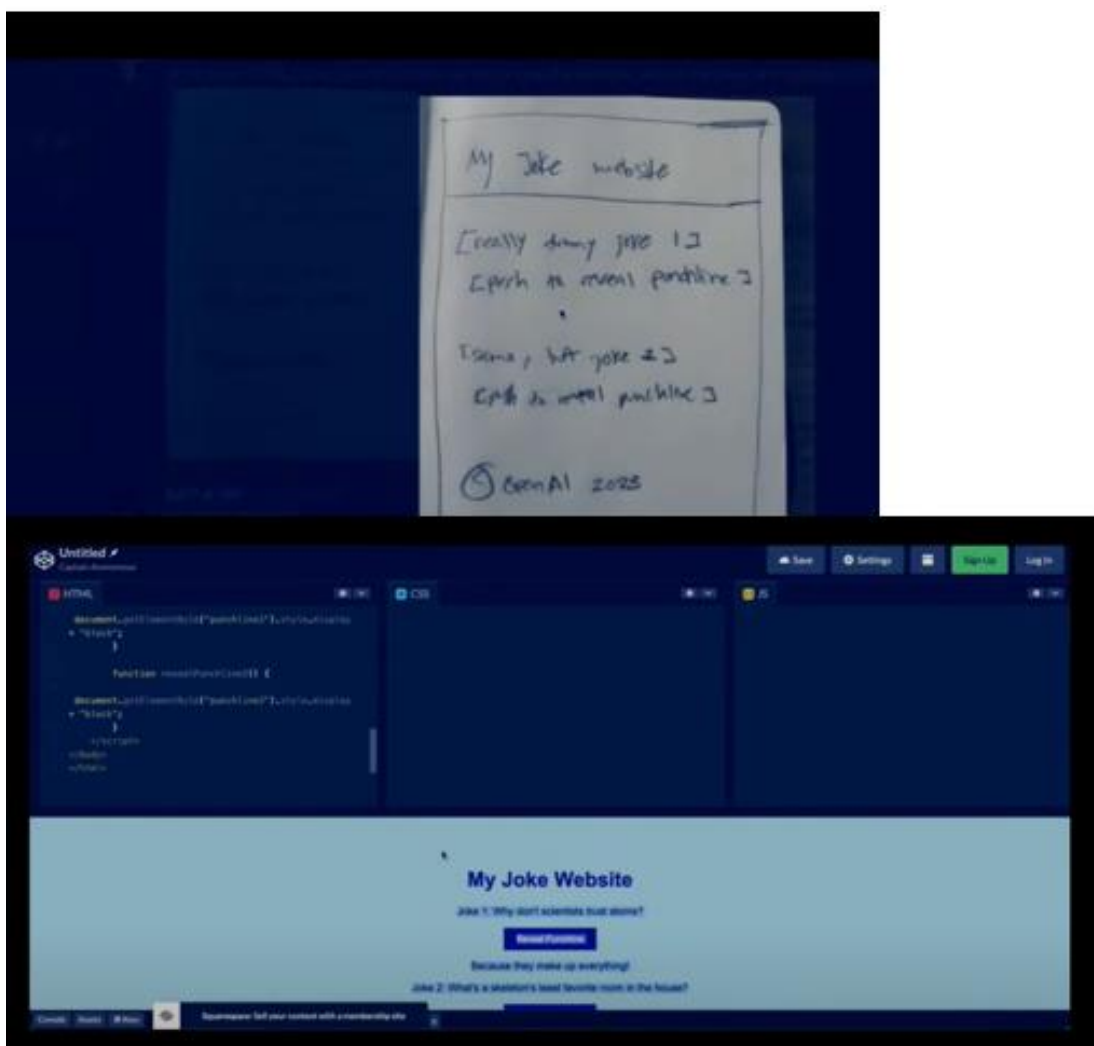


Рис. 8. ChatGPT-4

Недостатки:

- не учитывается положение элементов на макете (получившуюся HTML-сетку необходимо вручную дополнительно регулировать, без знаний HTML и CSS это сделать невозможно);
- отсутствие UI-инструментов для редактирования полученного макета (можно только использовать текстовые запросы, или корректировать макет с помощью новых изображений);
- полученный HTML-код не является адаптивным, то есть при изменении ширины окна элементы не будут изменять размер и положение.

Таким образом, изучив существующие решения данной задачи, можно сделать вывод: на текущий момент нет ни одного решения, которое бы формировало сетку в соответствии с макетом, элементы располагаются не так, как рисовал пользователь (в некоторых случаях пропадает центрирование или элементы идут вразнобой), при этом не формируется автоматически адаптивная версия, которую можно было бы просмотреть на более узких экранах.

1.2. Описание предметной области генерирования HTML-шаблонов по рукописным рисункам-макетам

В современном мире возрастает спрос на разработку кроссплатформенных приложений. Современные приложения могут разрабатываться с помощью популярных фреймворков: React Native или Ionic. Для заказчика это открывает следующие преимущества:

1. Меньше необходимо затрачивать бюджета на разработку приложений под разные платформы (Android, iOS, Web);
2. Меньшая трата времени на разработку. React Native и Ionic используют HTML + CSS для создания интерфейсов, поэтому приходится тратить время на поиск нужного шаблона или написания кода с нуля.

Генерация шаблонов по рукописным рисункам-макетам позволит сформировать шаблон нужного вида и далее работать с ним, что также сократит время на разработку кроссплатформенного приложения.

Рукописный рисунок-макет является графическим представлением концепции дизайна или шаблона веб-страницы, который создается вручную с использованием ручного инструмента, такого как карандаш, ручка или маркер, или компьютерная мышь (в случае с цифровым рисунком). Такие рисунки могут включать в себя элементы визуального оформления, расположение контента, и другие детали дизайна. Рукописные макеты часто используются для визуализации идеи дизайнера или разработчика в начальной стадии проектирования веб-сайта. Они помогают перевести концепцию дизайна из ума на бумагу и дать представление о том, как будет выглядеть и функционировать конечный продукт. Такие рисунки могут быть разнообразными: от простых эскизов и макетов интерфейсов до детальных чертежей. Для создания рукописных макетов используются различные типы бумаги, карандаши, ручки, маркеры, а также графические планшеты и различные программы для создания цифровых эскизов. В целом, рукописный рисунок можно изобразить на многих поверхностях:

- магнитная доска;
- бумага (обычная, цветная, клетчатая, в клетку, в линейку);
- дневник или блокнот;
- скетчбук;
- холст;
- планшет для рисования.

На рис.9-12 изображены примеры поверхностей, на которых можно изобразить рукописный рисунок.

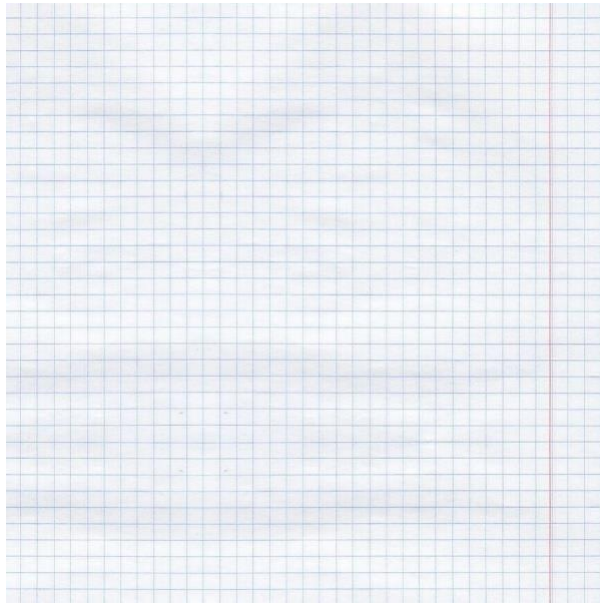


Рис. 9. Тетрадь в клетку

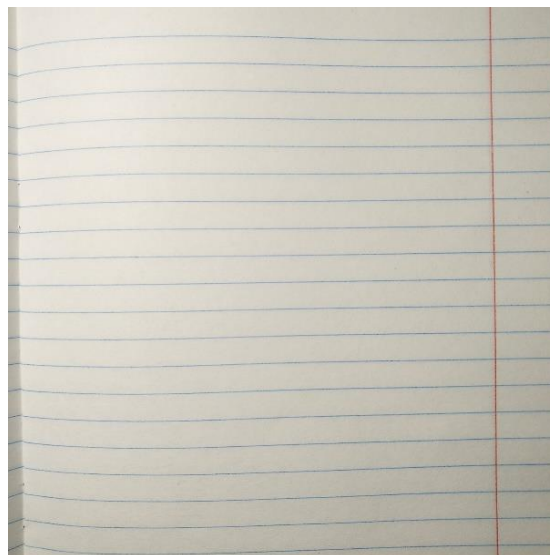


Рис. 10. Тетрадь в линейку



Рис. 11. Чистый лист А4

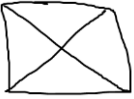
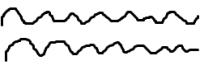
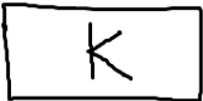


Рис. 12. Магнитная доска

Поскольку HTML-шаблон может состоять из однотипных объектов, то необходимо определиться с условным отображением каждого класса объекта, для их дальнейшего распознавания. В табл.1 можно увидеть условные обозначения различных классов, которые будет распознавать программа с рукописных рисунков и формировать из них HTML-код:

Таблица 1

Условные обозначения классов объектов, распознаваемые с рукописных рисунков

№	Наименование	Описание	Отображение
1	2	3	4
1	Контейнер	Класс, описывающий прямоугольную или квадратную геометрию, ей можно задать произвольный фон в виде определённого цвета или картинки	
2	Изображение	Пользовательское изображение, может быть квадратным или прямоугольным	
3	Текст	Пользовательский многострочный текст без форматирования	
4	Кнопка	Кнопка для переходов между различными шаблонами, кликабельна, можно прикрепить произвольную ссылку или действие	

Продолжение табл.1

1	2	3	4
5	Ссылка	Ссылка на определенный сайт или другой шаблон, отличается от класса «Кнопка» тем, что не имеет фонового цвета по умолчанию, а также отсутствие возможности прикрепления пользовательской обработки клика	
6	Заголовок	Аналогичен классу «Текст», но по умолчанию имеет жирное выделение и более крупный размер шрифта	
7	Пользовательский HTML-код	Класс отвечает за возможность добавления в шаблон пользовательского фрагмента HTML-кода	

В дальнейшем могут быть добавлены дополнительные классы, такие как: «Слайдер» (для взаимодействия с набором изображений), «Формы» (включающие в себя элементы для ввода текста, чисел, выбора даты), «Видео» (для прикрепления ссылки на видео и его воспроизведения в шаблоне), и многие другие.

Примеры допустимых макетов в соответствии с описанными классами указаны на рис.13-14.

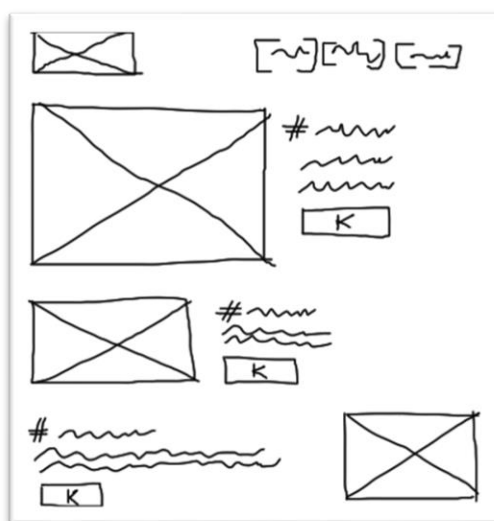


Рис. 13. Пример допустимого макета №1

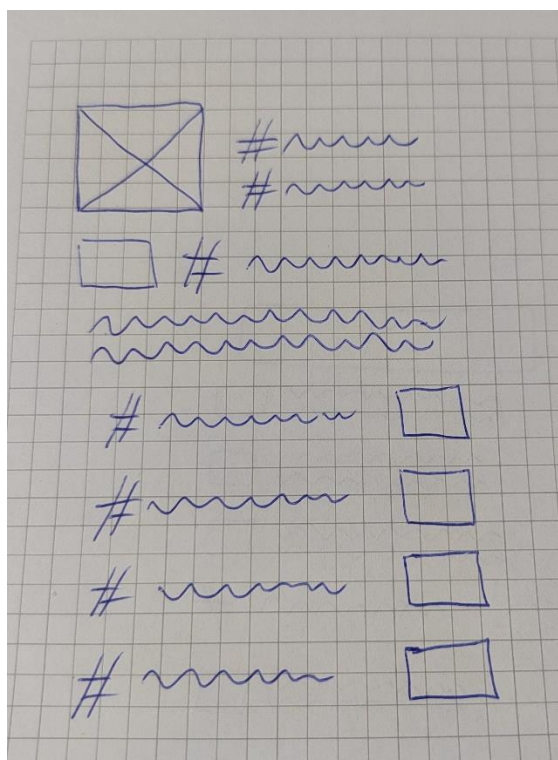


Рис. 14. Пример допустимого макета №2

Что касается требований к рисованию объектов: размеры нарисованных объектов не должны превышать размеров поверхности, на которой они рисуются; контуры нарисованных объектов должны быть четко различаемы; нарисованный объект должен быть нарисован в соответствии с условным обозначением и относиться к какому-либо из классов.

1.3. Постановка задачи, разработка требований к информационному и аппаратно-программному обеспечению генерирования HTML-шаблонов по рукописным рисункам-макетам

Постановка задачи – это процесс определения конкретной цели, которую необходимо достичь, и определение способов ее достижения. Этот процесс включает в себя определение проблемы, выявление целей и ожидаемых результатов, а также определение всех необходимых шагов для ее выполнения.

Программный продукт должен обеспечить возможность генерации HTML-шаблонов по рукописным рисункам-макетам.

Пользовательский интерфейс должен быть интуитивно понятным, должна быть возможность выгрузки HTML-кода для его последующего применения или редактирования.

Программный продукт не должен использовать более 4Гб видеопамати при обучении модели, а также при работе на реальных изображениях.

По требованиям заказчика, реализуемое программное обеспечение генератора HTML-шаблонов по рукописным рисункам-макетам должно выполнять следующие функции:

- скорость обработки изображений: не более 10-ти секунд на изображение;
- возможность детектирования объектов с рукописных рисунков-макетов, как минимум, необходимо детектировать следующие классы — «контейнер», «кнопка», «изображение», «заголовок», «текст», «ссылка», «пользовательский HTML»;
- из объектов, которые распознаны нейросетью необходимо формировать HTML-сетку, сетка должна формировать строки и столбцы (в виде иерархической структуры, используя тэг «div» и прикрепленный к нему атрибут класса «row» или «column»), чтобы можно было в случае необходимости интегрировать сетку с CSS-фреймворками Bootstrap или Materialize;
- в пользовательском интерфейсе клиентского приложения предусмотреть возможность выбора заготовленных шаблонов по категориям;
- в пользовательском интерфейсе предусмотреть возможность добавления новых блоков, а также их отображения, блоком считать фрагмент HTML-кода, на странице может быть множество блоков;
- элементы внутри блока должны быть редактируемы, то есть нужны инструменты для редактирования типовых элементов —

редактированию подлежит текст, стилевые свойства элемента (цвет, цвет фона, отступы, ширина, высота, поворот, скругление краев) (Прил.1).

Цель внедрения программного обеспечения – сокращение трат времени на написание HTML-кода при разработке кроссплатформенных приложений (Android, Web, iOS, Desktop).

Компьютер, на котором будет выполняться эксплуатирование программного обеспечения должен обладать следующими минимальными требованиями:

- объем оперативной памяти не менее 4 Гб;
- монитор с разрешением не менее 1280x720;
- операционная система семейства Windows (Windows 7, Windows 8, Windows 10, Windows 11);
- твердотельный накопитель объемом не менее 512 Гб;
- процессор Intel Core I5, не ниже 6-го поколения, тактовая частота 2.1 ГГц;
- установленная NodeJS и NPM (менеджер пакетов);
- наличие установленного браузера современной версии (Google Chrome, Яндекс Браузер, Opera, Firefox).

1.4. Выводы по первому разделу

В данном разделе представлены основные выводы по анализу предметной области генерирования HTML-шаблонов по рукописным рисункам-макетам. Были проанализированы существующие системы и выделены основные преимущества и недостатки таких систем.

В текущем разделе были сформулированы основные задачи и требования к программному решению, требования к аппаратно-программным характеристикам. В результате анализа существующих решений было установлено, что они имеют ряд недостатков, которые возможно устранить.

2. Математическое и алгоритмическое обеспечение генерирования HTML-шаблонов по рукописным-макетам

2.1. Описание математической модели детектирования объектов с рукописных рисунков-макетов для генерирования HTML-шаблонов

В первую очередь необходимо определять объекты на изображении с рукописного рисунка. Для решения поставленной задачи необходимо воспользоваться алгоритмами компьютерного зрения. Компьютерное зрение – это область исследований, которая занимается разработкой методов и технологий для компьютерной обработки изображений, полученных из реального мира. Целью этой области является создание систем, способных распознавать и анализировать объекты на изображениях без прямого участия человека.

Задачи компьютерного зрения в общем случае можно разделить на четыре категории [11].

Категории задач компьютерного зрения:

- Классификация – задача состоит в том, чтобы классифицировать изображение целиком. Для этого на изображении выделяются ключевые области и по ним происходит классификация, например с помощью решающих деревьев, или сверточных нейронных сетей, то есть определяется принадлежность изображения к какому-либо из заранее определенных классов;
- Локализация – необходима для определения местоположения объектов. Обычно найденные объекты выделяются в ограничивающие прямоугольники (bounding boxes);
- Детектирование – включает в себя одновременно задачу классификации и локализации одного или нескольких объектов на изображении;
- Сегментация – это отнесение пикселей изображения к определенной категории, например кошке, собаке, или заднему фону. В отличие от

детектирования, подсвечивается лишь те пиксели, которые принадлежат объекту (нет необходимости в поиске ограничительных рамок).

В случае с генерацией HTML-шаблонов по рукописным рисункам-макетам решается задача детектирования (рис.15), поскольку необходимо определить положение объектов на макете и классифицировать эти объекты. Безусловно, можно использовать подход, связанный с сегментацией, но он будет избыточен конкретно в этом случае, в работе достаточно получить координаты ограничительных рамок и распознанный класс.



Рис. 15. Категории задач компьютерного зрения

Исходя из поставленной задачи сформируем несколько критериев для поиска необходимого подхода для детектирования объектов:

- подход должен учитывать, что на изображении могут располагаться множество объектов разных классов;
- должен учитываться размер объектов (их ширина и высота), а также их положение;
- выбранный подход должен быть устойчив к изменениям освещения и качеству изображения.

Задачу можно решить двумя подходами: использование классических методов компьютерного зрения; использование алгоритмов распознавания на основе глубокого обучения. Классические методы включают в себя поиск

ключевых точек, выделение границ, сегментацию, геометрические преобразования, такие методы хорошо подходят для более простых и детерминированных задач с точки зрения внешних факторов: освещение и расстояние до объекта съемки, небольшая вариация форм объекта. В свою очередь, распознавание на основе глубокого обучения более устойчиво к ложным срабатываниям при изменении освещенности, размера или ракурса объекта, позволяют строить более совершенные и отказоустойчивые системы компьютерного зрения [13].

Для детектирования объектов на изображении в наше время одним из наиболее эффективных показывает себя сверточная нейронная сеть. Сверточная нейронная сеть — специальная архитектура искусственных нейронных сетей, предложенная Яном Лекуном в 1988 году и нацеленная на эффективное распознавание образов, входит в состав технологий глубокого обучения. Архитектура CNN аналогична структуре связей нейронов в мозгу человека, учёные черпали вдохновение в организации зрительной коры головного мозга. Отдельные нейроны реагируют на стимулы только в некоторой области поля зрения, также известного как перцептивное поле. Множество перцептивных полей перекрывается, полностью покрывая поле зрения CNN [12] (рис.16).

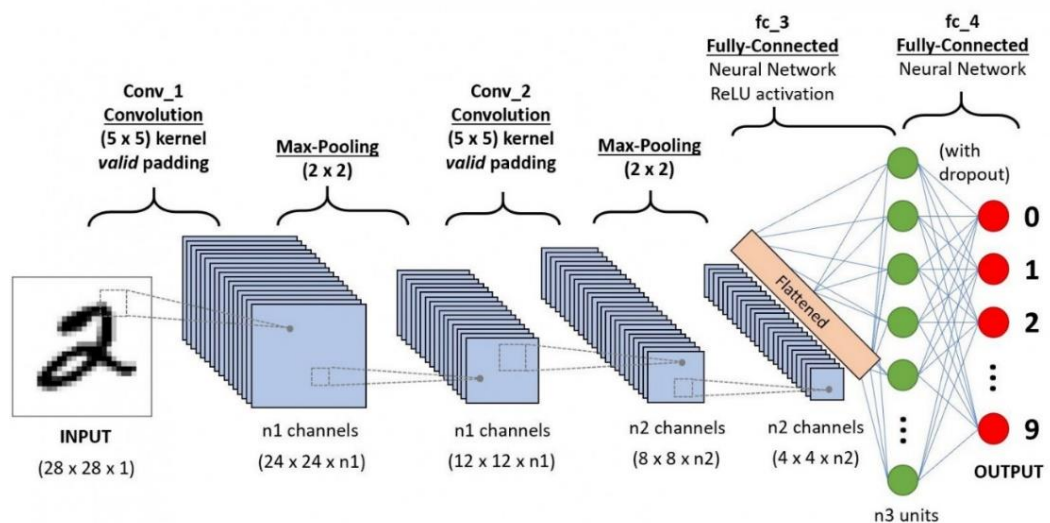


Рис. 16. Обобщенная структура сверточной нейронной сети

Поэтому, для решения поставленной задачи лучше всего подойдут сверточные нейронные сети, которые являются технологиями глубокого обучения.

Для первичного понимания задач и структуры разрабатываемой модели будет приведена модель чёрного ящика данной системы. Модель чёрного ящика представлена на рис.17.

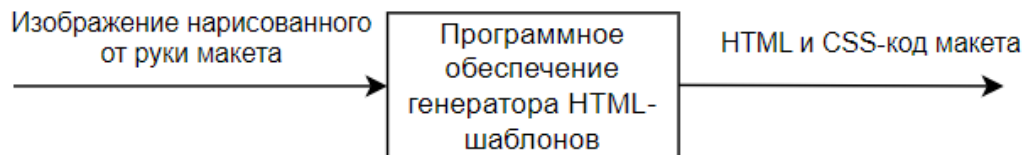


Рис. 17. Модель черного ящика

Как можно наблюдать из рис.17, на вход системы подаётся изображение нарисованного от руки макета, которое в результате работы программы в конечном итоге будет преобразовано в HTML и CSS-код, который можно запустить в любом современном браузере (в том числе и мобильном).

Поскольку ранее была выбрана модель сверточной нейронной сети, то ниже пойдет более подробное её описание, а также выбор конкретной архитектуры, которая будет удовлетворять решаемой задаче. Конечно, можно использовать и полносвязные нейронные сети при решении подобной задачи, но такие сети будут хуже обобщать информацию с изображений, а также потребуются большее использование количества оперативной памяти.

В общем случае, архитектуры сверточных нейросетей можно разделить на две группы:

- одноэтапные – такие алгоритмы используют сверточную нейронную сеть, чтобы сразу определить местоположение объекта и его класс без дополнительной обработки изображения. Примерами таких алгоритмов являются YOLO (You Only Look Once), SSD (Single Shot Multibox Detector) и их вариации;

- двухэтапные - относятся алгоритмы основанные на разбиении изображения на регионы. Например R-CNN, Fast R-CNN, Faster R-CNN и другие их разновидности [14]. Двухэтапность этих алгоритмов заключается в том, что перед этапом обнаружения и классификации объекта нейросеть должна провести этап анализа изображения и выделения на нем участков с возможным присутствием искомым объектов.

Очевидно, что двухэтапные алгоритмы в большинстве своем работают медленнее одноэтапных, поскольку выполняют дополнительный этап обработки изображения, поэтому остановимся на выборе одноэтапных моделей. Одной из самых известных таких моделей является архитектура сверточной нейронной сети YOLO (You Only Look Once — смотри только один раз).

Далее пойдёт общее описание архитектуры сверточной нейросети и конкретно о YOLO: она имеет несколько поколений, включая YOLO, YOLOv2, YOLOv3 и YOLO9000 — модель, основанная на YOLOv2, способная различать 9000 классов объектов, а также более новые версии. Основой работы таких моделей является архитектура глубоких сверточных нейронных сетей, использующая операции поэлементной свертки для извлечения более глубоких характеристик изображений [15].

Количество нейронов входного слоя должно соответствовать количеству пикселей в изображении для каждого цвета. Если изображение имеет размер 32×32 пикселя в трех цветах (RGB), то входной слой должен содержать $32 \times 32 \times 3 = 3072$ нейрона. Сверточный слой вычисляет выходной объем путем поэлементного умножения фильтров и областей изображения. В этой операции важнейшее значение играют применяемые фильтры, которые позволяют выявлять те или иные ключевые характеристики на изображении, например, фильтры границ. Для произвольного количества каналов в изображении, результат работы сверточного слоя описывается следующей формулой:

$$[H]_{i,j,d} = \sum_a^\Delta = -\Delta \sum_b^\Delta = -\Delta \sum_c^\Delta = 1[V]_{a,b,c,d}[X]_{i+a,j+b,c} \quad , \quad (1)$$

где d индексирует выходные каналы в результирующем изображении H , а (i,j) – координаты каждого пикселя на изображении, k – количество каналов в изображении, (a,b) - координаты элементов фильтра.

В общем виде архитектура сверточной нейронной сети представлена на рис.18.

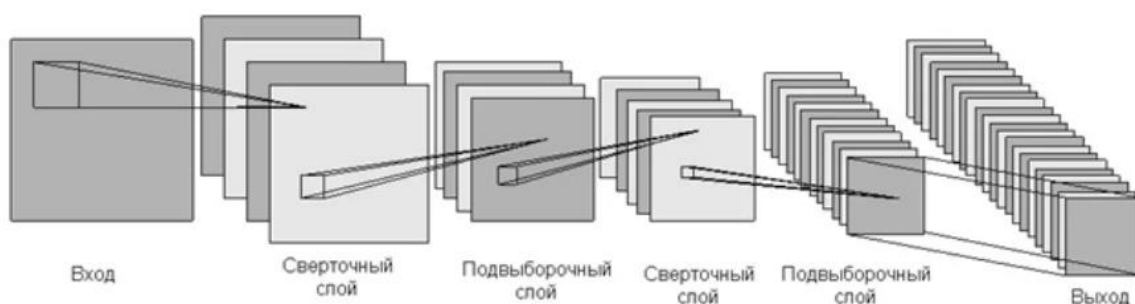


Рис. 18. Архитектура сверточной нейросети в общем виде

Все слои СНС представляют собой набор плоскостей. Нейроны одной плоскости имеют одинаковые весовые коэффициенты, ведущие ко всем локальным участкам предыдущего слоя. Каждый нейрон слоя получает входы от некоторой области предыдущего слоя, т.е. входное изображение предыдущего слоя получает входное изображение предыдущего слоя сканируется небольшим окном и пропускается сквозь набор весовых коэффициентов, а результат отображается на соответствующий нейрон текущего слоя. Таким образом, набор плоскостей представляется собой карты характеристик, и каждая плоскость находит конкретные участки изображения в любом месте предыдущего слоя [16].

В результате анализа существующей литературы, были изучены наиболее приоритетные архитектуры для решения поставленной задачи – «Faster R-CNN» и «YOLO». Архитектура Faster R-CNN является двухэтапной архитектурой, которая на первом этапе обнаруживает области, в которых

могут находиться объекты, а на втором этапе – определяет класс объекта и уточняет его ограничивающие рамки. В противопоставление, архитектура «YOLO» является одноэтапной архитектурой, которая основывается на моделях регрессии и просматривает изображением целиком в один этап. Так как в данной архитектуре содержится только один этап, то она является наиболее быстрой, нежели архитектура Faster R-CNN.

Модели семейства YOLO основаны на использовании глубоких сверточных нейронных сетей, которые используют операции свертки для извлечения более высокоуровневых образных признаков изображения. В рамках выполнения задачи было решено остановиться на выборе третьего поколения модели YOLO по следующей причине: данная модель относительно проста и легка для внедрения даже непрофессиональные пользователи могут использовать YOLOv3 для создания систем обнаружения объектов без необходимости обширных знаний в области машинного обучения. Кроме того, YOLOv3 имеет хорошую обобщающую способность и демонстрирует устойчивое обнаружение объектов при различных условиях освещения, изменениях масштаба и перекрытиях объектов. YOLOv3 по-прежнему остается привлекательным и эффективным выбором для решения задач обнаружения объектов в реальном времени.

Архитектура модели YOLOv3 представлена на рис.19.

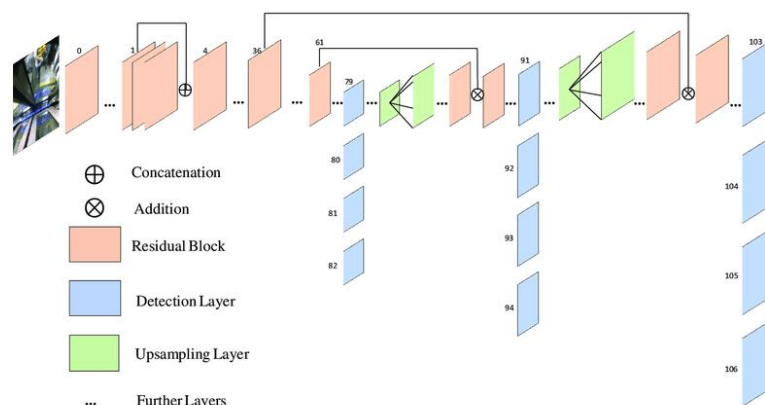


Рис. 19. Архитектура модели YOLOv3

Алгоритм работы YOLOv3 состоит из следующих шагов:

1. Подготовка изображения: Входное изображение подвергается предобработке, включая изменение размера, нормализацию и конвертацию в формат, который совместим с моделью YOLOv3, в нашей модификации входной тензор будет размером 448x448x1, будет использоваться один цветовой канал для экономии видеопамяти и вычислительных ресурсов компьютера, так как нет необходимости учитывать цвет с рукописных рисунков;
2. Формирование карты признаков на основе предобработанного изображения с предыдущего шага. Для этого исходное изображение подаётся на вход сверточной нейронной сети CNN, которая состоит из нескольких сверточных слоев, батч-нормализации, активаций и слоев объединения (pooling);
3. Определение прямоугольников с предполагаемыми объектами: на выходе нейронной сети получается сетка клеток, в каждой из которых предсказываются прямоугольники, в которых находятся объекты, а также их класс и уверенность (confidence) в правильности предсказания;
4. Отсечение ненадежных результатов: Полученные прямоугольники фильтруются на основе уверенности в их правильности предсказания. Те прямоугольники, уверенность в которых ниже заданного порога, отсекаются;
5. Подавление немаксимальных результатов: для избегания множественных детекций одного и того же объекта, прямоугольники с высокой уверенностью объединяются и убираются дубликаты (этот алгоритм известен под названием Non-maximum suppression).
6. На выходе будет получена модель $14*14*[x, y, w, h, p, (c_1, c_2, \dots, c_n)]$. Здесь x и y соответствуют координатам центра рамки, w и h — высота и ширина по отношению ко всему изображению, p — вероятность правильно детектирования класса, складывающегося из суммы всех вероятностей, c_n — вероятность n -го класса.

Для извлечения карты признаков с изображений будет использоваться архитектура семейства ResNet – это мощные глубокие нейронные сети, были предложены Хэ Каймингом и другими выдающимися учеными в лаборатории Microsoft в 2015 году. ResNet представляет собой первый случай использования сетевой структуры с более чем 1000 уровнями. Особенностью является введение остаточного модуля, который использует пакетную нормализацию для ускорения обучения, отказываясь от первоначального метода отсева. Это существенно смягчает недостатки сети VGG [17].

До разработки ResNets, наращивание количества слоев для повышения глубины нейронной сети рано или поздно вызывало ограничение или даже быстрое ухудшение производительности сети. Это происходило из-за проблемы исчезающих градиентов, которая возникает из-за того, что множество функций активации (таких, как сигмоида) сокращают большое входное пространство до гораздо меньшего выходного (например, от 0 до 1 для сигмоиды). Поскольку большие изменения входа преобразуются в намного меньшие изменения выхода, производная, естественно, становится намного меньшей.

Ключевая идея, лежащая за ResNet – это введение остаточных блоков (residual blocks), которые содержат "обходную связь идентичности" (identity shortcut connection), обходящую один или большее количество слоев (рис.20).

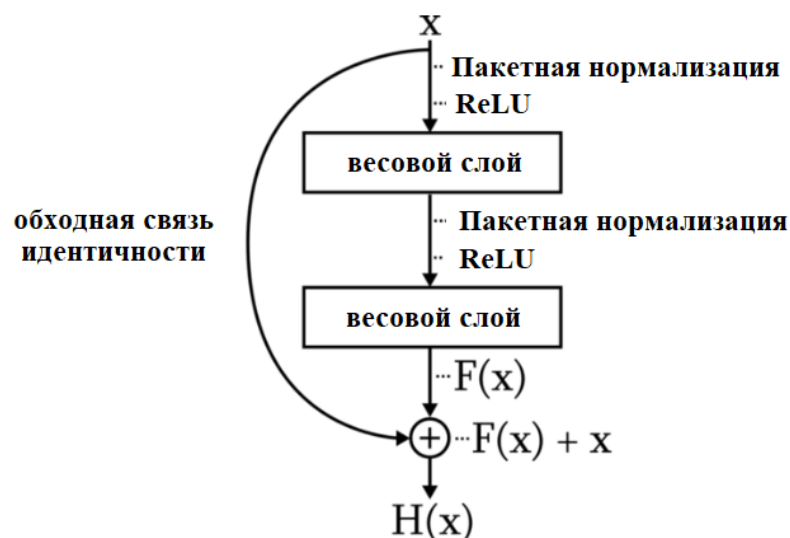


Рис. 20. Структурный элемент ResNet (остаточный модуль)

Формально, структурный блок определяется по формуле:

$$y = F(x, \{W_i\}) + x, \quad (2)$$

где x – это входной вектор, y – выходной вектор слоёв и $F(x, \{W_i\})$ представляет остаточное отображение для изучения. Основным преимуществом этой конфигурации является то, что вычислительная сложность поэлементного сложения незначительна. Размеры x и F должны быть равны, однако, если это не так, то можно выполнить линейную проекцию x . Функция F может представлять несколько полносвязных или сверточных слоев, в последнем случае поэлементное сложение выполняется канал за каналом.

CNN начинается со входного ствола, за которым следуют четыре блока, построенные по одному образцу. Ключевые компоненты ResNet50 показаны на рис.21 [18]:

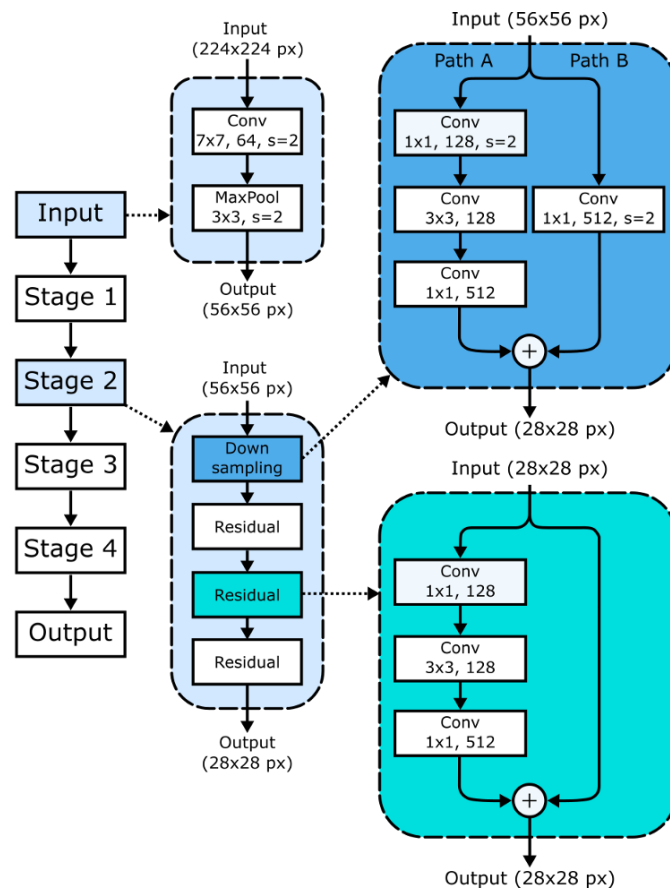


Рис. 21. Ключевые компоненты ResNet 50

Для того, чтобы нейронная сеть обучалась необходимо определиться с функцией потерь. Функция потерь модели YOLOv3 представляет собой совокупность математических операций, используемых для оценки различий между предсказанными и фактическими значениями. В контексте модели YOLOv3, функция потерь используется для определения отклонения прогнозов объектов (класс, координаты и размеры) от их фактического положения в изображении. Формула функции потерь модели YOLOv3 включает в себя различные компоненты, такие как среднеквадратичное отклонение (MSE) для координат и размеров, а также функция потерь перекрестной энтропии (cross-entropy loss) для оценки вероятности принадлежности объекта к определенному классу. Все эти компоненты объединяются в одно общее значение функции потерь, которое оптимизируется во время обучения модели с целью минимизации ошибок предсказаний [19]. Формула функции потерь указана на рис.22.

$$\begin{aligned}
& \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\
& + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{obj} [(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{obj} (C_i - \hat{C}_i)^2 \\
& + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{noobj} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} 1_i^{obj} \sum_{c \in classes} (p_i(c) - \hat{p}_i(c))
\end{aligned}$$

Рис. 22. Формула функции потерь модели YOLOv3

Для измерения точности работы детектирующего алгоритма используется метрика Intersection Over Union (IoU) – это метрика степени пересечения между двумя ограничивающими рамками (рис.23) [20].

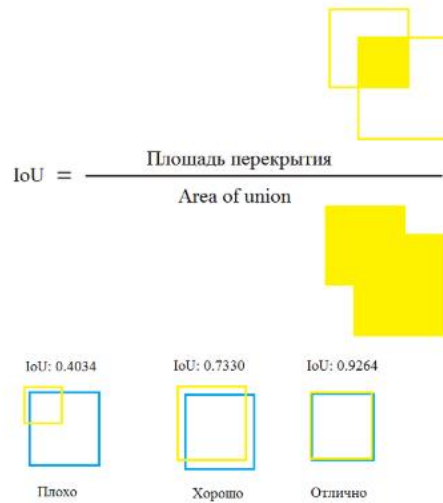


Рис. 23. Intersection Over Union, вычисление для различных ограничительных рамок (bounding boxes)

Возможна ситуация, когда один и тот же объект с высокой вероятностью обнаруживается несколькими ограничительными рамками (рис.24).

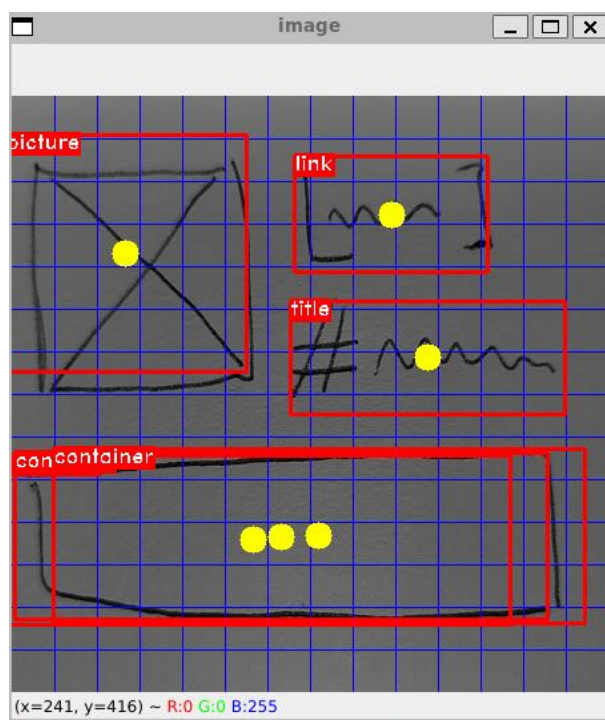


Рис. 24. Повторное обнаружение объекта с классом «container»

Для того, чтобы устранить данную проблему, существует алгоритм Non-Maximum Suppression (NMS). Он заключается в сравнении ограничительных рамок с наибольшим значением вероятности нахождения в нем объекта класса со всеми остальными ограничительными рамками. Сравнение происходит с помощью функции IoU, которая отбрасывает ограничительные рамки, пересекающиеся со сравниваемыми рамками больше, чем на 50%, этот алгоритм повторяется для всех последующих ненулевых значений, последовательно для каждого класса. В результате, нейросеть перестает обнаруживать один и тот же объект повторно [21].

2.2. Адаптация и тестирование математической модели

Адаптация математической модели — это процесс изменения параметров модели таким образом, чтобы достичь наилучших результатов. При обучении нейронной сети также происходит настройка определенного набора

параметров. В табл.2 представлены параметры, которые были использованы при обучении сверточной нейронной сети для обнаружения объектов на изображении рукописного рисунка-макета.

Таблица 2

Параметры обучения нейронной сети

Название параметра	Описание	Значение
1	2	3
gridSize	Размер сетки, например, 14x14	[14, 14]
anchorsCount	Количество якорей на одну ячейку сетки (т.е. количество ограничительных рамок)	5
optimizer	Оптимизатор (метод повышения производительности модели)	Adam
numClasses	Количество распознаваемых классов	7
lambdaNoobj	Коэффициент функции потерь для ячеек, в которых ни один объект не обнаружен	0.5
lambdaCoord	Коэффициент функции потерь для усиления значимости координат в формуле	20
learningRate	Размер шага на каждой итерации обучения, при уменьшении размера шага увеличивается время обучения, но может возрасти точность	0.0001

Продолжение табл.2

1	2	3
batchSize	Количество одновременно обрабатываемых изображений, при увеличении размера будет больше потребляться видеопамяти, но при этом модель будет быстрее обучаться	8
epochs	Количество эпох для обучения	100

При указанных выше параметрах нейронная сеть дает наилучшие результаты по метрике «mean average precision» на тестовых изображениях. Данные параметры были получены экспериментальным путем, не исключено, что можно попробовать подобрать параметры лучше.

Поскольку нейронные сети имеют склонность к переобучению при большом количестве эпох – был предусмотрен механизм ранней остановки обучения. То есть, если в течение нескольких эпох (в нашем случае пяти) значение функции потерь на тестовых данных не уменьшается, то в таком случае обучение останавливаем, не дожидаясь окончания последней эпохи, при этом лучшие веса нейросети сохраняются на жестком диске.

Для того, чтобы не рисовать от руки множество макетов при подготовке тренировочных изображений – было принято решение написать утилитарную программу, которая по заданным наборам изображений объектов классов будет формировать одно изображение с неким отступом между этими объектами, таким образом, можно подготовить тренировочный набор изображений, алгоритм данной утилитарной программы более подробно описан в главе 2.5. Приведем формальное краткое описание работы данной программы при подготовке тренировочных изображений:

- для начала выбирается случайная директория, содержащая разные вариации заданного класса, из неё случайным образом выбирается изображение (рис.25);

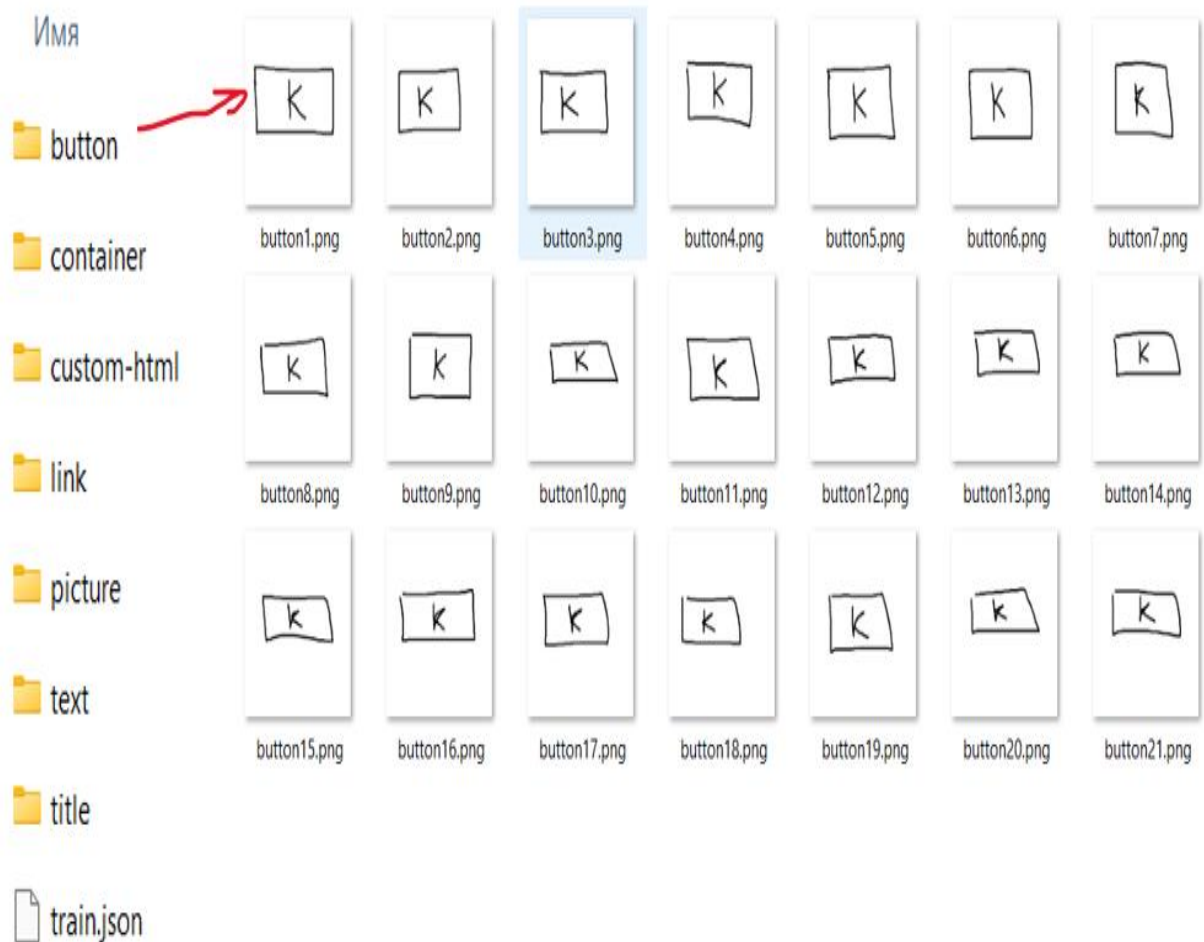


Рис. 25. Выбор случайной директории со случайным изображением

- далее набор изображений с предыдущего шага помещается с отступом на итоговое изображение, при этом изображения объектов проходят случайную аугментацию (повороты, увеличение/уменьшение размеров) для большего разнообразия тренировочных данных;
- на итоговое изображение добавляется случайный фон или не добавляется вовсе, при этом применяем случайные шумы или

ухудшение качества изображения для того, чтобы симитировать, будто изображение получено с плохой камеры (рис.26-27).

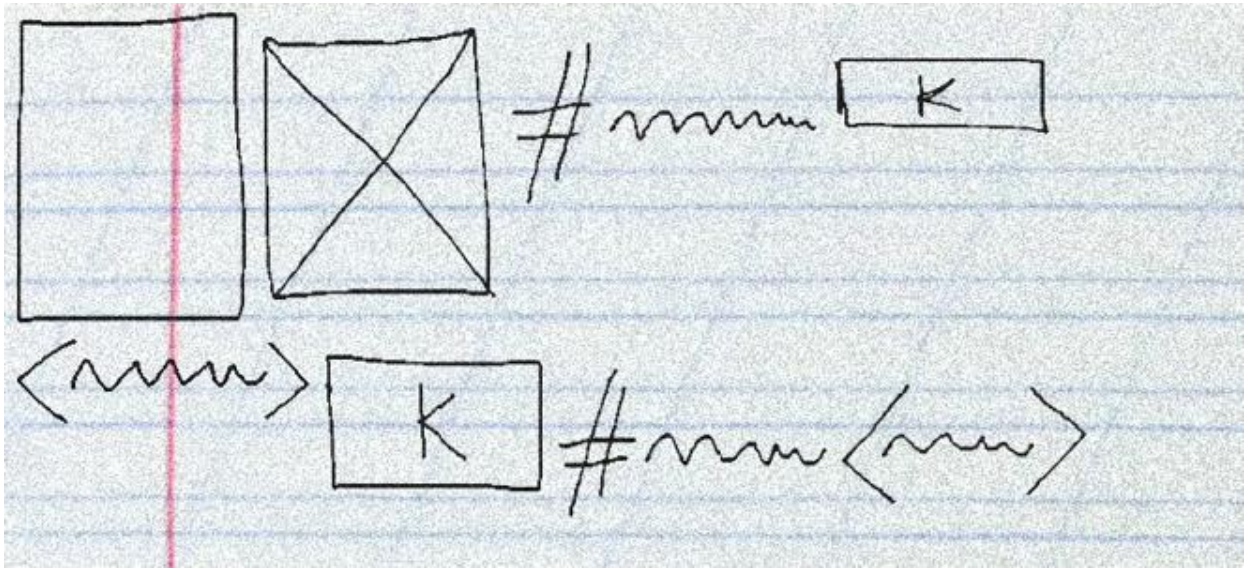


Рис. 26. Пример сгенерированного тренировочного изображения №1

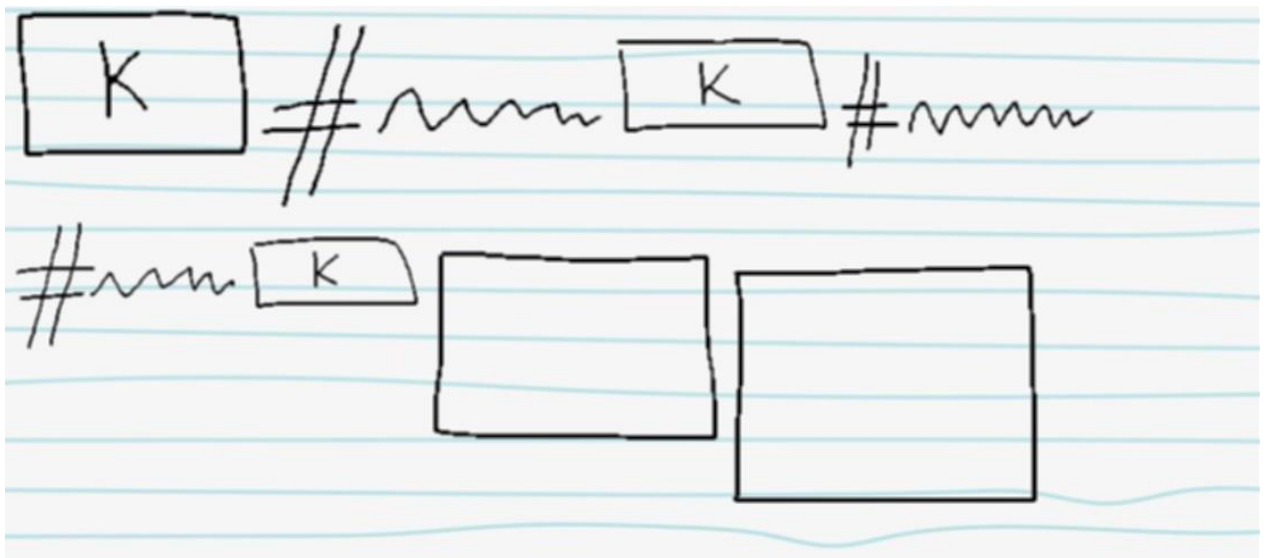


Рис. 27. Пример сгенерированного тренировочного изображения №2

В итоге благодаря описанной выше программе был сгенерирован набор из 2500 тренировочных изображений. Для того, чтобы качественно обучить нейронную сеть, необходимо разделить исходную выборку на обучающую и тестовую. Чаще всего, принято делить выборку в соотношении 70% на 30%. Количество получившихся тестовых и обучающих изображений представлено в табл.3.

Таблица 3

Соотношение обучающих и тестовых изображений

Тип	Процент от общей выборки	Количество
Обучающее	70	1750
Тестовое	30	750

На рисунке ниже (рис.28) можно увидеть количество объектов на тренировочных и тестовых изображениях, принадлежащих к определенному классу.

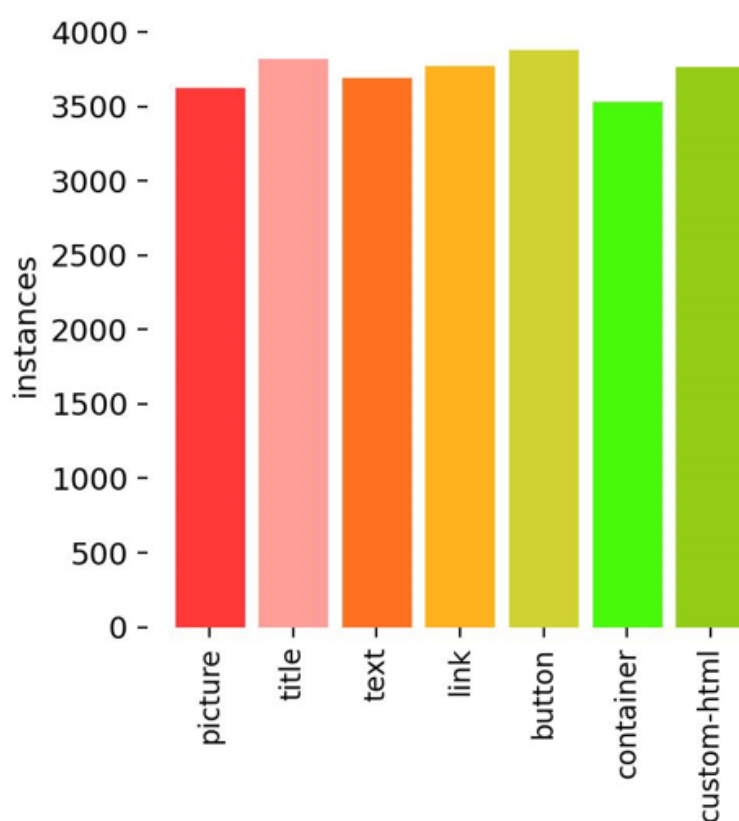


Рис. 28. Количество объектов на тренировочной и тестовой выборке

Как можно заметить, объекты относительно классов распределены равномерно, количество объектов в каждом классе примерно около трех с половиной тысяч, а всего объектов приблизительно двадцать четыре с половиной тысяч.

На рис.29 можно увидеть график уменьшения потерь предсказания классификации относительно эпох во время обучения.

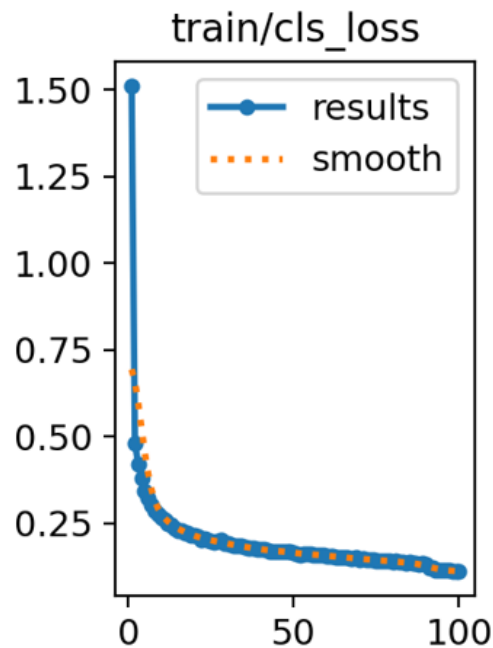


Рис. 29. Потери, связанные с классификацией

На рис.30 можно увидеть график уменьшения потерь предсказания ограничительных рамок относительно эпох во время обучения.

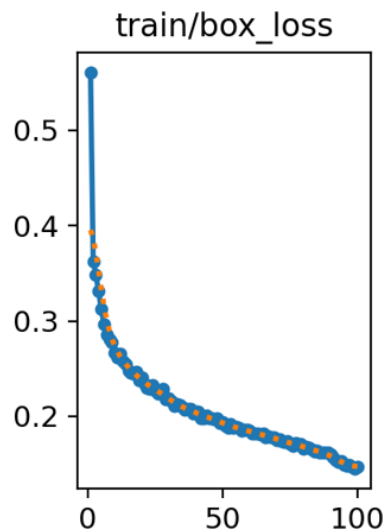


Рис. 30. Потери, связанные с предсказанием ограничительных рамок

Если обратить внимание на потери, то с каждой эпохой они постепенно уменьшаются, что соответствует нормальному процессу обучения и способности модели обобщать данные.

Также полезно оценить обучение с помощью метрик: точности (precision), полноты (recall), и mAP (mean average precision).

На рисунке изображен график точности относительно эпох (рис.31).

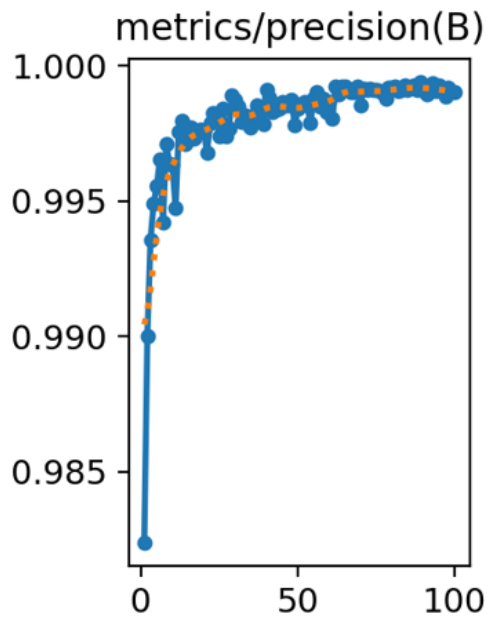


Рис. 31. График точности относительно эпох

На рис.32 изображен график полноты относительно эпох.

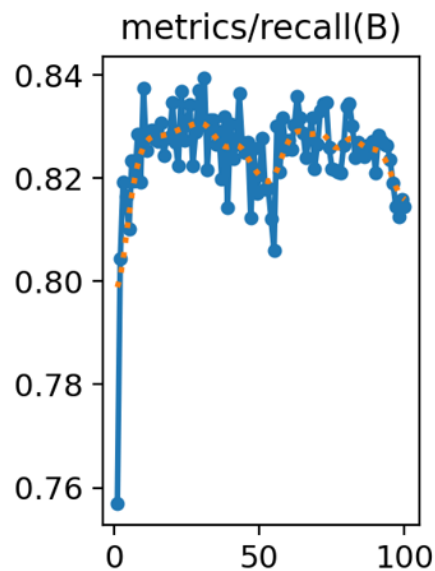


Рис. 32. График полноты относительно эпох

На рисунке ниже изображен график результатов метрики mAP при пороге Intersection Over Union равным 50% (рис.33).

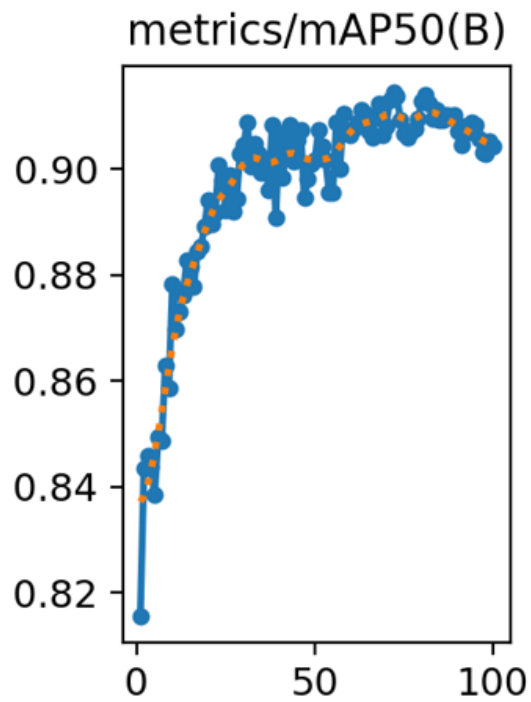


Рис. 33. График результатов метрики mAP при пороге Intersection Over Union равный 50%

На рисунке ниже изображен график результатов метрики mAP при пороге Intersection Over Union варьирующимся от 50 до 95% (рис.34).

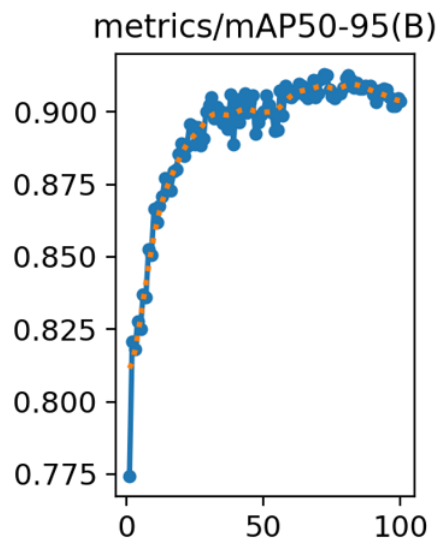


Рис. 34. График результатов метрики mAP при пороге Intersection Over Union варьирующийся от 50% до 95%

Исходя из графиков выше можно наблюдать уменьшение функции потерь во время обучения, что свидетельствует о том, что сеть успешно учится и снижает ошибку предсказания.

Графики Precision, Recall, mAP0.5 и mAP50-95 также демонстрируют отличные результаты работы модели. Precision показывает, насколько точно модель выявляет объекты, среди всех классифицированных как таковые. Recall отражает способность модели обнаруживать все реальные объекты из определенного класса. mAP0.5 и mAP50-95 – это средняя точность обнаружения объектов с различными порогами IoU (Intersection over Union).

Таким образом, на основании представленных графиков и результатов нейронной сети, можно с уверенностью сделать вывод, что модель успешно обучена и демонстрирует высокое качество обнаружения объектов.

2.3. Результаты моделирования детектора объектов с рукописных рисунков-макетов для генерирования HTML-шаблонов

В результате моделирования была подобрана подходящая для выполнения задачи архитектура сверточной нейронной сети. Дано её формальное описание. Также была разработана программная реализация данной модели и успешно обучена, что описывается в предыдущем разделе. Используя, например, метрику mAP (mean Average Precision) – это популярная метрика измерения точности детекторов объектов таких как Faster R-CNN, SSD и т.д., можно сравнить используемую модель с другими существующими моделями. Чем выше будет показатель mAP, тем лучше работает модель [22].

В процессе тренировки были записаны потери и точность модели на тренировочной и тестовой выборке для оценки производительности. Результаты тренировки модели показали, что сеть быстро снижает потери на каждой эпохе, сходится к минимуму уже после первых нескольких эпох и достигает высокой точности. При вычислительной стоимости на каждой итерации, YOLOv3 продемонстрировал значительно лучшую

производительность по сравнению с традиционными методами, обеспечивая более точные и стабильные результаты.

Покажем результаты работы нейросети на изображении без применения алгоритма «Non maximum suppression» (рис.35).

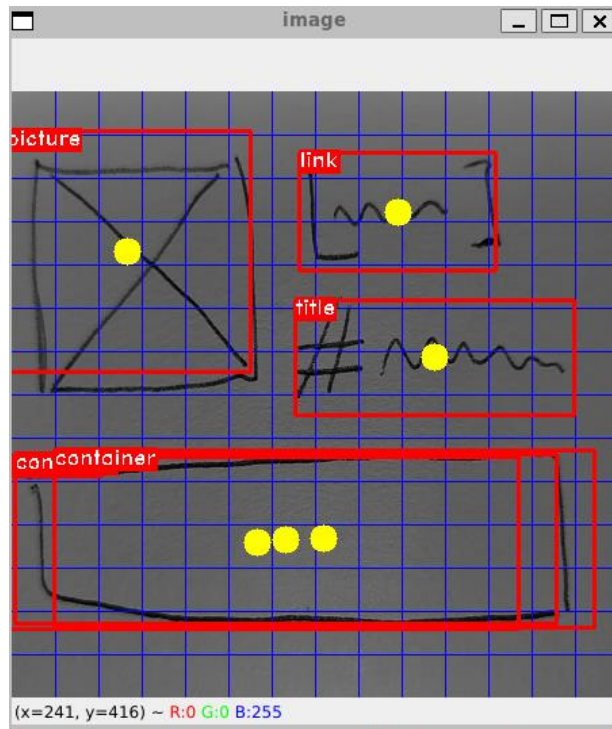


Рис. 35. Результат работы нейросети без применения алгоритма «Non maximum suppression»

Как можно заметить на рисунке выше, изначальное изображение делится на ячейки, в данном случае размером 14 на 14, то есть всего 196 ячеек, жёлтый круг обозначает, в какой из ячеек произошло обнаружение, то есть этот жёлтый круг является центром распознанного объекта, каждому из объектов присваивается принадлежность к классу. Без применения алгоритма «Non maximum suppression» могут быть лишние распознавания, которые описывают один и тот же объект, как и произошло с объектом класса «container».

С применением алгоритма «Non maximum suppression» лишние распознавания будут отсеяны, при этом уберутся только те распознавания, которые будут иметь меньшую уверенность («confidence score»). Приведем

пример распознавания с использованием алгоритма «Non maximum suppression» (рис.36-37).

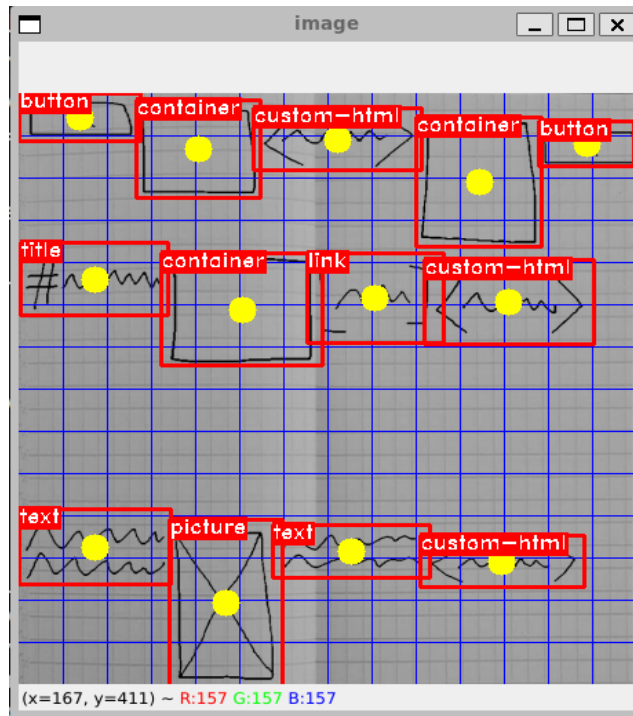


Рис. 36. Распознавания на тренировочном изображении с применением алгоритма «Non maximum suppression» №1

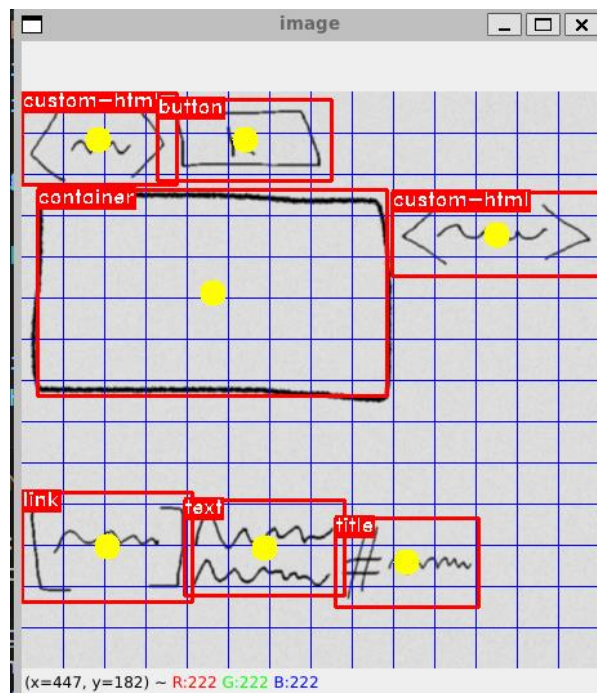


Рис. 37. Распознавания на тренировочном изображении с применением алгоритма «Non maximum suppression» №2

Как видно на рисунках выше, лишних распознаваний теперь нет, каждый объект распознается верно и верно соотносится их принадлежность к своему классу. Теперь покажем, каким образом нейронная сеть работает уже на тестовых изображениях (рис.38-41).

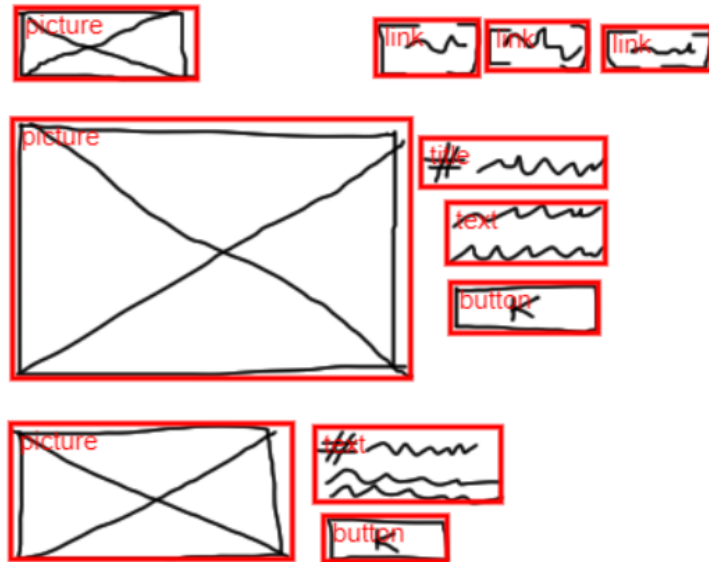


Рис. 38. Результат работы нейросети на тестовом изображении №1

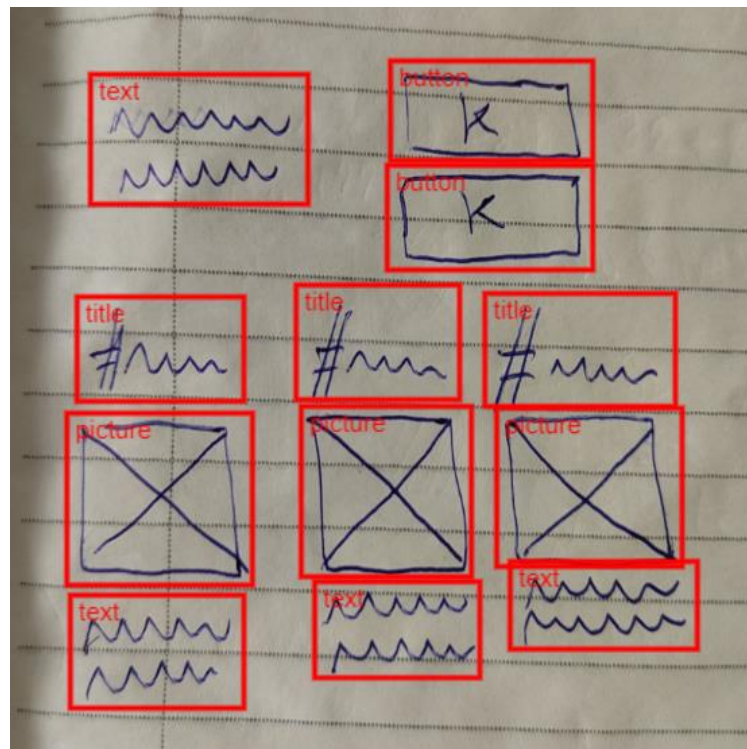


Рис. 39. Результат работы нейросети на тестовом изображении №2

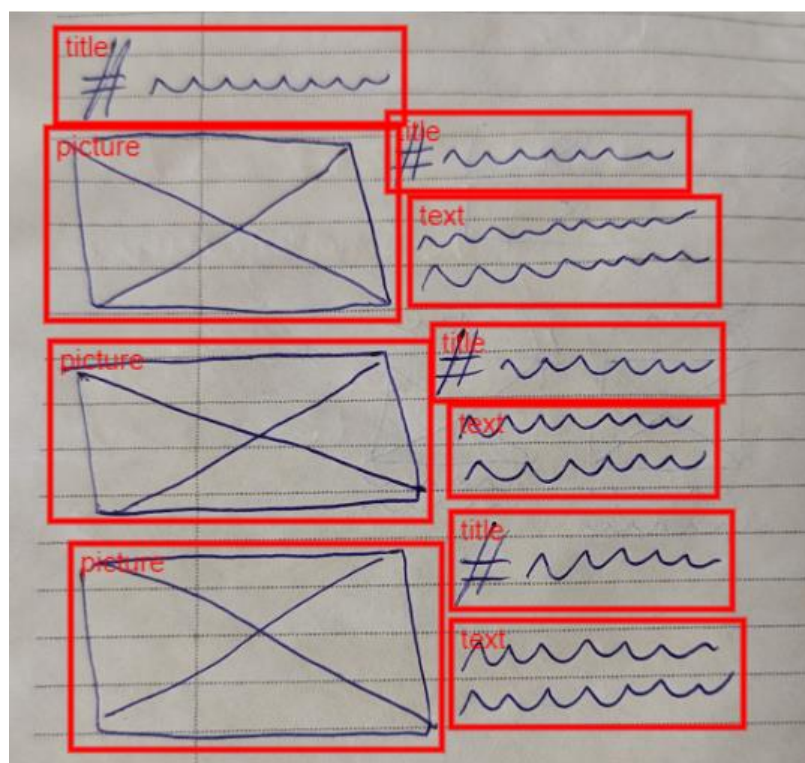


Рис. 40. Результат работы нейросети на тестовом изображении №3

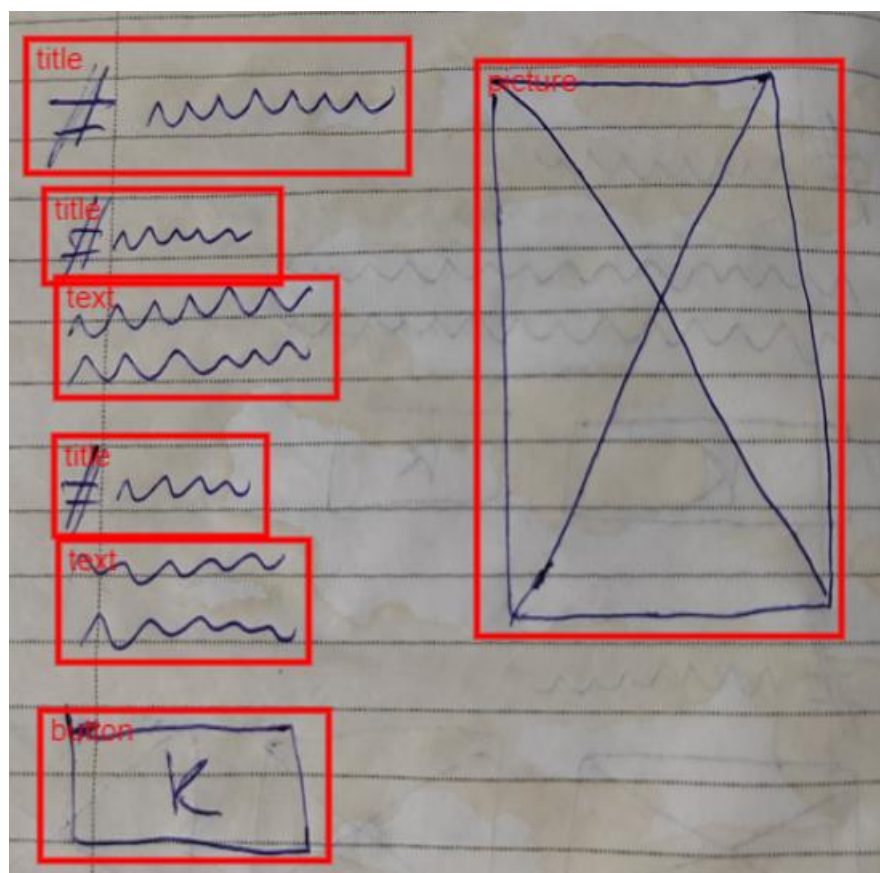


Рис. 41. Результат работы нейросети на тестовом изображении №4

В результате тестирования модели на тестовой выборке достигнута точность предсказаний по метрике mAP50 в размере около 93%, что свидетельствует о хорошей способности модели обобщать и обнаруживать объекты на новых данных. Метрики, такие как точность и полнота, были также оценены.

2.4. Разработка метода генерирования HTML-шаблонов по рукописным рисункам-макетам

Основополагающей частью работы является разработка метода генерации HTML-шаблонов по рукописным рисункам-макетам. Поскольку было решено использовать сверточную нейросеть для детектирования объектов на изображении, то при разработке метода будем отталкиваться от этого. На рис.42. представлено, как схематически должно работать разрабатываемое ПО.

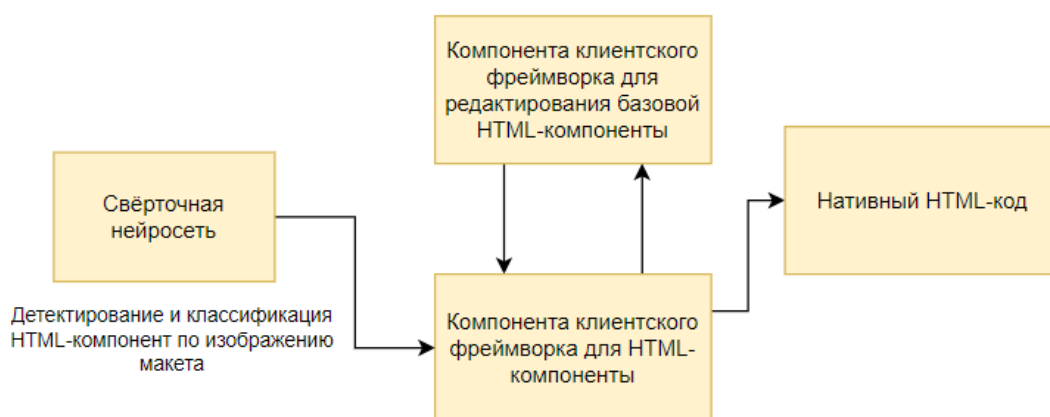


Рис. 42. Схематическое изображение ПО

То есть, на вход программе подаётся изображение рукописного рисунка-макета, далее сверточная нейросеть детектирует объекты на изображении и передаёт результат на следующий этап. На следующем этапе строится HTML-шаблон из распознанных объектов, в результате чего получается HTML-код, который можно просмотреть или отредактировать. Этот HTML-код можно запустить в браузере, чтобы увидеть полученный результат.

2.5. Разработка алгоритмов генерирования HTML-шаблонов по рукописным рисункам-макетам

Прежде всего было необходимо подготовить тренировочный набор изображений для обучения сверточной нейронной сети, для этого использовался следующий алгоритм, как показано на рис.43.

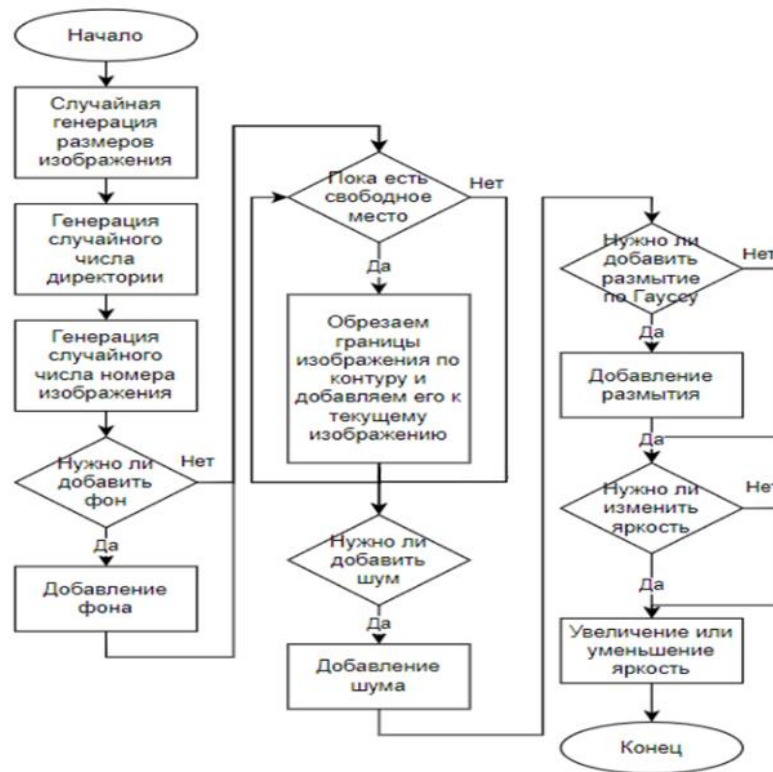


Рис. 43. Алгоритм подготовки тренировочных изображений

То есть алгоритм является циклическим и генерирует целый набор изображений, каждое изображение имеет случайную ширину и высоту в определённом заранее диапазоне, также случайным образом генерируются на изображении объекты, добавляются различные шумы и другие искажения, чтобы симитировать, будто изображение было сфотографировано с телефона. Таким образом, получившиеся изображения можно использовать для тренировки нейросети, чтобы уже на конечном этапе применять нейронную сеть на реальных изображениях.

На рис.44. представлен алгоритм обучения сверточной нейронной сети.



Рис. 44. Алгоритм обучения сверточной нейросети

Изображение изначально переводится в формат серого, чтобы сэкономить видеопамять на двух дополнительных цветовых каналах, поскольку цвет для детектирования объектов в данном случае роли не играет.

После этапа детектирования объектов на изображении – необходимо грамотно расположить эти объекты на HTML-шаблоне для того, чтобы результат генерирования шаблона удовлетворял заданным требованиям. А именно, нужно учесть, чтобы итоговые блоки были гибкими и подстраивались под разные разрешения устройств. Для этого предусмотрен модуль гибких макетов CSS (CSS Flexible Box Layout Module), также известный как Flexbox, он представляет собой новый метод позиционирования блоков с помощью CSS3. До появления Flexbox для позиционирования блоков требовалось использовать свойства `float: left | right | none | inherit`. Основным преимуществом Flexbox является возможность использования новой модели

верстки даже в случае, когда размеры контейнеров неизвестны или динамически изменяются. Основными элементами Flexbox являются:

- flex –контейнер;
- flex – элементы [23].

Flexbox вёрстка – один из наиболее популярных и важных типов вёрстки, который обеспечивает корректный дизайн шаблонов, адаптирующийся в зависимости от разрешения электронных устройств [24].

Flexbox имеет следующие возможности:

- вертикальное выравнивание блоков внутри родителя;
- автоматическое выравнивание высоты колонок независимо от контента;
- гибкое выравнивание элементов по вертикали или горизонтали без привязки к границам окна браузера;
- возможность задавать произвольный порядок блоков;
- установка размеров блоков в пропорциональном соотношении [25].

По этой причине при разработке алгоритма генерации шаблонов будет использоваться упор на использование именно Flexbox-подхода.

Непосредственно сам проектируемый алгоритм генерации HTML-шаблонов в рамках исследования состоит из трех частей:

1. Формирование DSL-структуры (DSL – это предметно-ориентированный язык);
2. Вычисление отступов для строк и столбцов в соответствии с положением объектов на рукописном рисунке;
3. Формирование HTML-шаблона на основе DSL-структуры с учетом отступов для иерархического дерева элементов.

Поскольку требуется расположить распознанные с изображения объекты внутри HTML-шаблона, то необходимо эти объекты в первую очередь поместить в DSL-структуру (чтобы из этой DSL-структуры на следующих этапах сформировать HTML-шаблон с Flexbox-подходом). DSL представляет

из себя древовидную структуру, в качестве базового синтаксиса для DSL используется JSON (JavaScript Object Notation) (рис.45).

```

{
  rows : [<DSL-структура строк>]
  columns : [<DSL-структура столбцов>]
  object : <класс распознанного объекта>
  width : <относительная ширина>
  height : <относительная высота>
  minY : <минимальное положение объекта по оси Y для строки/столбца>
  maxY : <максимальное положение объекта по оси Y для строки/столбца>
  minX : <минимальное положение объекта по оси X для строки/столбца>
  maxX : <максимальное положение объекта по оси X для строки/столбца>
  paddingTop : <относительный отступ сверху>
  paddingBottom : <относительный отступ снизу>
  paddingLeft : <относительный отступ слева>
  paddingRight : <относительный отступ справа>
}

```

Рис. 45. Иерархическая DSL-структура

Первая часть алгоритма генерации HTML-шаблона, а именно формирование DSL-структуры показана на рис.46. В алгоритме на определенном шаге выполняется подсчет вертикального и горизонтального перекрытия между двумя разными объектами – за счет этого происходит соотнесение объектов к одной строке или одному столбцу в случае, если перекрытие больше заданного порога 0,2. Приведем пример вычисления вертикального перекрытия для двух объектов, для горизонтального всё вычисляется аналогично:

- сначала определяем, где начинается перекрытие объектов по вертикали. Берем наибольшее значение из верхних (верхняя граница) точек объектов;
- затем определяем, где заканчивается перекрытие объектов по вертикали. Берем наименьшее значение из нижних (нижняя граница) точек объектов;
- длина вертикального перекрытия равна разнице между верхней и нижней границами перекрытия. Если эта разница отрицательная (то есть нижняя граница выше верхней), значит, перекрытия нет, и длина равна 0;

- наконец, чтобы выразить степень перекрытия объектов по вертикали, мы делим длину перекрытия на вертикальную высоту меньшего по высоте объекта. Результат будет числом от 0 до 1, где 1 означает полное вертикальное перекрытие, а 0 - полное отсутствие перекрытия.

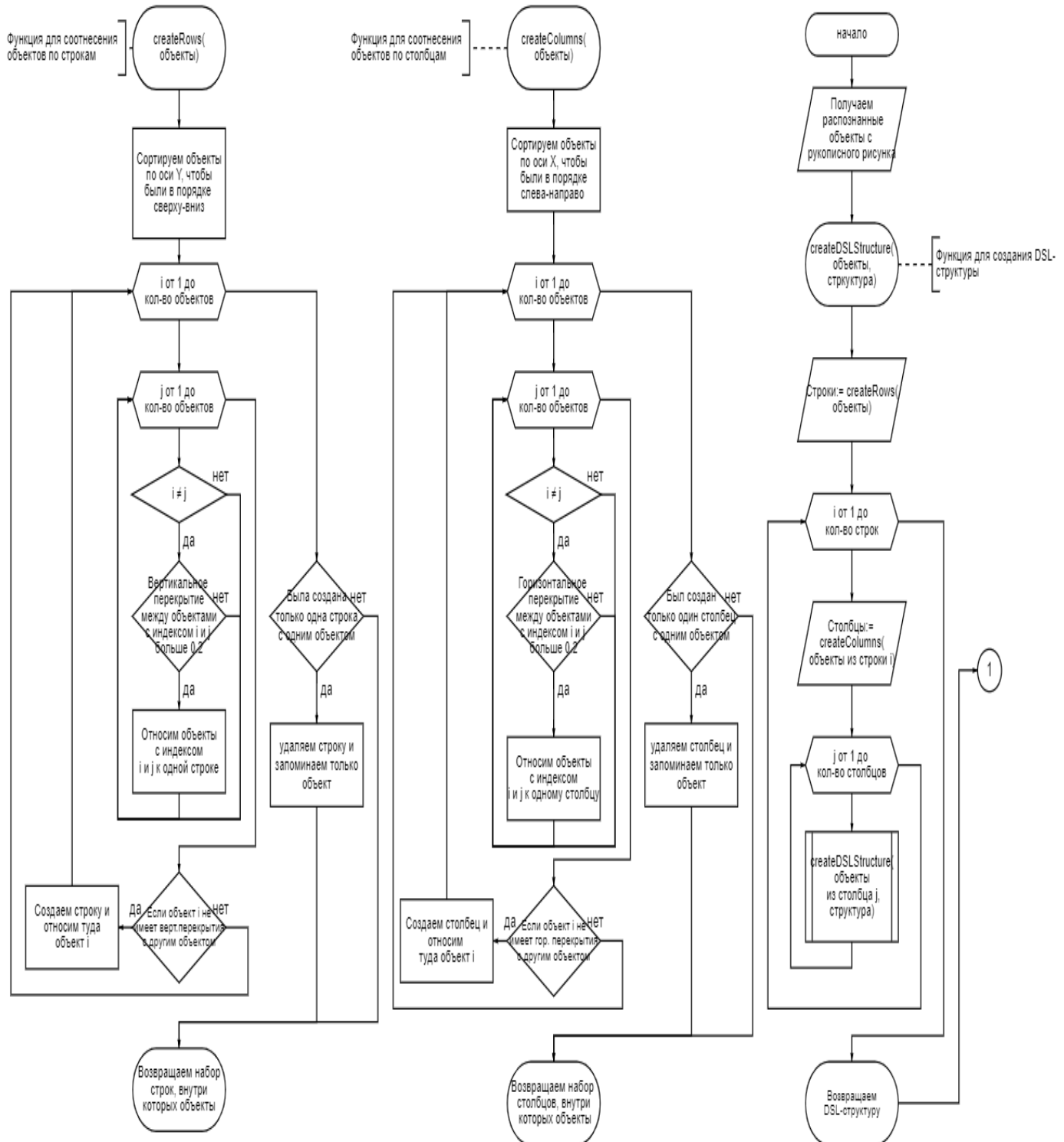


Рис. 46. Первая часть алгоритма генерации HTML-шаблона

DSL-структура формируется с помощью рекурсивной функции (поскольку сама структура является иерархичной), которая поочередно создает строки и столбцы, а также добавляет внутри строки или столбца объект, если дальнейшее разбиение на строки и столбцы невозможно. Функция «createDSLStructure» принимает оставшиеся объекты, а также объект структуры, изначально этот объект структуры пустой, и в ходе рекурсии постепенно заполняется согласно алгоритму. Вернувшись из этой функции DSL-структура используется в следующих частях алгоритма.

Вторая часть алгоритма генерации HTML-шаблона, а именно вычисление отступов для строк и столбцов показана на рис.47.

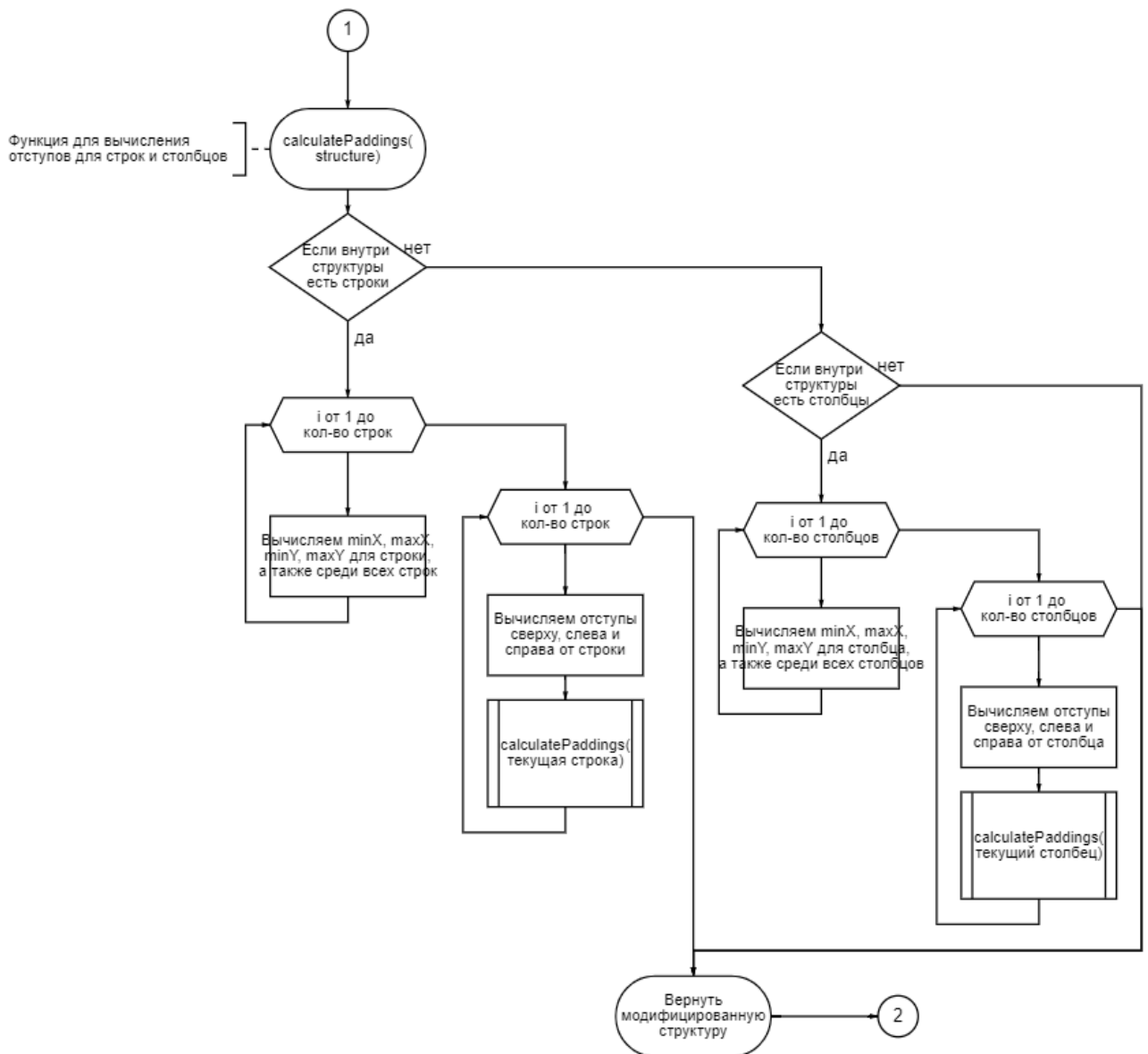


Рис. 47. Вторая часть алгоритма генерации HTML-шаблона

Вторая часть алгоритма использует непосредственно результаты первой части работы этого же алгоритма, то есть также рекурсивной функцией проходим созданную ранее DSL-структуру и вычисляем какими должны быть отступы, сами отступы считаются только для строк и столбцов.

Третья часть алгоритма генерации HTML-шаблона, а именно формирование HTML-шаблона на основе DSL-структуры с учетом отступов для иерархического дерева элементов показана на рис.48.

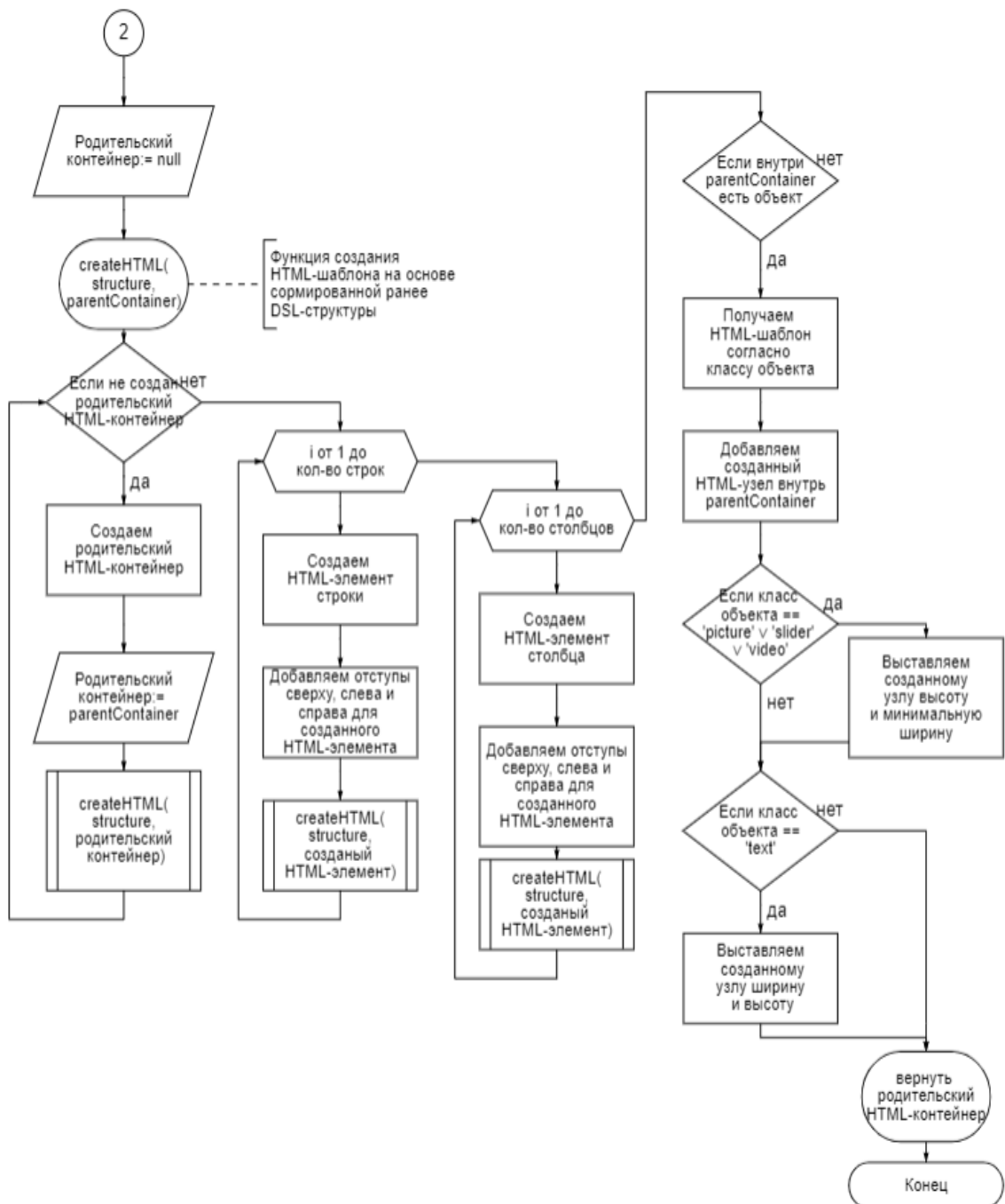


Рис. 48. Третья часть алгоритма генерации HTML-шаблона

Третья часть алгоритма также рекурсивно проходит по созданной ранее DSL-структуре, а затем применяются подсчитанные на второй части алгоритма отступы для строк и столбцов, и в результате формируется HTML-шаблон, который можно открыть для просмотра в браузере. Созданный HTML-шаблон по итогу является иерархическим и состоит из строк и столбцов (элементы «div» с классом «row» и «column»), аналогично, как формируют шаблон верстальщики.

2.6. Выводы по разделу 2

В данном разделе были сформированы математические модели процесса генерирования HTML-шаблонов по рукописным рисункам-макетам, а именно нейронная сеть для детектирования объектов на изображении, а также алгоритмы для обработки данных, приходящих с нейронной сети для построения HTML-шаблона. В том числе был разработан алгоритм генерации тренировочного набора изображений для нейросети, чтобы сэкономить время на обучении сети.

Так же были рассмотрены подходы к адаптации и критерии оценки адекватности работы модели. Для модели были подобраны наилучшие параметры при обучении, что соответствует высокой точности распознавания объектов на изображении.

Был разработан алгоритм генерирования HTML-шаблона на основе рукописного рисунка-макета и использован в программном обеспечении (Прил.2).

3. Проектирование информационного и программного обеспечения генерирования HTML-шаблонов по рукописным рисункам-макетам

3.1. Концептуальное моделирование проектируемой системы

3.1.1. Модель «черного ящика»

Модель черного ящика – это метод исследования или анализа, при котором изучается некий объект или система. Внимание в этой модели уделяется внешнему поведению и реакциям объекта или системы на воздействие внешних факторов [26]. Такой подход позволяет описать и прогнозировать поведение объекта или системы, не обращая внимания на его внутренние процессы. Пример черного ящика разрабатываемой системы представлен на рис.49.

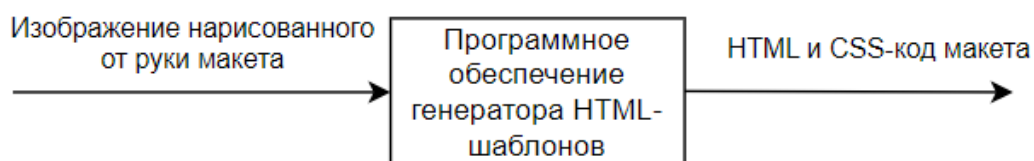


Рис. 49. Модель черного ящика разрабатываемой системы

В табл.4. представлено описание входного и выходного параметра системы.

Таблица 4

Входы и выходы системы

Входы и выходы	Описание
Изображение нарисованного от руки макета	Двумерный массив значений пикселей изображения
HTML и CSS-код макета	Сгенерированный HTML-код на основе распознанных объектов с рукописного рисунка-макета

3.1.2. Модель состава

Модель состава – это метод исследования или анализа, при котором объект или система анализируются с точки зрения их внутреннего устройства,

структуры без рассмотрения взаимодействия составных частей. Такой подход позволяет понять, какие элементы составляют объект или систему, и какие свойства у различных компонентов. Модель состава помогает углубленно изучить внутренние процессы и структуру объекта или системы [27].

Модель состава представлена на рис.50.

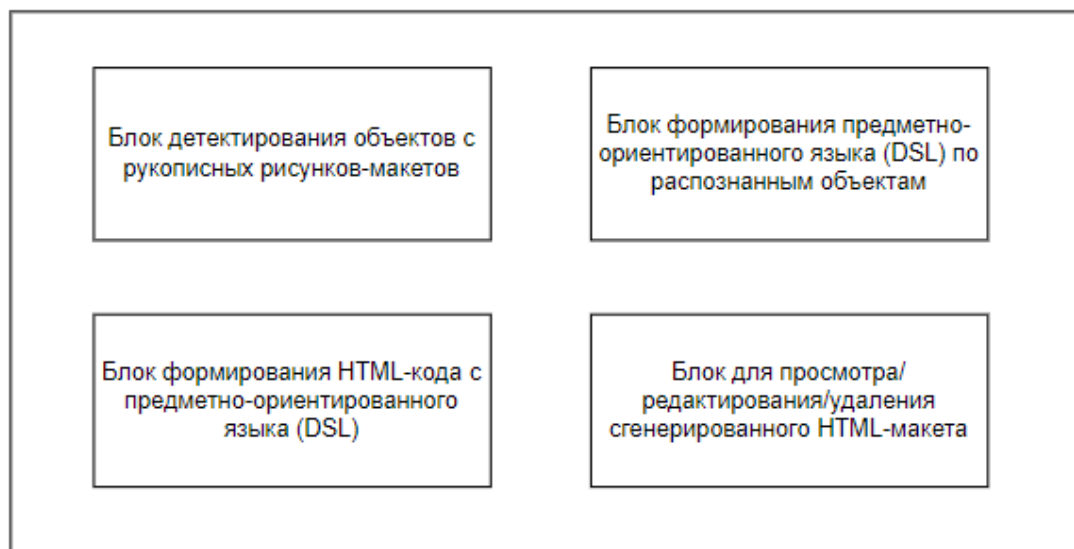


Рис. 50. Модель состава

В табл.5 представлено краткое описание подсистем проектируемой модели состава. Таблица включает в себя указанные на рисунке выше подсистемы и их краткое описание.

Таблица 5

Описание подсистем проектируемой системы

Подсистема	Описание
Блок детектирования объектов с рукописных рисунков-макетов	Данный блок отвечает за возможность детектировать predetermined классы объектов на изображении
Блок формирования предметно-ориентированного языка (DSL) по распознанным объектам	Блок отвечает за формирование полученных результатов детектирования в предметно-ориентированный язык
Блок формирования HTML-кода с предметно-ориентированного языка (DSL)	Блок отвечает за преобразование из предметно-ориентированного языка в HTML-код, который будет отображать структуру объектов
Блок для просмотра/редактирования/удаления сгенерированного HTML-макета	Блок отвечает за возможность редактирования готового HTML-макета в нужный для пользователя вид, позволяет

	просматривать сгенерированный код а также удалять лишние фрагменты
--	--

3.1.3. Модель структуры

Модель структуры – это метод анализа, при котором объект или система рассматриваются с точки зрения их организации, взаимосвязей и внутренней структуры. Этот подход позволяет изучать различные элементы, их взаимосвязи, иерархию и организацию внутри объекта или системы. Модель структуры позволяет понять, как устроена система, какие элементы в нее входят, как они связаны между собой и какие функции они выполняют в рамках этой структуры. В отличие от модели состава, модель структуры выражает связи между подсистемами.

Модель структуры представлена на рис.51.

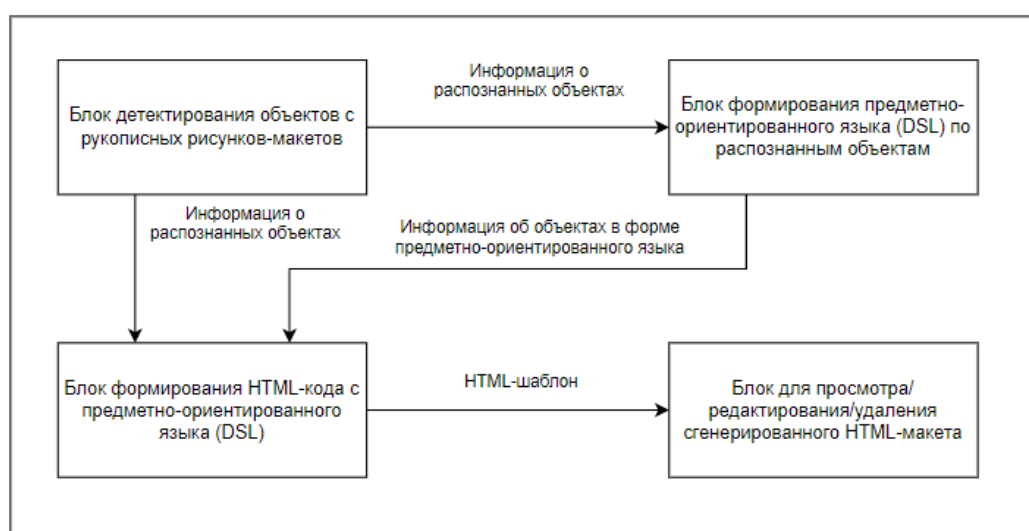


Рис. 51. Модель структуры

В табл.6. представлено краткое описание связей подсистем модели структуры.

Таблица 6

Описание подсистем

Подсистема	Описание
1	2

Блок детектирования объектов с рукописных рисунков-макетов	Сообщает другим подсистемам информацию о распознанных объектах, отправляет массив, в котором хранится наименование класса объекта, его пиксельные координаты, ширина и высота в пикселях
--	--

Продолжение табл.6

1	2
Блок формирования предметно-ориентированного языка (DSL) по распознанным объектам	Получает информацию о распознанных объектах и переводит в формат предметно-ориентированного языка, который уже отправляет в другую подсистему
Блок формирования HTML-кода с предметно-ориентированного языка (DSL)	Получает информацию об объектах с нейросети а также в формате предметно-ориентированного языка, чтобы проверить что ничего не потерялось и объекты соответствуют друг другу, формирует HTML-код, соответствующий формату DSL
Блок для просмотра/редактирования/удаления сгенерированного HTML-макета	Получает HTML-шаблон, который можно будет редактировать или просматривать

3.1.4. Модель «белого ящика»

Модель «белого ящика» представляет собой методологию анализа и разработки систем, основанную на подробном изучении внутренней структуры системы. В рамках данной модели разработчики имеют ограниченное или полное знание о внутренних механизмах функционирования программы, что позволяет им создавать более эффективные и оптимизированные системы.

Проектирование программного обеспечения по модели «белого ящика» предполагает углубленное понимание структуры данных, алгоритмов, логики выполнения и внутренних взаимосвязей компонентов системы. Знание этих аспектов позволяет разработчикам принимать обоснованные архитектурные решения и выбирать подходящие паттерны проектирования для достижения необходимой функциональности и высокой производительности системы.

На рис.52. представлена модель «белого ящика» системы.

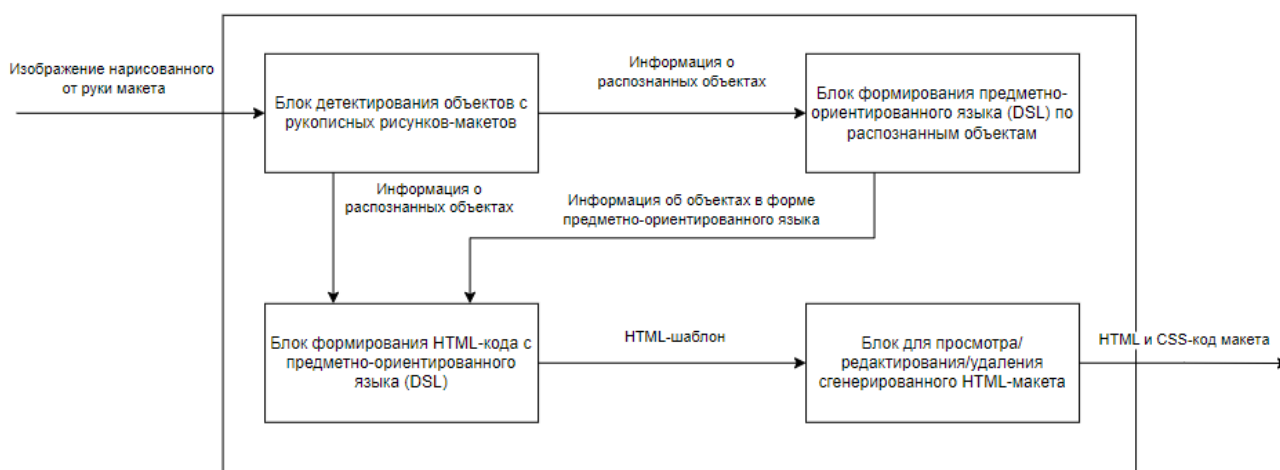


Рис. 52. Модель «белого ящика»

3.2. Проектирование программного обеспечения

3.2.1. Выбор технологии проектирования, среды и языка

программирования

3.2.1.1. Выбор модели жизненного цикла

Перед выбором среды разработки и языка программирования для решения задачи, первым делом необходимо определиться с моделью жизненного цикла (ЖЦ). Жизненный цикл программного обеспечения – это непрерывный процесс, который начинается с момента принятия решения о необходимости его создания и заканчивается в момент его полного изъятия из эксплуатации. Модели жизненного цикла ПО – упрощенное и обобщенное представление о том, как развивается продукт.

Среди моделей жизненного цикла программного обеспечения наиболее известны следующие:

- каскадная модель;
- итерационные модели;
- инкрементная модель;
- спиральная модель.

В качестве модели жизненного цикла была выбрана поэтапная модель с промежуточным контролем – это итерационная модель разработки с циклами обратной связи между этапами (рис.53).

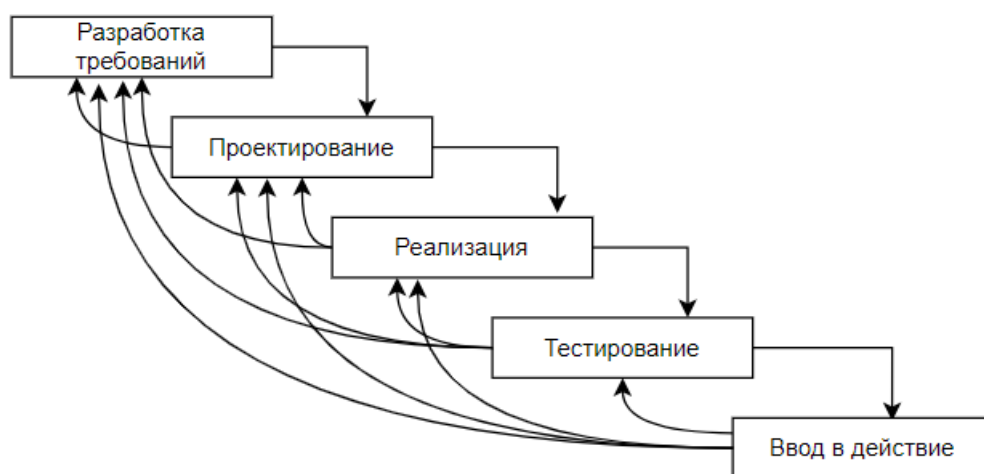


Рис. 53. Поэтапная модель ЖЦ с промежуточным контролем

Модель характеризуется следующими свойствами взаимодействия этапов:

- модель состоит из последовательно расположенных этапов (точно так же, как и «водопад»);
- каждый этап имеет обратную связь с предыдущими этапами;
- исправление ошибок происходит на каждом из этапов, сразу при выявлении проблемы – это промежуточный контроль;
- этапы перекрываются во времени по причине наличия обратной связи: следующий этап не начинается, пока не завершится предыдущий; при первом проходе по модели вниз, как только обнаружена ошибка, осуществляется возврат снизу-вверх к предыдущим этапам, которые повлекли ошибку. Таким образом, фактически этапы оказываются растянутыми во времени;
- результат появляется только в конце разработки, как и в модели «водопад».

Достоинства итерационной модели:

- возможность оперативно вносить коррективы в проект;

- более гибкая, позволяет учесть какие-то новые факторы, устранять выявленные несоответствия, спецификации.

Недостатки итерационной модели:

- при большом числе итераций растёт время проектирования, возникают расхождения в проектных решениях и документации, запутывается функциональная и системная архитектура ПО, необходимость в перепроектировании старой или создании новой системы может возникнуть сразу после этапа внедрения или эксплуатации;
- проект может никогда не закончиться (постоянное внесение изменений и возврат назад может привести к тому, что никогда не остановится в этой последовательности действий) [28].

3.2.1.2. Выбор подхода к разработке

Следующим шагом необходимо выбрать подход к разработке, существует два основных подхода – структурный и объектный.

Сущность структурного подхода к разработке ИС заключается в ее декомпозиции (разбиении) на автоматизируемые функции: система разбивается на функциональные подсистемы, которые в свою очередь делятся на подфункции, подразделяемые на задачи и так далее. Процесс разбиения продолжается вплоть до конкретных процедур. При этом автоматизируемая система сохраняет целостное представление, в котором все составляющие компоненты взаимосвязаны. При разработке системы "снизу-вверх" от отдельных задач ко всей системе целостность теряется, возникают проблемы при информационной стыковке отдельных компонентов [29].

Все наиболее распространенные методологии структурного подхода базируются на ряде общих принципов. В качестве двух базовых принципов используются следующие:

- принцип «разделяй и властвуй» - принцип решения сложных проблем путем их разбиения на множество меньших независимых задач, легких для понимания и решения;
- принцип иерархического упорядочивания - принцип организации составных частей проблемы в иерархические древовидные структуры с добавлением новых деталей на каждом уровне.

На стадии проектирования ИС модели расширяются, уточняются и дополняются диаграммами, отражающими структуру программного обеспечения: архитектуру ПО, структурные схемы программ и диаграммы экранных форм.

Как показала практика, традиционные методы процедурного программирования не способны справиться ни с нарастающей сложностью программ и их разработки, ни с необходимостью повышения их надежности. В середине 80-х годов XX века для разработки ПО большого объема было предложено использовать объектный подход, а в качестве технологии выбрано объектно-ориентированное программирование (ООП) - методика разработки программ, в основе которой лежит понятие объекта.

Объект – это некоторая структура, соответствующая объекту реального мира, его поведению. Каждый объект имеет состояние, обладает четко определенным поведением и уникальной идентичностью. Совокупность атрибутов (свойств) и их значений характеризует объект. Объекты, описываемые одинаковыми наборами атрибутов, объединяются в классы. Данные класса называются полями, процедуры и функции — методами. Все экземпляры одного класса (объекты, порожденные от одного класса) имеют один и тот же набор свойств и общее поведение, то есть одинаково реагируют на одинаковые сообщения. Каждый объект имеет определенное время жизни. В процессе выполнения программы или функционирования реальной системы, могут создаваться новые объекты и уничтожаться уже существующие.

Важнейшими понятиями ООП являются инкапсуляция, наследование, полиморфизм, которые позволяют конструировать сложные объекты из сравнительно простых. Программа, написанная с использованием ООП, состоит из множества объектов, взаимодействующих между собой путем передачи сообщений.

В основе объектного подхода к разработке ПО лежит объектная декомпозиция, т. е. представление разрабатываемого программного продукта в виде совокупности объектов, в процессе взаимодействия которых через передачу сообщений и происходит выполнение требуемых функций. Объектно-ориентированный подход имеет следующие преимущества:

- уменьшение сложности программного обеспечения;
- повышение надежности программного обеспечения;
- обеспечение возможности модификации отдельных компонентов программного обеспечения без изменения остальных его компонентов;
- обеспечение возможности повторного использования отдельных компонентов программного обеспечения.

Систематическое применение объектного подхода позволяет разрабатывать хорошо структурированные, надежные в эксплуатации, достаточно просто модифицируемые программные системы, поэтому ООП является одним из наиболее интенсивно развивающихся направлений теоретического и прикладного программирования. Однако при объектном подходе сразу можно выполнить декомпозицию только очень простого ПО [30].

Проектирование программного решения проводилось в соответствии со стандартом объектного моделирования UML (Unified Modeling Language).

Выбор данного стандарта проектирования обусловлен следующими причинами:

- UML - объектно-ориентированный язык, благодаря чему методы описания результатов анализа и проектирования семантически

близки к методам программирования на современных объектно-ориентированных языках;

- UML позволяет описать систему практически со всех возможных точек зрения и разные аспекты поведения системы;
- диаграммы UML сравнительно просты для чтения после достаточно быстрого ознакомления с его синтаксисом.

3.2.1.3. Выбор среды и языка программирования

Для программиста одним из ключевых инструментов является текстовый редактор кода. В рейтингах многих авторитетных источников можно выделить несколько лидеров на сегодняшний день, среди которых следующие: Visual Studio Code, Sublime Text, Atom.

Программное обеспечение разрабатывается в IDE Visual Studio Code (сокращенно VS Code) — редактор исходного кода, разработанный Microsoft для Windows, Linux и macOS. VS Code обладает рядом преимуществ, таких как: кроссплатформенность, поддержка системы контроля версий Git, синхронизация настроек редактора среди рабочих станций, развитая инфраструктура, способная дополнить/настроить редактор в соответствии с самыми изысканными потребностями [31].

Позиционируется как «лёгкий» редактор кода для кроссплатформенной разработки веб- и облачных приложений (рис.54).

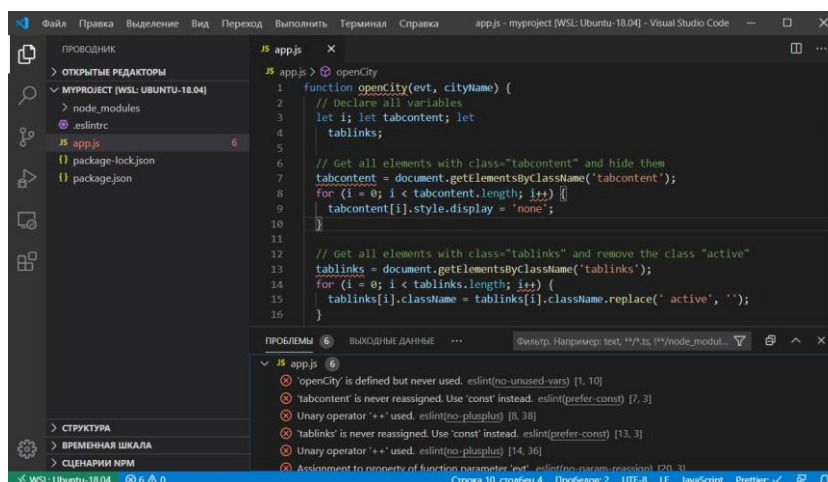


Рис. 54. Интерфейс IDE VS Code

Для разработки нейронной сети был выбран язык программирования Python – это высокоуровневый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью, ориентированный на повышение производительности разработчика, читаемости кода и его качества, а также на обеспечение переносимости написанных на нём программ. Python обладает определенными характеристиками: он простой и логичный, гибкий и мультиплатформенный, обладает необходимыми библиотеками для машинного обучения и работы с искусственным интеллектом, а также имеет обширное сообщество разработчиков. Благодаря этим качествам Python является одним из наиболее популярных языков программирования в мире и активно используется в разработке нейронных сетей и искусственного интеллекта [32]. Также для разработки нейросети используется фреймворк TensorFlow – это открытая программная библиотека для машинного обучения, созданная компанией Google для решения задач построения и обучения нейронных сетей с целью распознавания и классификации образов [33].

3.2.2. Разработка логической модели

3.2.2.1. Диаграмма вариантов использования

Диаграмма вариантов использования представляет собой исходное представление системы и не отражает её физической реализации. Проектируемая система представлена как набор сущностей (актеров), которые взаимодействуют между собой с помощью вариантов использования [34].

На рис.55. представлена диаграмма вариантов использования.

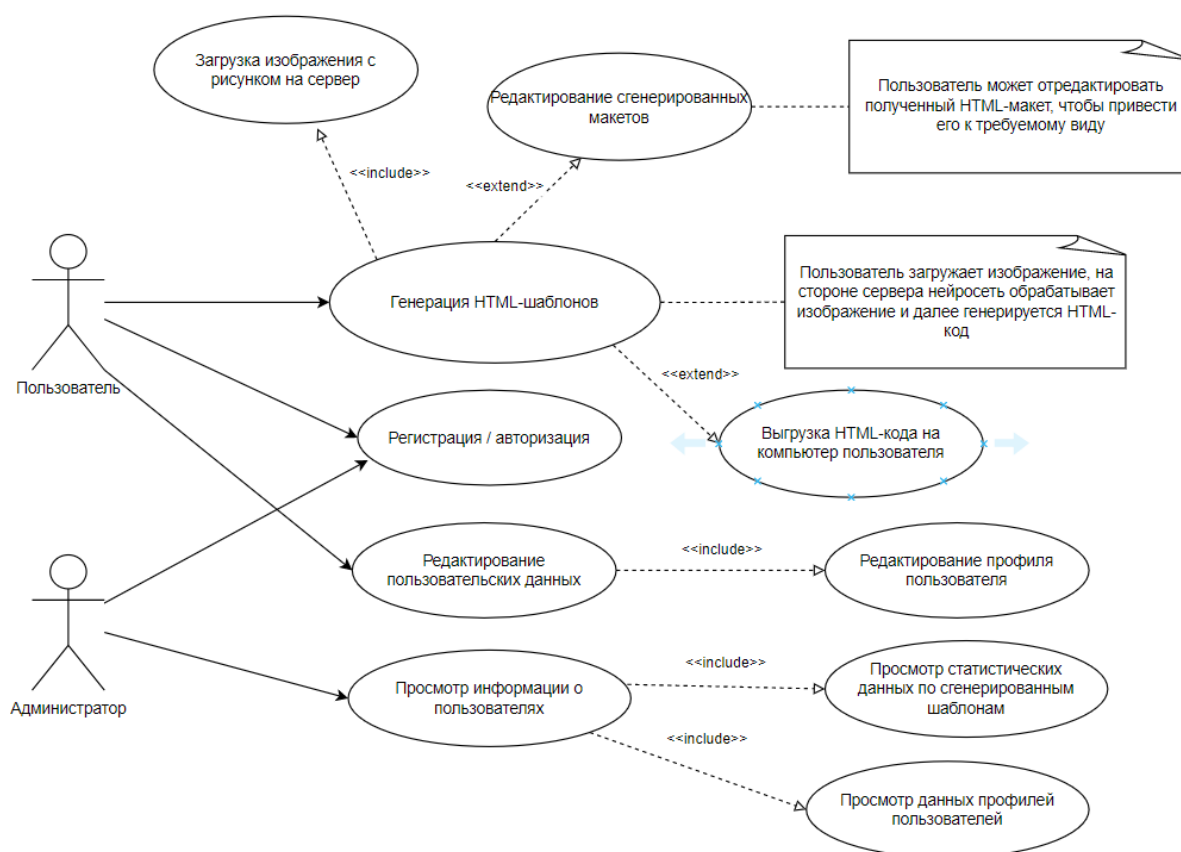


Рис. 55. Диаграмма вариантов использования

На диаграмме изображены Администратор и Пользователь. Пользователь взаимодействует с основной частью системы, а именно ему доступна возможность генерировать HTML-шаблоны. Администратору же доступна статистика по пользователям, он может просматривать и агрегировать информацию о том, кто и когда генерировал и какие шаблоны.

Для каждого варианта использования были составлены описания, которые представлены в табл.7-14.

3.3.2.1.1. Вариант использования «Регистрация/авторизация»

Таблица 7

Краткое описание ВИ «Регистрация/авторизация»

Название варианта	Регистрация / авторизация
Цель	Регистрация или авторизация действующего лица в системе
Действующие лица	Пользователь, Администратор
Краткое описание	Пользователь или администратор может зарегистрироваться или авторизоваться в системе
Тип варианта	Основной

Таблица 8

Детальное описание ВИ «Регистрация/авторизация»

Действия исполнителя	Отклик системы
1. Заполняет данные для регистрации/авторизации в соответствующем окне	2. При успешной регистрации/авторизации на сервере генерируется JWT-токен и отправляется пользователю, чтобы в дальнейшем его не разлогинило, а также происходит редирект на основную страницу сервиса.

Альтернатива

Если действующее лицо ввело некорректные данные, то в таком случае сервер ему возвращает ошибку в зависимости от её типа (пользователя не существует, неправильный логин или пароль).

3.3.2.1.2. Вариант использования «Редактирование профиля пользователя»

Таблица 9

Краткое описание ВИ «Редактирование профиля пользователя»

Название варианта	Редактирование профиля пользователя
Цель	Поменять свои данные на актуальные, чтобы полноправно пользоваться системой
Действующие лица	Пользователь
Краткое описание	Редактирование имени, фамилии, почты или пароля пользователя
Тип варианта	Расширенный

Таблица 10

Детальное описание ВИ «Редактирование профиля пользователя»

Действия исполнителя	Отклик системы
1. Пользователь нажимает на кнопку «Профиль»	2. Система перенаправляет пользователя на адрес редактирования данных профиля
3. Пользователь вводит актуальные данные своего профиля	4. Сервер сохраняет результат, отправляет ответ об успешном сохранении, далее клиент оповещает пользователя об успешном сохранении данных

Альтернатива

Если на стороне сервера произошла ошибка при сохранении данных, то пользовательские данные на клиенте не обновляются и пользователь оповещается об ошибке.

3.3.2.1.3. Вариант использования «Просмотр статистических данных по сгенерированным шаблонам»

Таблица 11

Краткое описание ВИ «Просмотр статистических данных по сгенерированным шаблонам»

Название варианта	Просмотр статистических данных по сгенерированным шаблонам
Цель	Администратор просматривает кто и какие шаблоны создавал
Действующие лица	Администратор
Краткое описание	Администратор может перейти в раздел для просмотра статистики, где будут генерироваться отдельно по пользователям таблицы и графики с созданными шаблонами
Тип варианта	Расширенный

Таблица 12

**Детальное описание ВИ «Просмотр статистических данных по
сгенерированным шаблонам»**

Действия исполнителя	Отклик системы
1. Администратор нажимает кнопку «Статистика по шаблонам»	2. Система перенаправляет пользователя на страницу с просмотром статистики
3. Администратор выбирает интересующего его пользователя, отправляется запрос на сервер	4. Система ищет в базе данных пользователя и возвращает статистические данные о нём в виде JSON-объекта, на клиенте формируются графики и таблицы

Альтернатива

При отсутствии данных о запрашиваемом пользователе система выдаст сообщение «Нет данных».

**3.3.2.1.4. Вариант использования «Просмотр данных профилей
пользователей»**

Таблица 13

Краткое описание ВИ «Просмотр данных профилей пользователей»

Название варианта	Просмотр данных профилей пользователей
Цель	Администратор получает данные о профиле пользователей
Действующие лица	Администратор
Краткое описание	Администратор может просматривать данные о профилях любого пользователя
Тип варианта	Расширенный

Таблица 14

Детальное описание ВИ «Просмотр данных профилей пользователей»

Действия исполнителя	Отклик системы
1	2
1. Администратор нажимает кнопку «Просмотреть профили»	2. Администратор перенаправляется на страницу просмотра профилей

1	2
3. Администратор выбирает интересующего его пользователя	4. Сервер возвращает информацию о профиле пользователя в JSON-формате

Альтернатива

Если у пользователя не заполнен профиль, то сервер вернёт ошибку 404 и на клиенте администратор увидит сообщение «Нет данных».

3.2.2.2. Контекстная диаграмма классов

Диаграмма классов — структурная диаграмма языка моделирования UML, демонстрирующая общую структуру иерархии классов системы, их коопераций, атрибутов (полей), методов, интерфейсов и взаимосвязей между ними. Широко применяется не только для документирования и визуализации, но также для конструирования посредством прямого или обратного проектирования [35].

Основными элементами являются классы и связи между ними. Классы характеризуются при помощи атрибутов и операций.

Атрибуты описывают свойства объектов класса. Большинство объектов в классе получают свою индивидуальность из-за различий в их атрибутах и взаимосвязи с другими объектами. Однако, возможны объекты с идентичными значениями атрибутов и взаимосвязей. То есть индивидуальность объектов определяется самим фактом их существования, а не различиями в их свойствах. Имя атрибута должно быть уникально в пределах класса. За именем атрибута может следовать его тип и значение по умолчанию. Операция есть функция или преобразование. Операция может иметь параметры и возвращать значения.

На рис.56. представлена контекстная диаграмма классов.

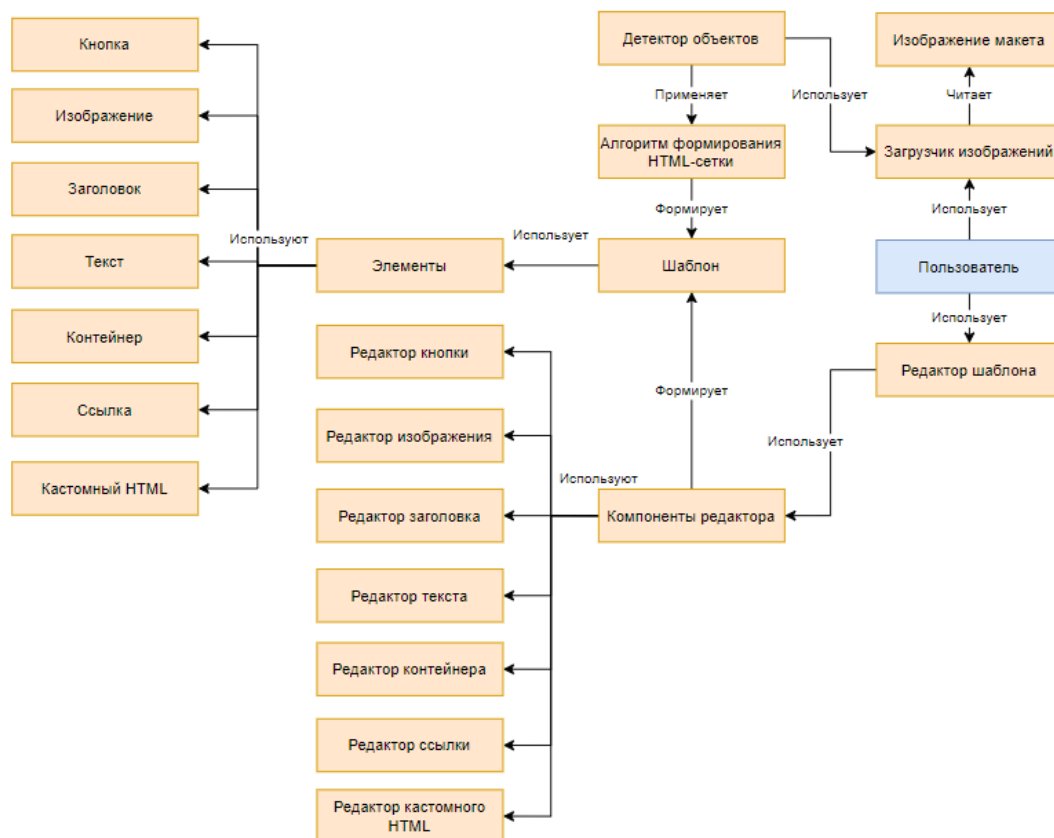


Рис. 56. Контекстная диаграмма классов

Описание контекстной диаграммы классов представлено в табл. 15.

Таблица 15

Описание контекстной диаграммы классов

Наименование	Описание
1	2
Пользователь	Представляет лицо, которое обращается к системе и работает с ней
Редактор шаблона	Сущность для редактирования сгенерированного HTML-шаблона
Компоненты редактора	Сущность, представляющая из себя набор компонентов для редактирования шаблонов
Редактор кнопки	Сущность для редактирования кнопки
Редактор изображения	Сущность для редактирования изображения

Продолжение табл.15

1	2
Редактор заголовка	Сущность для редактирования заголовка
Редактор текста	Сущность для редактирования текста
Редактор контейнера	Сущность для редактирования контейнера
Редактор ссылки	Сущность для редактирования ссылки
Редактор кастомного HTML	Сущность для редактирования кастомного HTML
Шаблон	Сущность, представляющая HTML-шаблон
Загрузчик изображений	Сущность для загрузки изображений макетов
Изображение макета	Сущность, представляющая собой изображение макета
Детектор объектов	Сущность для детектирования объектов с изображения рисунка-макета
Алгоритм формирования HTML-сетки	Сущность, реализующая алгоритм для формирования HTML-сетки
Элементы	Сущность, представляющая из себя набор элементов, из которых состоит шаблон

Разработанная контекстная диаграмма классов подтверждает высокую степень обособленности сущностей, при этом высокий уровень взаимодействия между объектами классов предметной области, поэтому был выбран объектный подход проектирования.

3.2.2.3. Диаграмма переходов состояния

Диаграмма переходов состояния – это графическое представление состояний системы и переходов между ними. Она используется для моделирования поведения системы, и позволяет показать, как система изменяет свое состояние в зависимости от определенных событий или действий. Диаграмма переходов состояния часто используется в разработке программного обеспечения, управлении проектами, проектировании бизнес-процессов и других областях [36].

На рис.57. изображена диаграмма переходов состояния

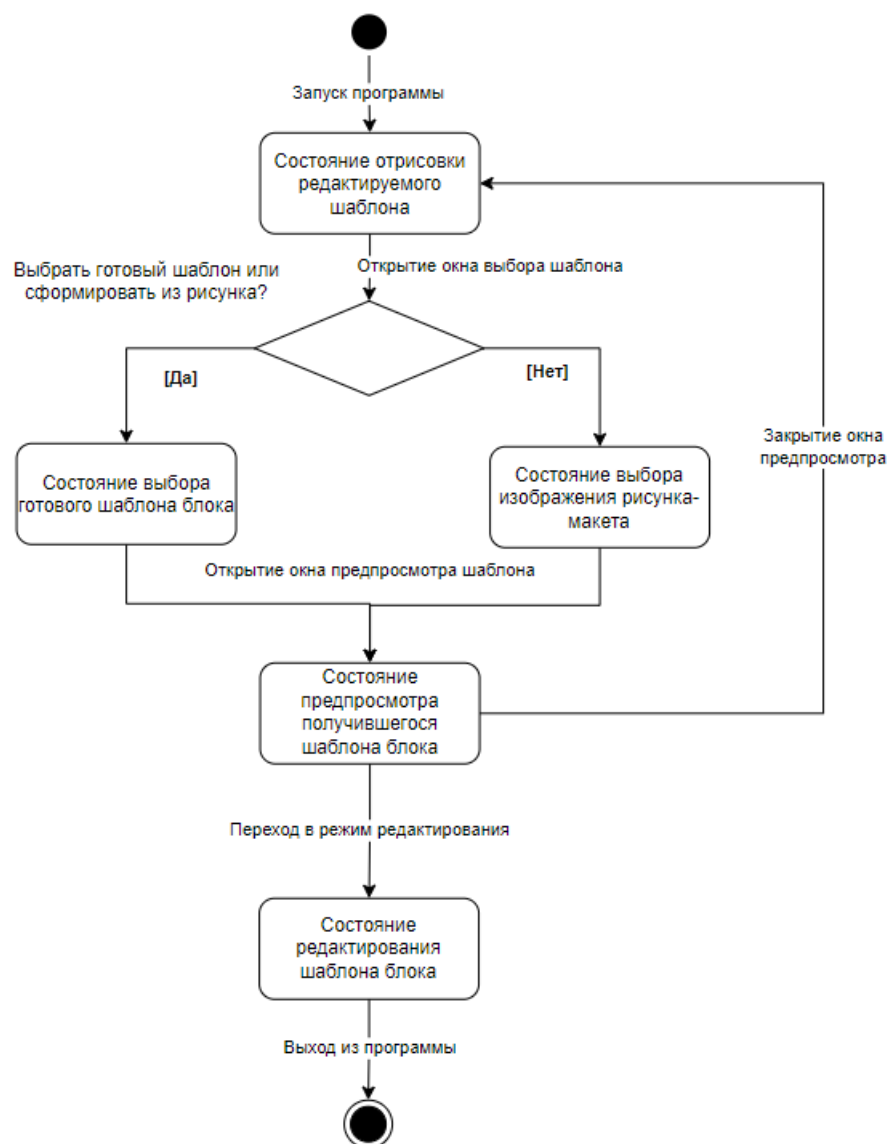


Рис. 57. Диаграмма переходов состояния

3.2.2.4. Диаграмма пакетов

Диаграмма пакетов – это вид диаграммы UML, который отображает структуру системы и ее компоненты в виде пакетов или группировки элементов. Диаграмма пакетов используется для организации и визуализации компонентов системы, их связей и зависимостей [37].

На диаграмме пакетов компоненты системы представлены в виде прямоугольников, которые содержат название компонента. Связи между компонентами отображаются в виде стрелок, указывающих на направление связи. С помощью диаграммы пакетов можно определить иерархию

компонентов, зависимости между ними, а также логическую структуру системы.

Диаграмма пакетов позволяет разделять систему на логические модули, что упрощает управление и поддержку системы, а также облегчает восприятие ее структуры для разработчиков и пользователей. Также она помогает представить компоненты системы в рамках ее контекста и отображает взаимодействие между ними.

В целом, диаграмма пакетов является мощным инструментом для моделирования и проектирования систем, а также обеспечивает понимание и визуализацию сложной структуры системы.

На рис.58. представлена диаграмма пакетов проектируемого ПО, а в табл.16 указано описание пакетов.

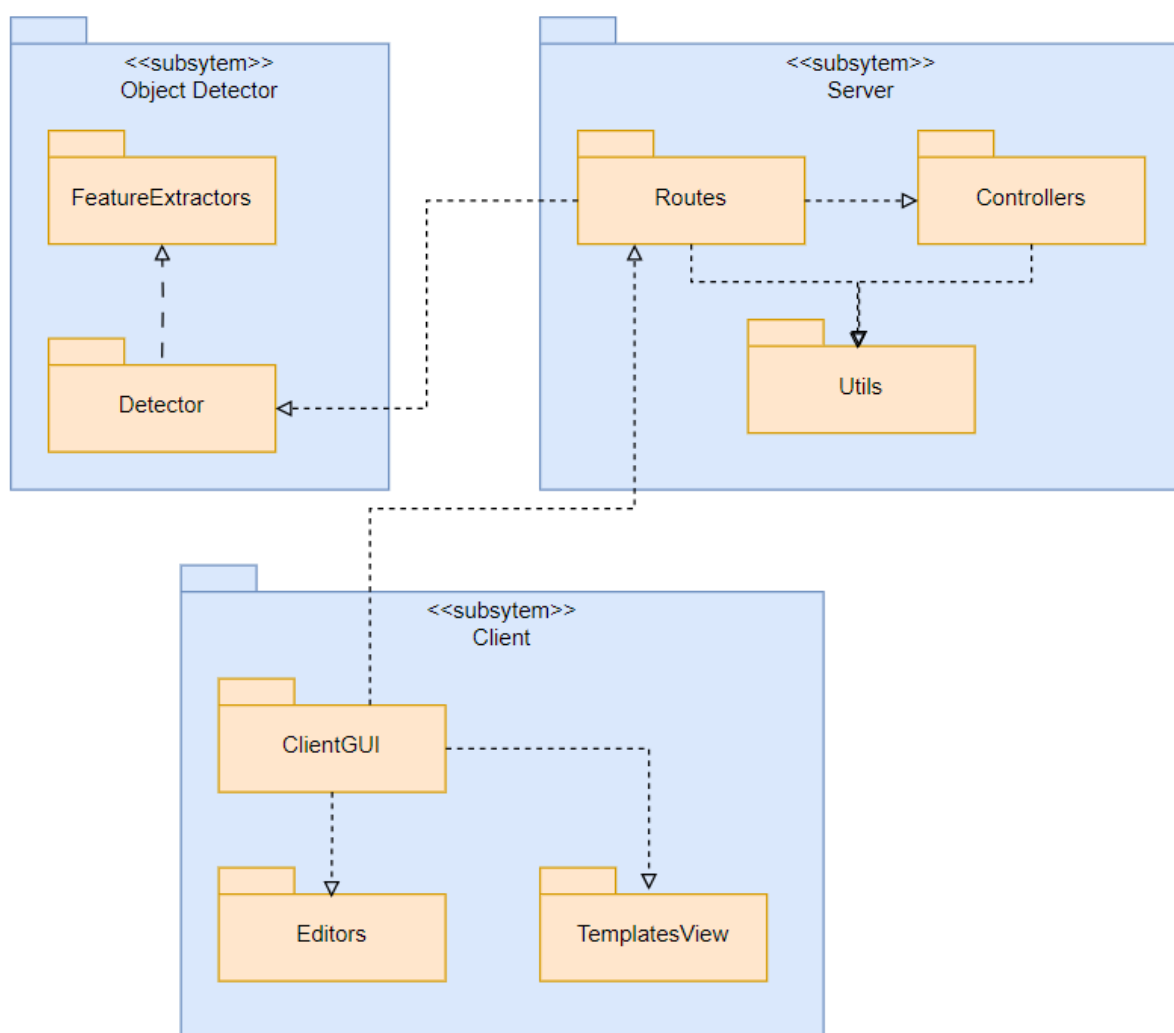


Рис. 58. Диаграмма пакетов

Пакеты

Название	Описание
Detector	Пакет, который содержит классы для детектирования объектов на изображении
FeatureExtractors	Пакет, который содержит классы для извлечения признаков с изображения (один из таких классов в дальнейшем используется в пакете детектирования)
Routes	Пакет, содержащий классы с API-маршрутами
Controllers	Пакет, содержащий классы, которые реализуются типовые функции
Utils	Пакет, содержащий классы с различными вспомогательными утилитами
ClientGUI	Пакет, содержащий классы с выводом всего пользовательского интерфейса, будь это страницы или отдельные компоненты
Editors	Пакет, содержащий классы для вывода на экран компонент для редактирования HTML-шаблонов
TemplatesView	Пакет, содержащий классы для вывода типовых объектов на экране пользователя

3.2.2.5. Детальная диаграмма классов пакета «Object Detector»

На рис.59 представлена диаграмма классов пакета «Object Detector», а в табл.17 представлено подробное описание классов данного пакета.

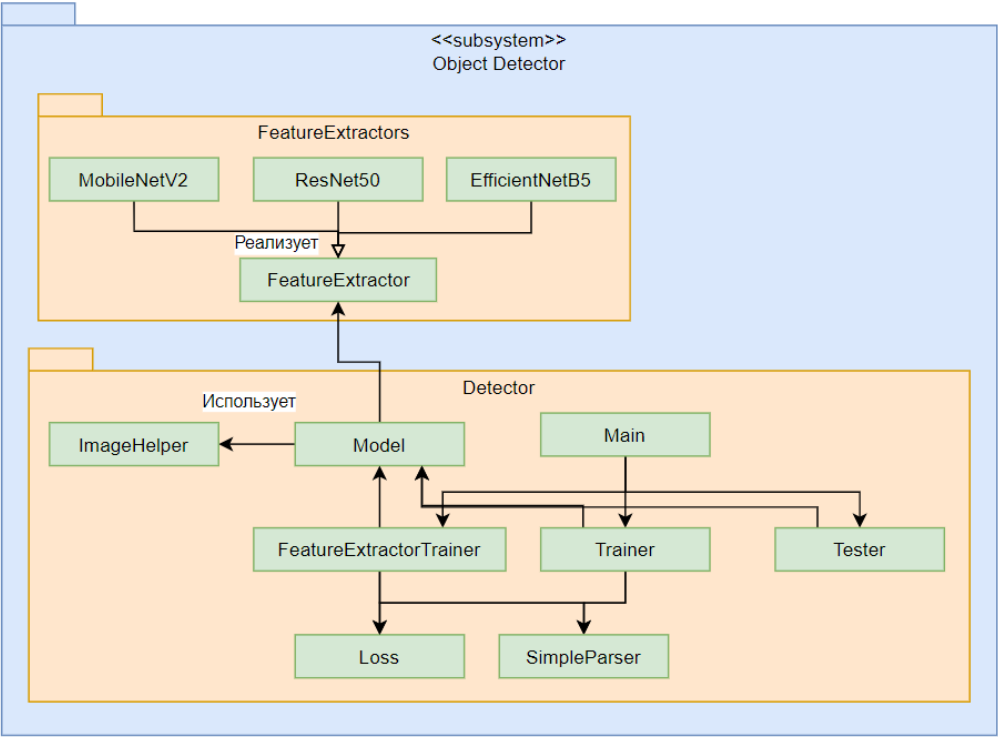


Рис. 59. Диаграмма классов пакета «Object Detector»

Таблица 17

Классы пакета «Object Detector»

Название	Описание
1	2
MobileNetV2	Класс для распознавания через архитектуру MobileNetV2
ResNet50	Класс для распознавания через архитектуру ResNet50
EfficientNetB5	Класс для распознавания через архитектуру EfficientNetB5
FeatureExtractor	Абстрактный базовый класс, применяющий одну из архитектур для распознавания
ImageHelper	Класс, реализующий вспомогательные функции для работы с изображениями
Model	Класс модели нейронной сети
FeatureExtractorTrainer	Класс, реализующий тренировочный цикл нейронной сети для извлечения признаков с изображения
Loss	Класс, реализующий функцию потерь нейронной сети
Main	Класс-точка входа в программу
Trainer	Класс, реализующий тренировочный цикл нейронной сети для распознавания объектов на изображении и их классов

Продолжение табл.17

1	2
Tester	Класс, реализующий проверку на тестовых данных
SimpleParser	Класс для парсинга из .txt-файла изображений и объектов на них

3.2.3. Разработка физической модели

3.2.3.1. Построение диаграмм компонентов

Диаграмма компонентов – это вид диаграммы в UML, который используется для моделирования структуры и отношений между компонентами системы. Компоненты, в данном контексте, могут включать как физические компоненты (например, библиотеки, исполняемые файлы, исполняемые классы), так и логические компоненты (например, подсистемы, модули, библиотеки).

На диаграмме компонентов каждый компонент представлен в виде прямоугольника с именем компонента. Он также может содержать интерфейсы, порты и зависимости между компонентами.

Цель диаграммы компонентов - показать структуру системы, выделить компоненты и их взаимосвязи, помочь при архитектурном проектировании, анализе и документировании системы. Диаграммы компонентов очень полезны для моделирования сложных систем, поскольку они помогают увидеть взаимодействие между компонентами и их распределение по различным уровням системы. Это помогает архитекторам и разработчикам лучше понять систему и ее компоненты, а также облегчает коммуникацию между участниками проекта [38].

Планируется использовать следующие компоненты, как показано на рис.60.

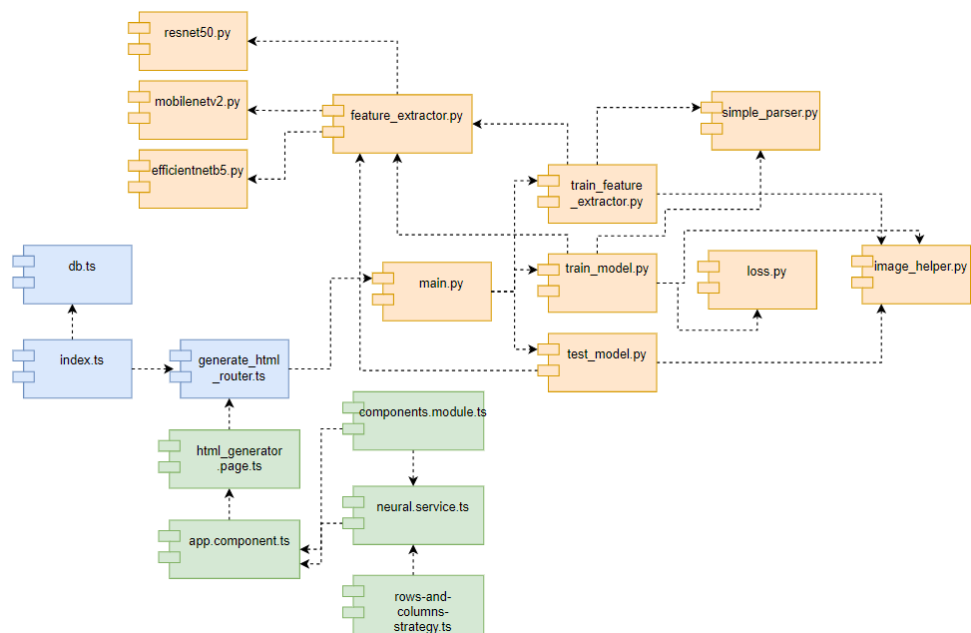


Рис. 60. Диаграмма компонентов

На обозначенной диаграмме компонентов обозначены компоненты, относящиеся к нейросети, написанной на языке программирования Python с расширением «.py». Также обозначены компоненты, относящиеся к серверной части, которая, в свою очередь, написана на языке TypeScript. Обозначены компоненты, которые относятся к клиентской части, которая написана на языке TypeScript с использованием фреймворка Angular. Для того, чтобы не перегружать данную диаграмму, на ней не указаны компоненты, которые импортирует компонента «components.module.ts», она импортирует все компоненты, отвечающие за формирование HTML в соответствии со своим распознанным классом.

В табл.18 указан список компонент, отображаемых на диаграмме компонент.

Таблица 18

Компоненты

Название	Описание
1	2
index.ts	Основной файл серверного приложения

Продолжение табл.18

1	2
db.ts	Файл для соединения сервера с базой данных
generate_html_router.ts	Файл сервера, предоставляющий API-маршруты для генерации шаблонов
app.component.ts	Основной файл клиентского приложения
html_generator.page.ts	Страница с генерацией HTML-шаблонов
main.py	Основной файл детектора объектов на рукописном рисунке-макете
train_model.py	Файл для обучения модели
test_model.py	Файл для тестирования модели
train_feature_extractor.py	Файл для обучения извлекателя признаков
loss.py	Файл, реализующий функцию потерь для нейросети
image_helper.py	Файл, предоставляющий утилитарные функции для работы с изображениями
simple_parser.py	Простейший парсер изображений и объектов на них, работает с файлами формата .txt
feature_extractor.py	Файл, предоставляющий реализацию одной из возможных архитектур СНС для извлечения признаков
resnet50.py	Файл, реализующий архитектуру СНС resnet50
mobilenetv2.py	Файл, реализующий архитектуру СНС mobilenetv2
efficientnetb5.py	Файл, реализующий архитектуру СНС efficientnetb5

3.2.3.2. Построение диаграмм размещения

Диаграммы размещения (или развертывания) используются для моделирования физической архитектуры системы. Эти диаграммы отображают аппаратные узлы, программные модули и связи между ними [39].

На диаграмме размещения компоненты системы представлены в виде узлов (nodes), которые могут быть физическими устройствами (например, серверы, компьютеры, мобильные устройства) или логическими узлами (например, сегменты сети, программные среды исполнения). Компоненты,

представленные на диаграмме, могут быть размещены на узлах или связаны с ними.

Цель диаграммы размещения - показать физическую конфигурацию системы, включая развертывание ее компонентов и их связей с физическими устройствами. Диаграммы размещения помогают визуализировать архитектурные решения, такие как конфигурации сетевых устройств, оборудование для хранения данных, серверы и другие компоненты системы, что обеспечивает лучшее понимание архитектуры всей системы и ее устройства.

Эта диаграмма особенно полезна в контексте разработки и планирования развертывания системы, а также в управлении инфраструктурой и оборудованием.

На диаграмме размещения можно увидеть, на каких устройствах ПО будет запускаться физически (рис.61).

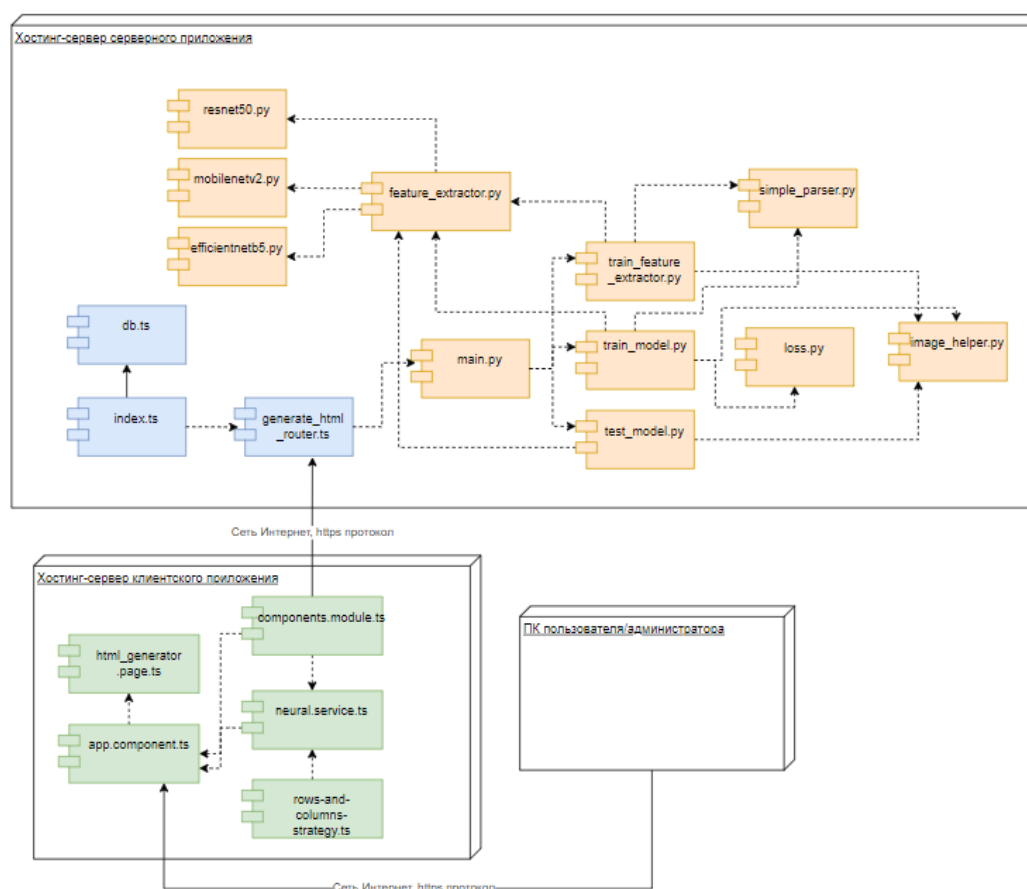


Рис. 61. Диаграмма размещения

Диаграмма размещения состоит из трёх блоков (так как это веб-приложение):

- ПК пользователя/администратора – персональный компьютер, с которого пользователь заходит в браузер по адресу веб-приложения;
- Хостинг-сервер клиентского приложения – удалённый ПК, доступный по определённому адресу, который по https-запросам осуществляет отрисовку клиентского приложения в браузере пользователя;
- Хостинг-сервер серверного приложения – предоставляет API-маршруты, с помощью которых можно получить сгенерированный HTML-шаблон, отправив по этому маршруту изображение с клиентской части.

3.2.4. Разработка интерфейсов

Графический интерфейс (GUI) – это способ взаимодействия пользователя с компьютерной программой с помощью визуальных элементов, таких как кнопки, поля для ввода текста, меню и диаграммы. Графический интерфейс создает удобное и интуитивно понятное окружение для взаимодействия пользователя с программой.

Графический интерфейс строится с целью облегчить пользовательскую работу с программой, разделить функционал на логические блоки, предоставить визуальные средства для ввода, представления данных и управления программными функциями. GUI упрощает и улучшает восприятие пользователем информации, а также уменьшает необходимость запоминания команд и иерархии программных функций.

3.2.4.1. Построение граф диалога

Граф-диалога – это структурный граф, используемый для представления взаимодействия между пользователями и компьютерной системой при

проектировании пользовательского интерфейса. Граф-диалога позволяет визуально отобразить различные сценарии взаимодействия пользователя с системой, а также определить возможные пути и варианты поведения пользовательского интерфейса.

Граф-диалога состоит из узлов и стрелок. Узлы представляют собой различные состояния системы или пользовательских действий, такие как ввод данных, нажатие кнопок, выбор опций и т.д. Стрелки указывают на возможные переходы между состояниями и действиями.

В процессе проектирования пользовательского интерфейса граф-диалога помогает определить все возможные пути взаимодействия пользователя с системой, учесть различные варианты ответов пользователя на различные ситуации, а также осуществить проверку на корректность и полноту дизайна интерфейса.

Граф-диалога является важным инструментом для разработки пользовательского интерфейса, поскольку обеспечивает понимание того, как пользователю будет взаимодействовать с системой в различных ситуациях, и позволяет учесть все возможные варианты поведения. Граф диалога работы с программой указан на рис.62.

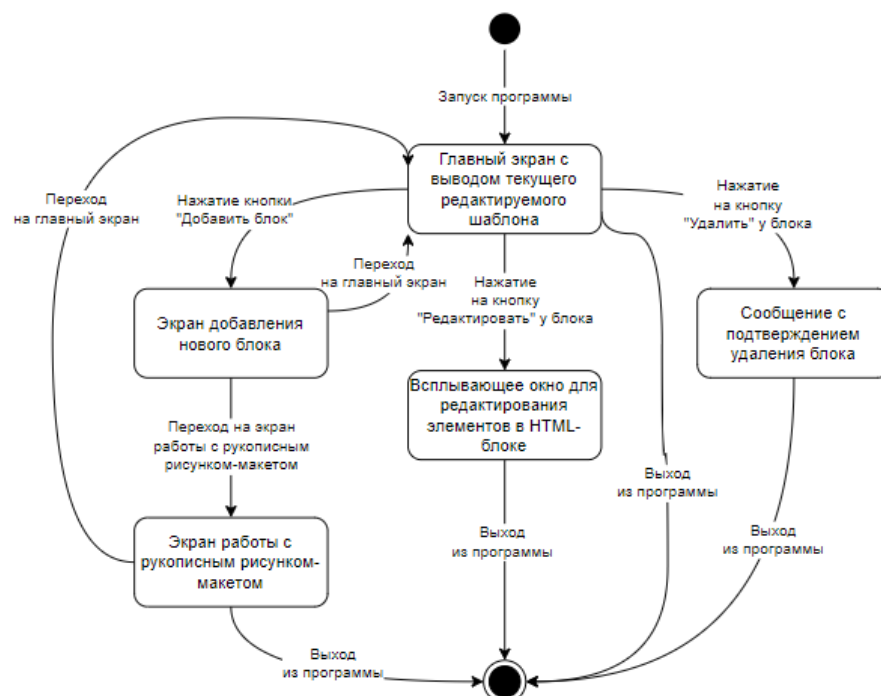


Рис. 62. Граф диалога работы с программой

3.2.4.2. Разработка форм ввода-вывода информации

Графический интерфейс пользователя формируется из окон, блоков диалога и различных элементов управления. Графические пользовательские интерфейсы поддерживаются операционными системами Windows и т. д. В рамках указанных операционных систем для таких интерфейсов разработаны наборы стандартных компонентов взаимодействия с пользователем. Эти наборы не идентичны, как и основные приемы работы с интерфейсами различных операционных систем.

Пользовательские интерфейсы большинства современных программ строятся по технологии WIMP: W - Windows (окна), I - Icons (пиктограммы), M - Mouse (мышь), P - Pop-up (всплывающие или выпадающие меню). Основными элементами графических интерфейсов, таким образом, являются: окна, пиктограммы, компоненты ввода-вывода и мышь, которую используют в качестве указующего устройства и устройства прямого манипулирования объектами на экране.

Сами формы ввода-вывода для клиентской части информационноой системы будут строиться с помощью языка разметки HTML, а также с помощью таблицы стилей CSS.

HTML – это язык гипертекстовой разметки документов. Он применяется во всем мире. Браузер интерпретирует код HTML для отображения его на компьютере, планшете или телефоне. CSS используется для определения стилей (правил) оформления документов — включая дизайн, вёрстку и вариации макета для различных устройств и размеров экрана.

Таким образом, с помощью этих двух средств можно разработать пользовательский интерфейс, который будет соответствовать любому дизайну.

К основным элементам интерфейса разрабатываемой программы можно отнести:

1. Главное окно (рис.63)

Основное окно программы включает в себя элементы для просмотра текущего редактируемого HTML-шаблона, присутствует кнопка для добавления нового блока, при добавлении блоков у каждого блока будет возможность удаления или редактирования.

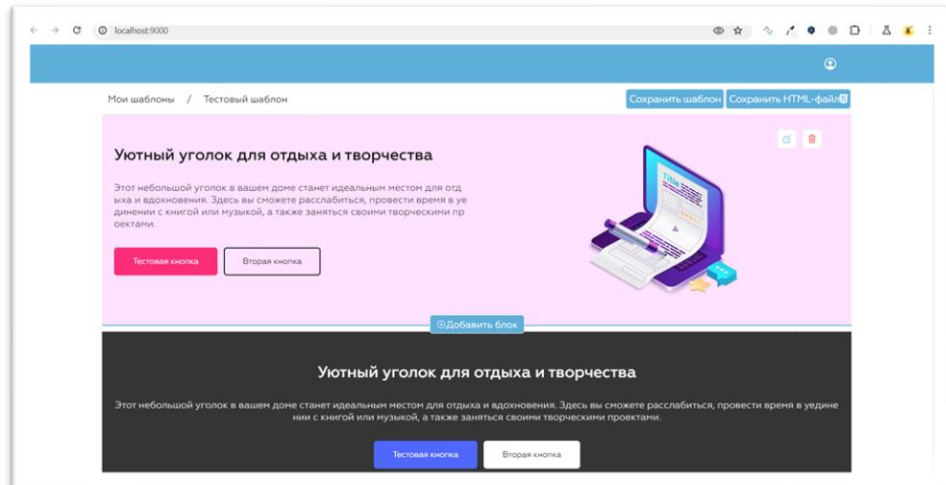


Рис. 63. Главное окно программы

2. Всплывающее окно для удаления блока (рис.64)

При нажатии на кнопку корзины у блока в правом верхнем углу будет открываться всплывающее окно с подтверждением удаления блока, удаление блока можно отменить, нажав на кнопку «Отмена», либо же подтвердить, нажав на кнопку «Удалить», в результате чего блок будет удалён.

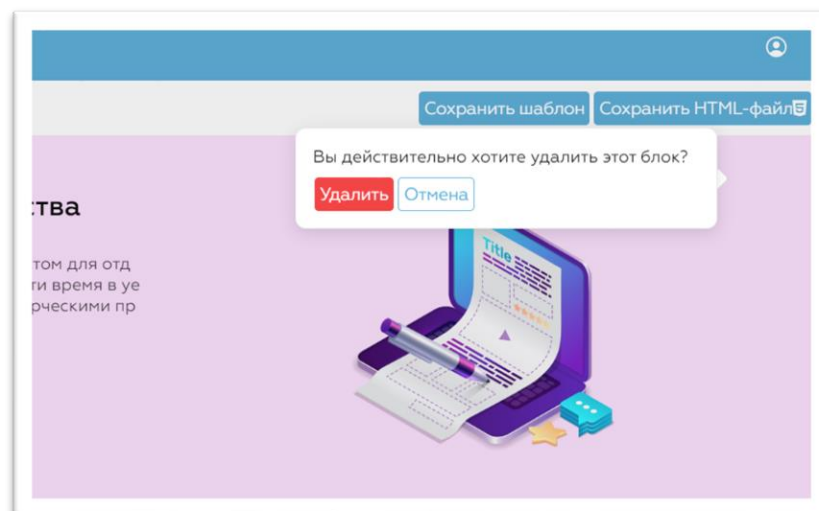


Рис. 64. Всплывающее окно для удаления блока

3. Всплывающее окно добавления нового блока (рис.65)

Для добавления нового блока необходимо на главном окне программы нажать кнопку «Добавить блок», после чего появится всплывающее окно для добавления заранее подготовленного шаблона. В левой части всплывающего окна можно выбрать категорию шаблона. После нажатия левой кнопкой мыши на выбранный шаблон – всплывающее окно закроется и выбранный шаблон отобразится на главном окне программы.

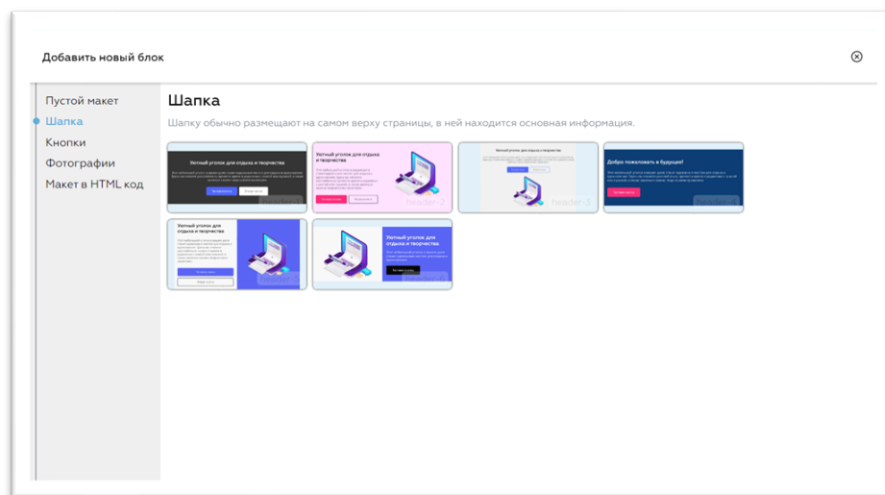


Рис. 65. Всплывающее окно для добавления блока

4. Окно работы с рукописным рисунком-макетом (рис.66)

Для того, чтобы перейти к окну работы с рукописным рисунком-макетом, необходимо открыть всплывающее окно для добавления нового блока и в левом меню перейти в раздел «Макет в HTML код».

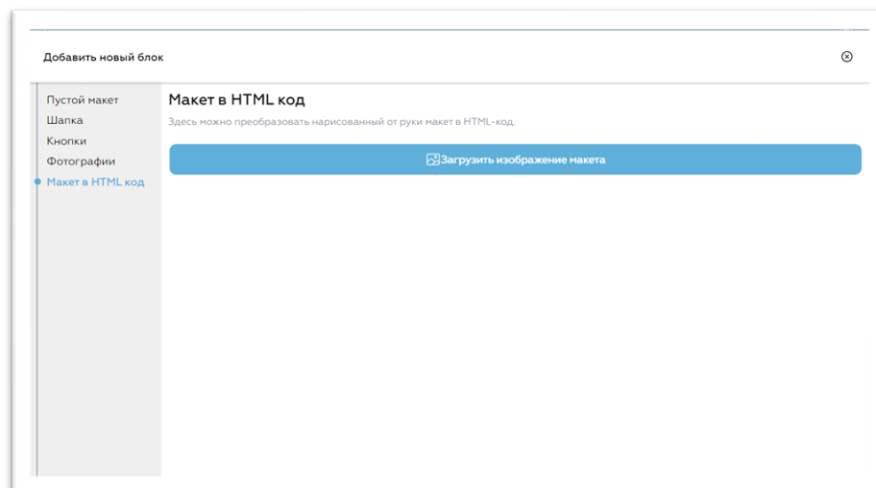


Рис. 66. Окно работы с рукописным рисунком-макетом

При нажатии на кнопку «Загрузить изображение макета» будет открыто окно для загрузки изображения (рис.67).

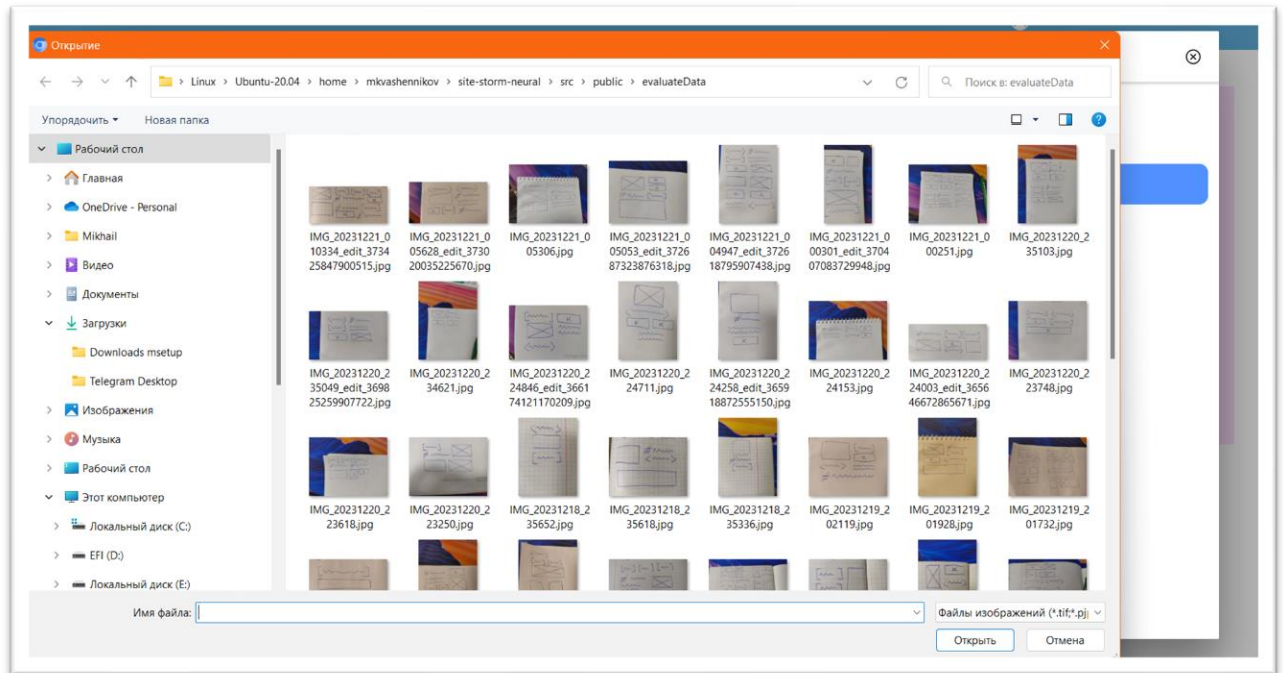


Рис. 67. Окно загрузки изображения

Если изображение будет загружено успешно, то выбранное изображение отобразится в текущем открытом окне (рис.68).

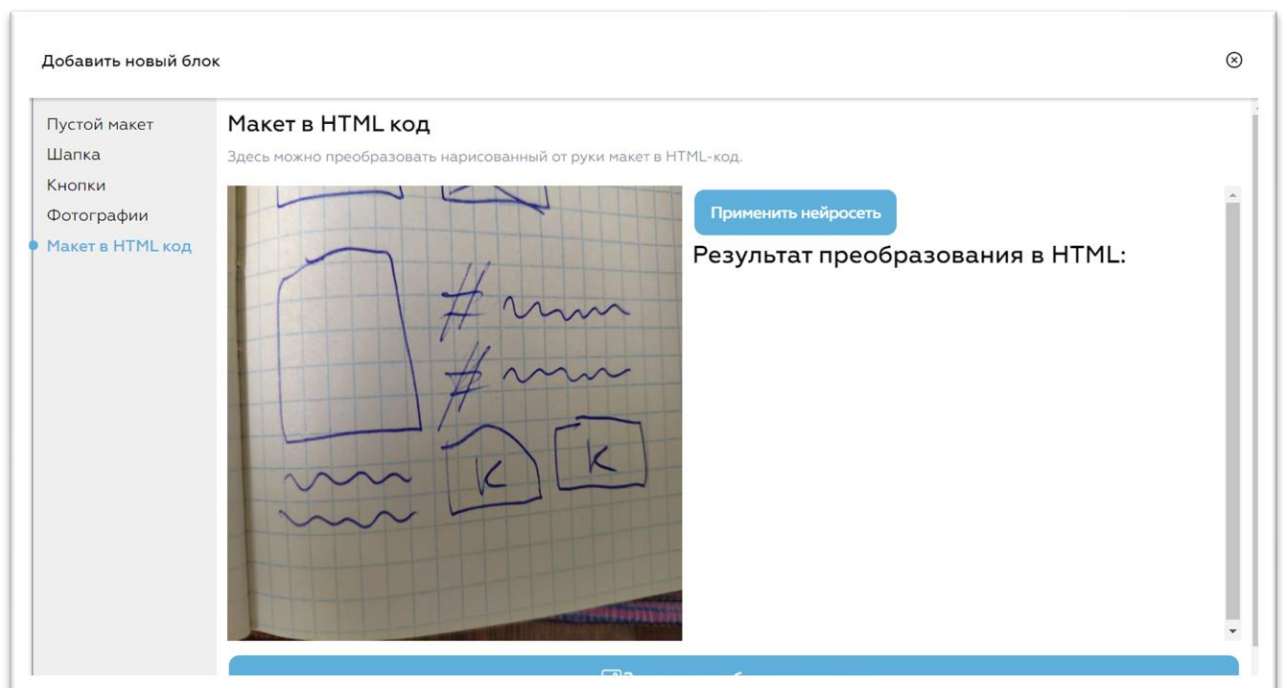


Рис. 68. Успешная загрузка рукописного рисунка-макета

При нажатии на кнопку «Применить нейросеть» в левой части для изображения будут выведены ограничительные рамки для тех объектов, которые распознала нейросеть. В правой части открытого окна под заголовком «Результат преобразования в HTML» будет выводиться внешний вид сгенерированного макета, а также его HTML-код, также можно просмотреть результаты метрик (то есть характеристики соответствия положению и размерам предсказанных объектов на изображении и в сгенерированном макете) (рис.69-71).

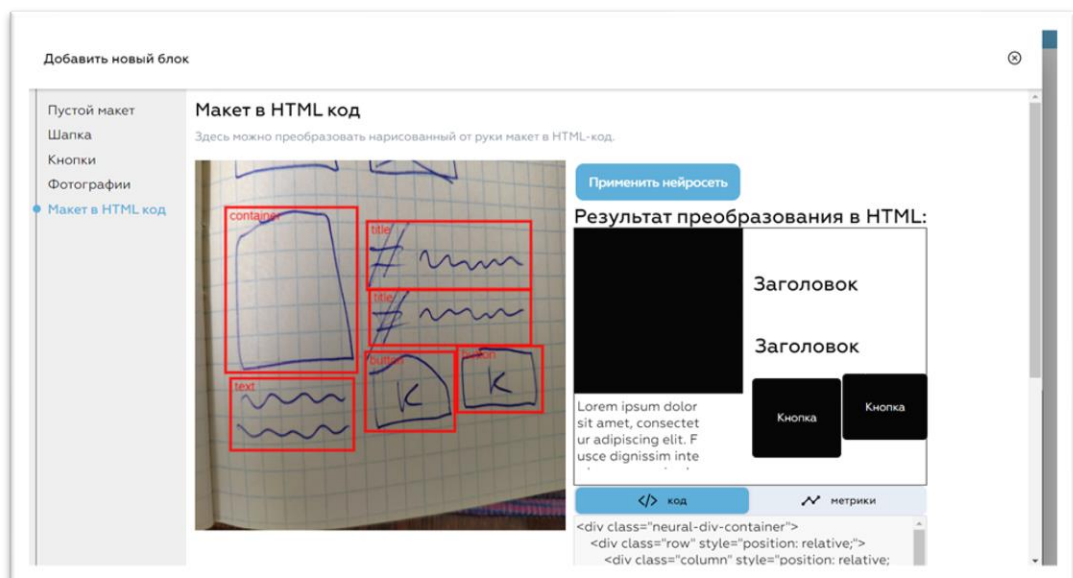


Рис. 69. Результат преобразования в HTML

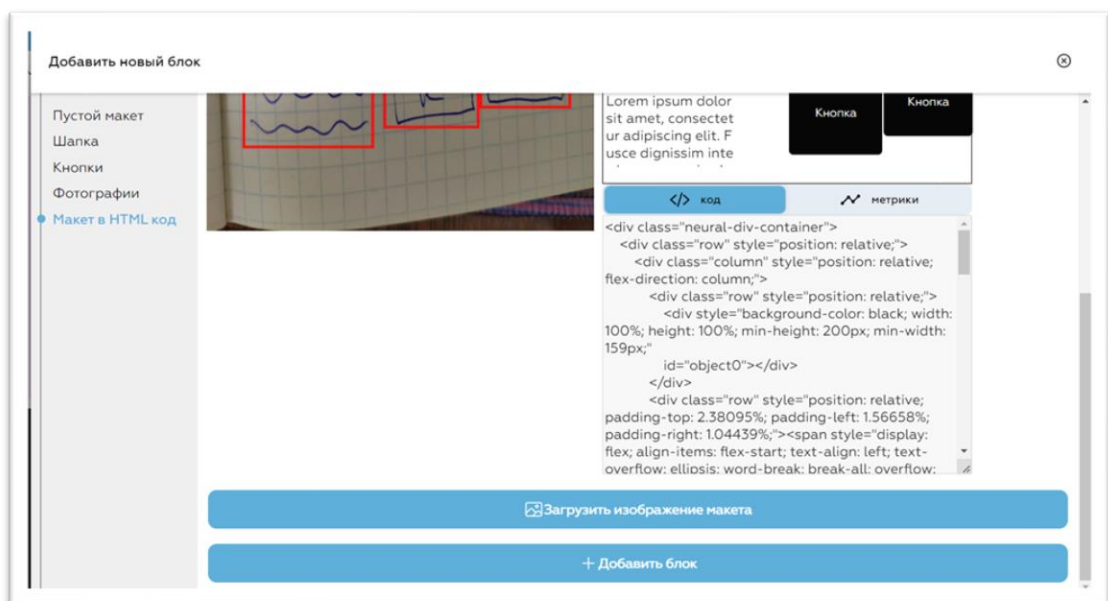


Рис. 70. Окно сгенерированного HTML-кода

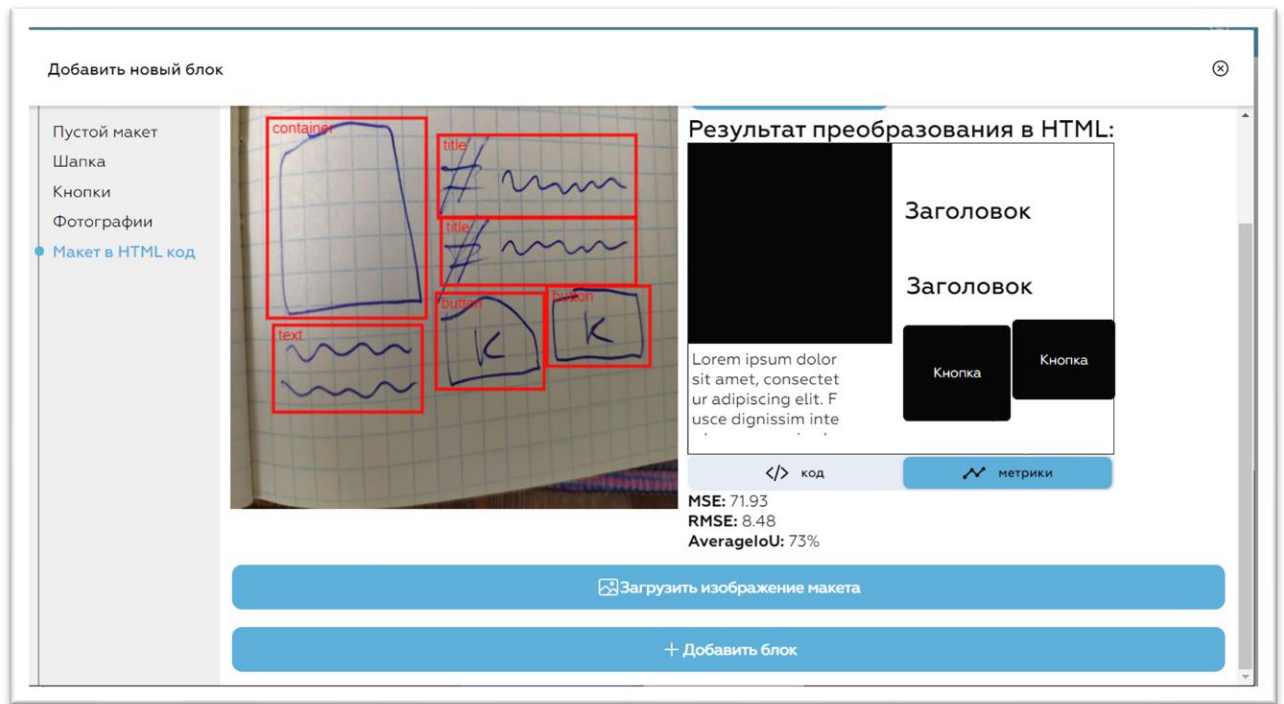


Рис. 71. Получившиеся результаты метрик

5. Всплывающее окно для редактирования элементов блока (рис.72)

При нажатии у любого блока кнопки «Редактировать» из правой части главного окна программы будет появляться анимированное всплывающее окно с возможностью редактирования элементов текущего редактируемого блока.

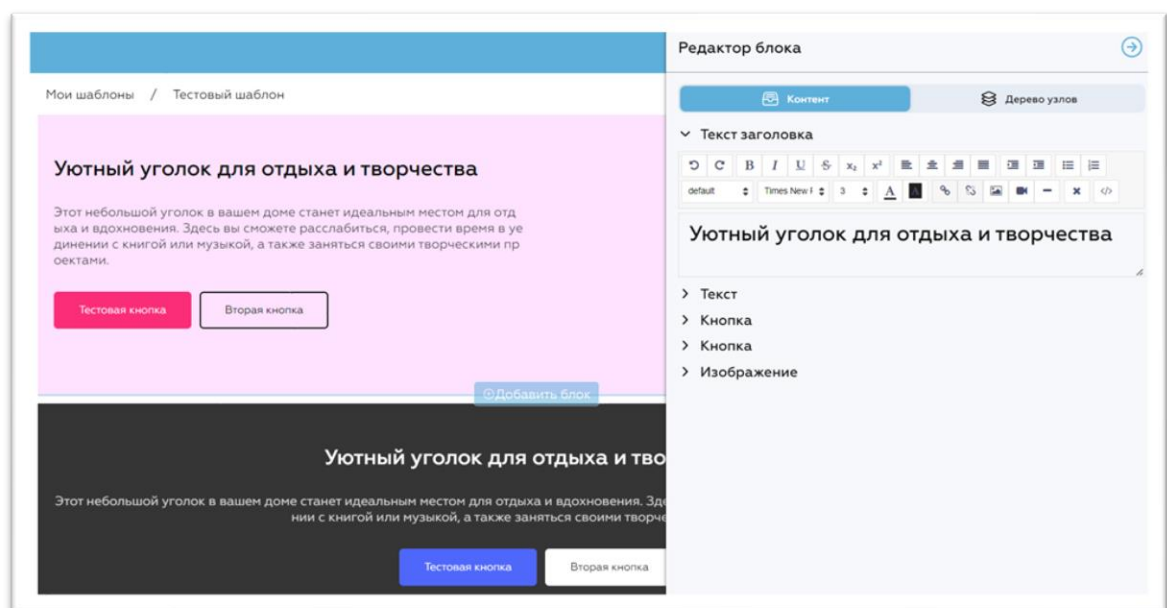


Рис. 72. Окно редактирования блока

6. Управление деревом узлов (рис.73).

В разделе «Редактор блока» при нажатии на кнопку «Дерево узлов» - можно просмотреть из каких узлов состоит текущий блок, эти узлы также можно редактировать, скрывать, удалять, а также внутри узла можно добавить новый узел. При наведении курсора мыши на любой из узлов – на главном окне этот узел визуально подсвечивается красным прямоугольником, при нажатии на узел при наличии дочерних элементов – он раскрывается и можно просматривать дочерние узлы.

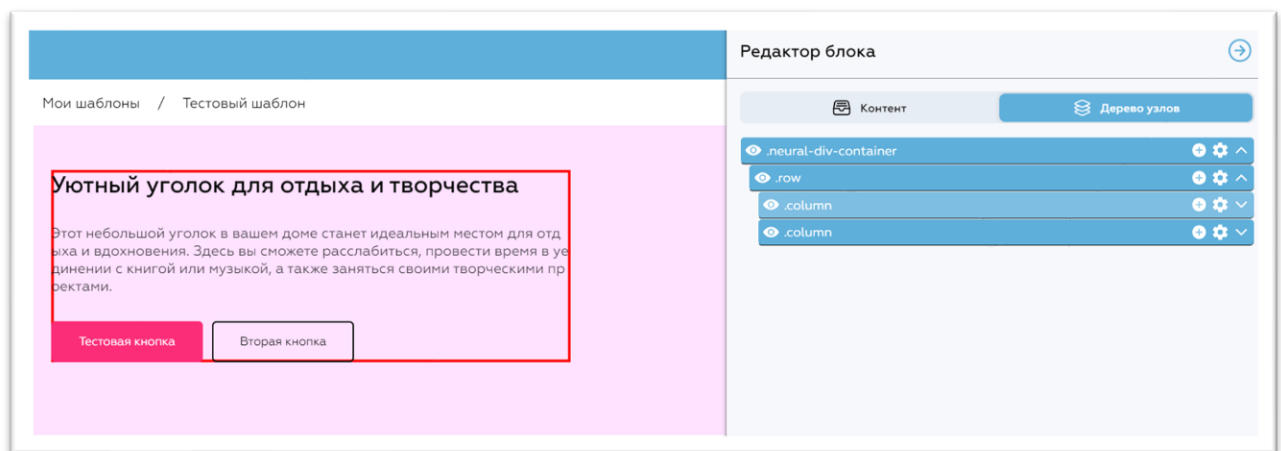


Рис. 73. Управление деревом узлов

При нажатии на иконку «шестеренки» у узла – можно редактировать текущий узел (рис.74).

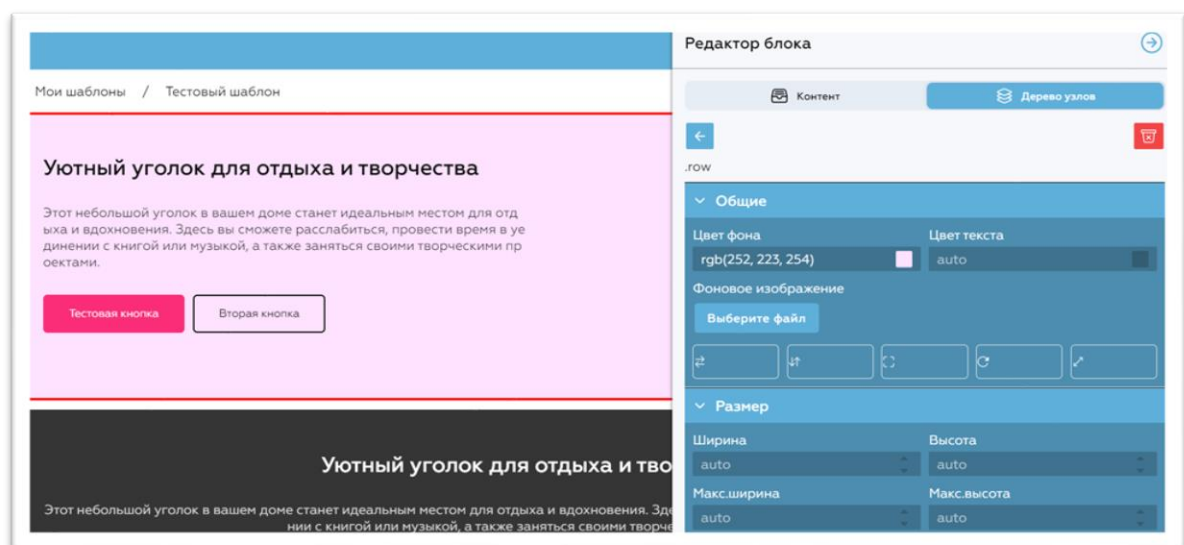


Рис. 74. Редактирование выбранного узла

3.3. Выводы по разделу 3

В данном разделе выполнены все этапы проектирования разрабатываемого программного обеспечения на основе объектно-ориентированного подхода к анализу и проектированию систем. В качестве средств реализации использованы Angular, объектно-ориентированный язык программирования TypeScript и среда разработки приложений Visual Studio Code. Руководство пользователя для запуска и использования программы указано в Прил.3.

На этапе концептуального моделирования разработаны модели «черного ящика», состава, структуры «белого ящика» системы. На этапе проектирования информационного обеспечения был произведен выбор подхода к разработке и модели жизненного цикла программного обеспечения, а также используемых инструментов.

Были разработаны диаграммы потоков данных в системе. С использованием унифицированного языка моделирования UML, разработаны логическая модель системы: диаграмма использования, контактная диаграмма классов, структуры данных, диаграммы пакетов, диаграммы классов пакетов и физическая модели системы. В ходе разработки интерфейса были описаны графы диалога пользователя и все формы ввода и вывода информации для генерирования HTML-шаблонов по рукописным рисункам-макетам.

4. Результаты экспериментальной проверки программного обеспечения генерирования HTML-шаблонов по рукописным рисункам-макетам

4.1. Описание эксперимента и экспериментальных данных

Для подтверждения работы разработанного алгоритма генерирования HTML-сетки после этапа детектирования объектов, необходимо сравнить работу алгоритма с существующими алгоритмами, которые используются у конкурирующих решений. На текущий момент, все аналогичные решения подобной задачи являются зарубежными (Sketch2Code, TeleportHQ, ChatGPT-4), первый сервис вообще на текущий момент не работает, а остальные два не работают на территории РФ, поэтому, для сравнения этих решений с разработанным алгоритмом, необходимо воспользоваться скриншотами работы аналогичных решений из официально опубликованных видеороликов, пример подобного изображения показан на рисунке (рис.75).

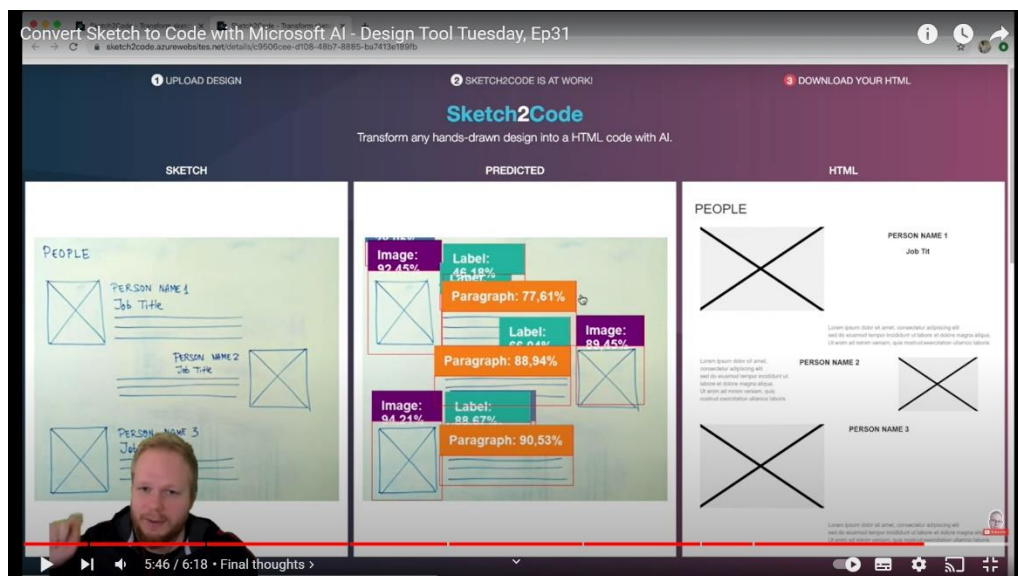


Рис. 75. Один из скриншотов работы аналогичного решения

Поскольку фрагменты генерируемого HTML-кода получить невозможно из зарубежных сервисов, то необходимо проводить сравнение по тому, какой результат расположения и размеров объектов формируют эти аналоги. То есть, будем сравнивать, насколько разработанный алгоритм добивается лучшего расположения и соответствия размеров элементов относительно исходного

изображения рукописного рисунка-макета (для этого будем использовать соответствующие метрики).

Таким образом, экспериментальными данными будут выступать изображения рукописных рисунков-макетов, объекты на которых соответствуют классам, которые были взяты за стандарт в рамках данной работы (раздел 2.1 табл.1) (рис.76).

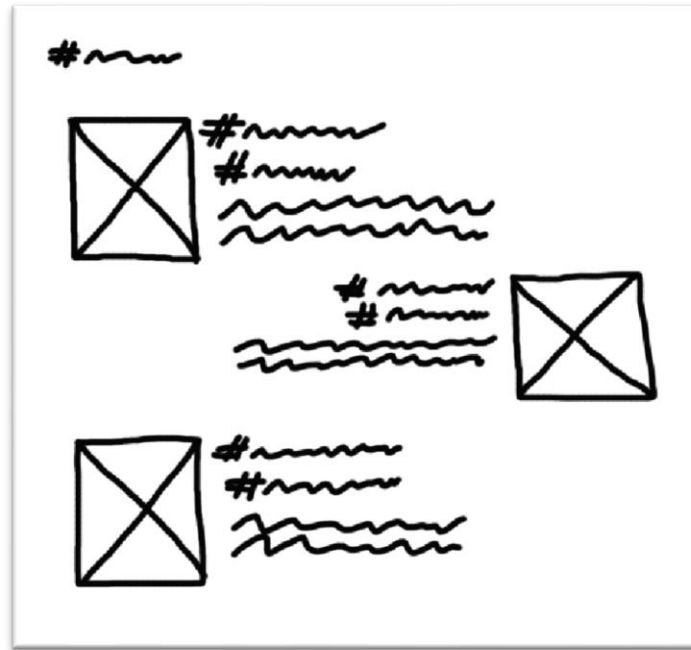


Рис. 76. Один из примеров экспериментальных данных

При этом, при сравнении с аналогами, меняем внешний вид класса на тот, который определен в рамках данной работы в случае, если у аналога внешний вид класса выглядит по-другому (например, класс «изображение» у аналога может выглядеть совершенно иначе), при этом сохраняем положение и размеры объекта идентичными искомому, чтобы адекватно сравнивать работу алгоритмов.

Для проведения экспериментальных исследований, в том числе включающих в себя обучение сверточной нейронной сети и получение результатов её работы на реальных данных, а также для запуска программы, реализующей алгоритм формирования HTML-сетки, необходимо

использовать ПК, который удовлетворяет следующим системным требованиям:

- Операционная система (OS) - Windows 11 с установленным дистрибутивом WSL2, либо Ubuntu 20.04;
- процессор (CPU) – не ниже Intel Core i5 с тактовой частотой 3.6 ГГц;
- видеокарта (GPU) - Nvidia Geforce GTX 1650 с объемом видеопамати не менее 4 Гб;
- оперативная память (RAM) объемом не менее 16 Гб, с частотой не менее 2200 ГГц.

Далее дадим формальное описание метрик, которые будут использоваться для оценки итогового положения и размера объектов.

- В качестве базовой метрики для оценки полученного результата будем использовать метрику **MSE** (Mean squared error) – эта метрика сообщает среднеквадратичную разницу между ожидаемыми и фактическими значениями в наборе данных, чем ее показатель ниже, тем лучше, ее формулу можно записать следующим образом:

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2; \quad (4)$$

- метрика **RMSE** (Root mean squared error) – является квадратным корнем из средней квадратичной разницы между ожидаемыми и фактическими значениями, чем ниже RMSE, тем лучше. Данная метрика будет показывать среднее отклонение в пикселях между ожидаемыми и фактическими значениями (формула 5).

$$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}; \quad (5)$$

- метрика **AverageIoU** (Average Intersection Over Union) – степень пересечения между двумя ограничивающими рамками, в данном случае сравниваются ожидаемые и фактические рамки для разных объектов с одного изображения и берется между ними среднее значение, чем больше результат, тем лучше, значения от 0 до 100% (формула 6).

$$\frac{\left(\sum_{i=1}^n \frac{\text{Area of Overlap}}{\text{Area of Union}}\right)}{n} * 100, \quad (6)$$

где Area of Overlap – площадь пересечения, Area of Union – площадь объединения, а n – количество объектов.

Фактически, сравнение работы алгоритмов сводится к сравнению размеров и положений прямоугольников, потому что каждый узел HTML-дерева является прямоугольным объектом, на который применили CSS-свойства.

Поскольку оцениваем именно алгоритм формирования HTML-сетки, то не берем во внимание те случаи, когда нейросеть не распознала какой-либо объект или игнорируем факт неверного распознавания, поскольку в данном разделе прежде всего оценивается работа алгоритма.

Так как работа алгоритма будет оцениваться в сравнении с зарубежными решениями по скриншотам – важно учесть тот факт, что скриншоты имеют разные размеры и в том числе объекты на них, поэтому изначально приводим каждое изображение к разрешению 448x448 и меняем размеры прямоугольников в соответствии с этим разрешением (то есть нормализуем положение и размеры объектов относительно разрешения 448x448), это разрешение выбрано не случайно – именно на таком разрешении изображений обучалась сверточная нейронная сеть.

Также следует обратить внимание, что при использовании описанных выше метрик для оценки результата – не стоит использовать изображение с большим количеством объектов, так как в таком случае при наличии

отклонений в верхних объектах – нижние начнут в том числе смещаться (это является свойством HTML-шаблона), и поэтому показатели метрик будут не такими информативными. В таком случае лучше поделить подобное изображение на несколько и сравнивать их отдельно.

4.2. Результаты экспериментальной проверки алгоритма генерирования HTML-шаблона

Поскольку зарубежные решения недоступны для тестирования, а формального описания алгоритмов их работы нет, поэтому для того, чтобы узнать насколько разработанное решение лучше конкурирующих решений – необходимо воспользоваться скриншотами с демонстрацией работы таких решений (таких скриншотов можно взять с официальных видеороликов с сайтов решений, но количество примеров ограничено).

Приведем пример, как будем выполнять измерения. Для начала берем скриншот рукописного рисунка с видео-демонстрации сервиса Sketch2Code (рис.77).

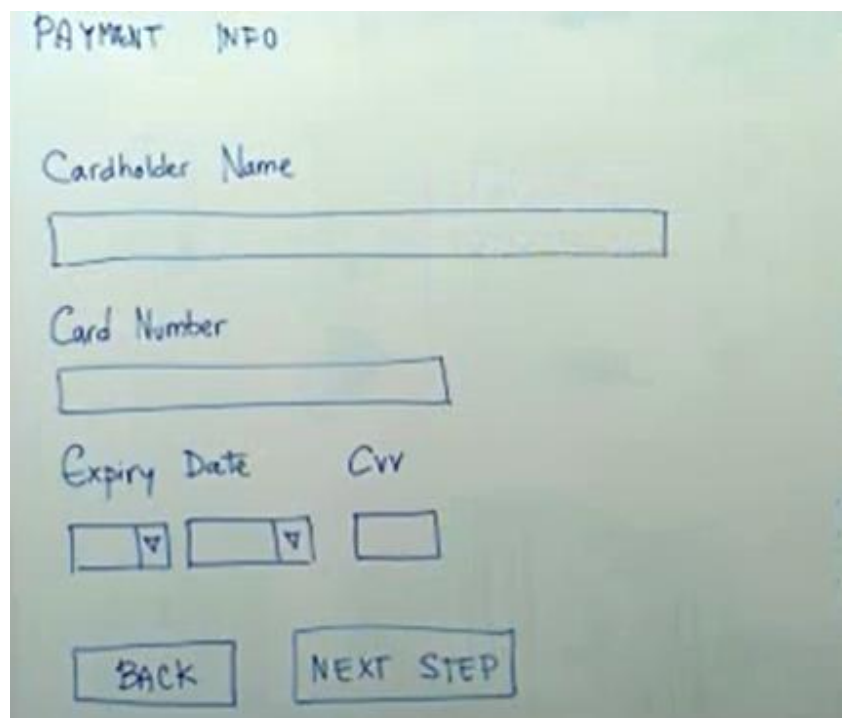


Рис. 77. Скриншот рукописного рисунка №1 из видео демонстрации сервиса Sketch2Code

После чего необходимо отрисовать новый рисунок на основе исходного, но заменить объекты на тот вид, как они определены в диссертации, при этом сохранив исходные размеры и положение объектов, чтобы результаты метрик были адекватны (рис.78).

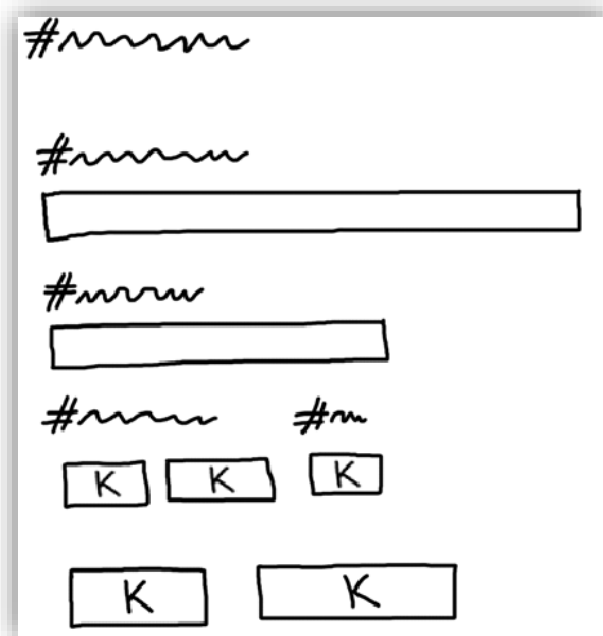


Рис. 78. Перерисованный рисунок в соответствии с нашим стандартом объектов

Далее берем скриншот результата формирования HTML-шаблона со стороннего сервиса (рис.79).



Рис. 79. Скриншот результата формирования HTML-шаблона стороннего сервиса №1

Даже на текущем этапе визуально можно отметить, что объекты потеряли выравнивание, а также находятся в произвольных местах и теряют исходный порядок отображения, при этом размеры также не учтены полностью, второе поле ввода текста, очевидно, имеет избыточную ширину.

Следующим шагом с помощью самостоятельно написанной утилитарной программы отмечаем объекты на исходном рисунке и на получившемся результате прямоугольниками (рис.80).

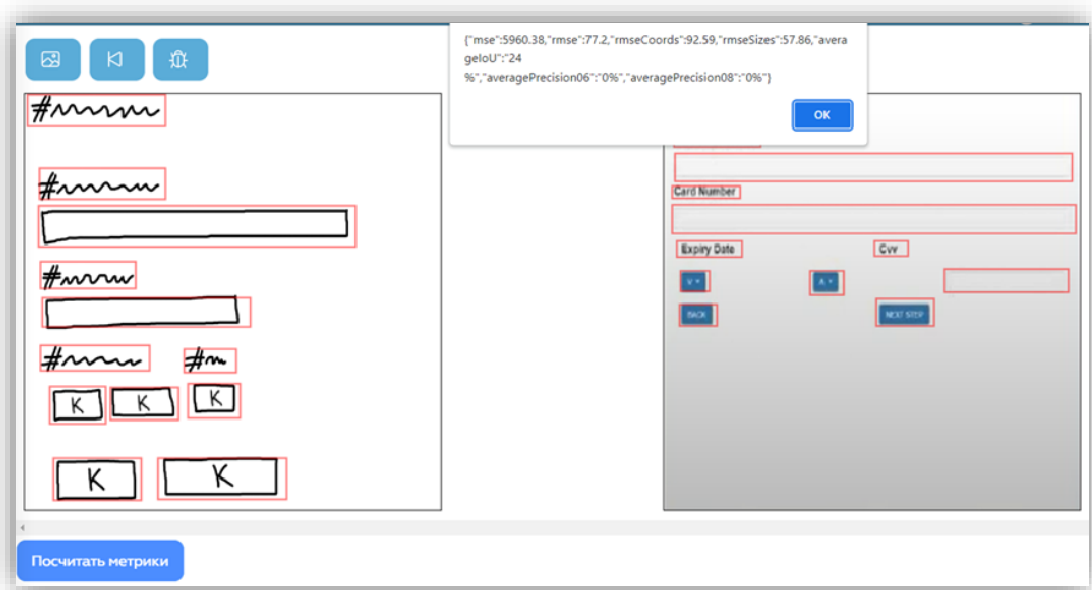


Рис. 80. Разметка исходных и распознанных объектов №1

После чего можно получить результаты метрик (табл.19).

Таблица 19

Результаты метрик оценки HTML-шаблона изображения №1 Sketch2Code

№	Название метрики	Результат
1	MSE	5960.38
2	RMSE	77.2
3	AverageIoU	24%

Исходя из полученных метрик, можно интерпретировать результат – в среднем для каждого объекта отклонение в размерах и положении составляет 77.2 единицы. В среднем исходные объекты совпадают по площади с объектами на шаблоне лишь на 24%.

Для того, чтобы сравнить этот результат с нашим алгоритмом – необходимо преобразовать рисунок в HTML в программе, при этом все метрики будут автоматически подсчитаны (в данном случае JS-скрипт получает размеры сгенерированных объектов и далее эти размеры сверяются с размерами объектов на макете). Получаем результат формирования шаблона с помощью нашего алгоритма (рис.81).

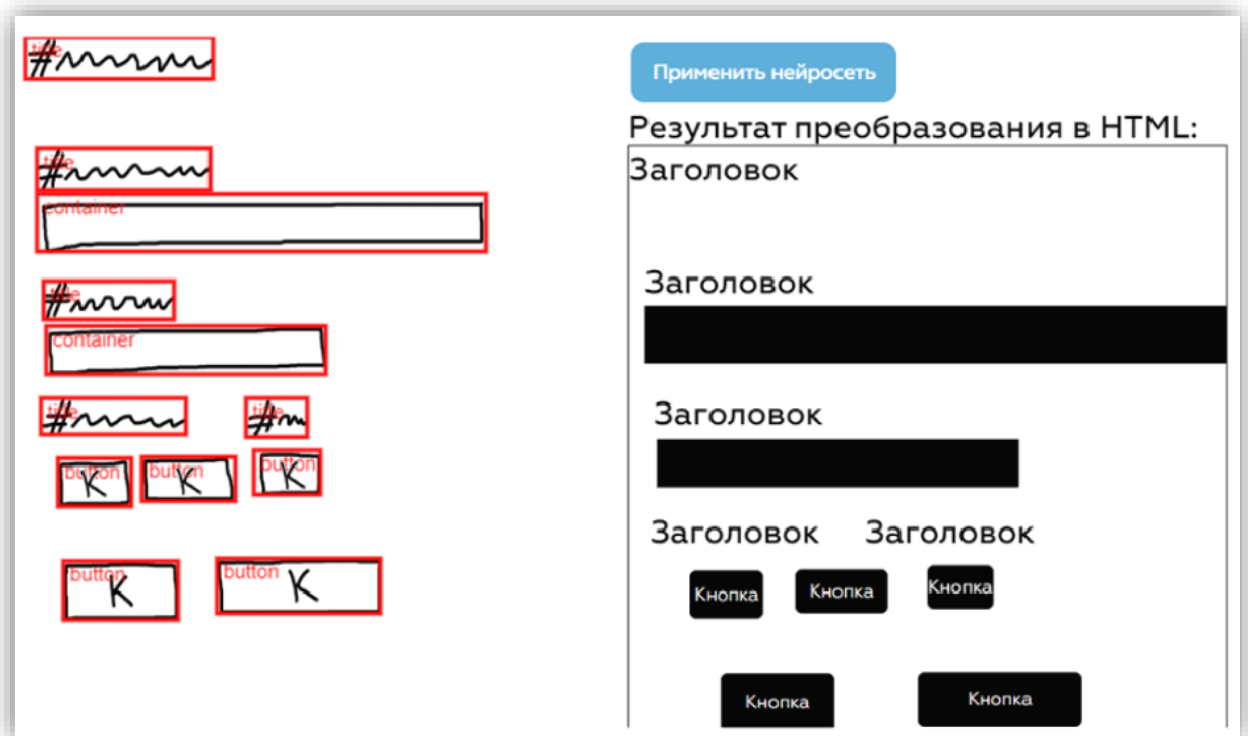


Рис. 81. Результат преобразования макета стороннего сервиса №1 в HTML
нашим алгоритмом

Таким образом, результаты метрик работы нашего алгоритма можно посмотреть в таблице (табл.20).

Таблица 20

Результаты метрик оценки HTML-шаблона изображения №1 нашим
алгоритмом

№	Название метрики	Результат
1	MSE	185.31
2	RMSE	13.61
3	AverageIoU	72%

Результат работы нашего алгоритма для изображения №1 – в среднем для каждого объекта отклонение в размерах и положении составляет 13.61 единицы. В среднем исходные объекты совпадают по площади с объектами на шаблоне на 72%.

Приведем гистограммы для оценки результатов метрик между нашим алгоритмом и сервисом sketch2Code для изображения №1 (рис.82-83).

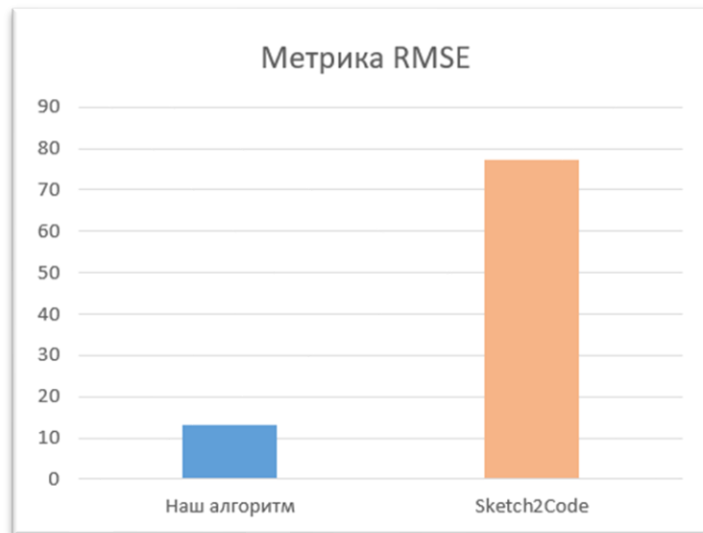


Рис. 82. Сравнение результатов метрики RMSE для изображения №1

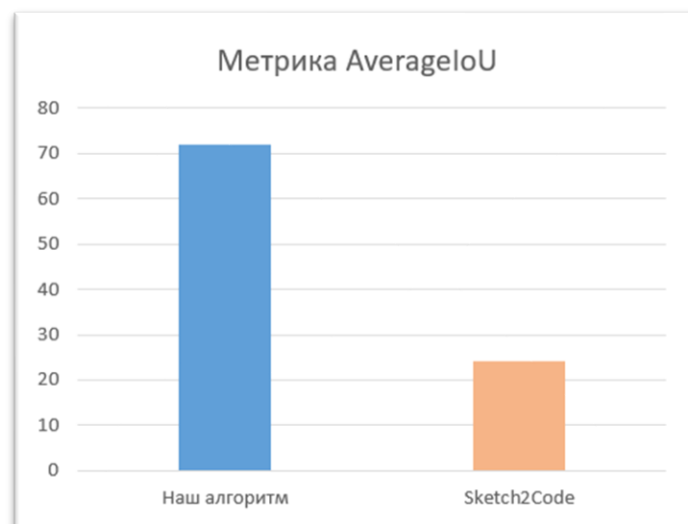


Рис. 83. Сравнение результатов метрики AverageIoU для изображения №1

Проделаем аналогичное сравнение для изображения №2 (рис.84-88).

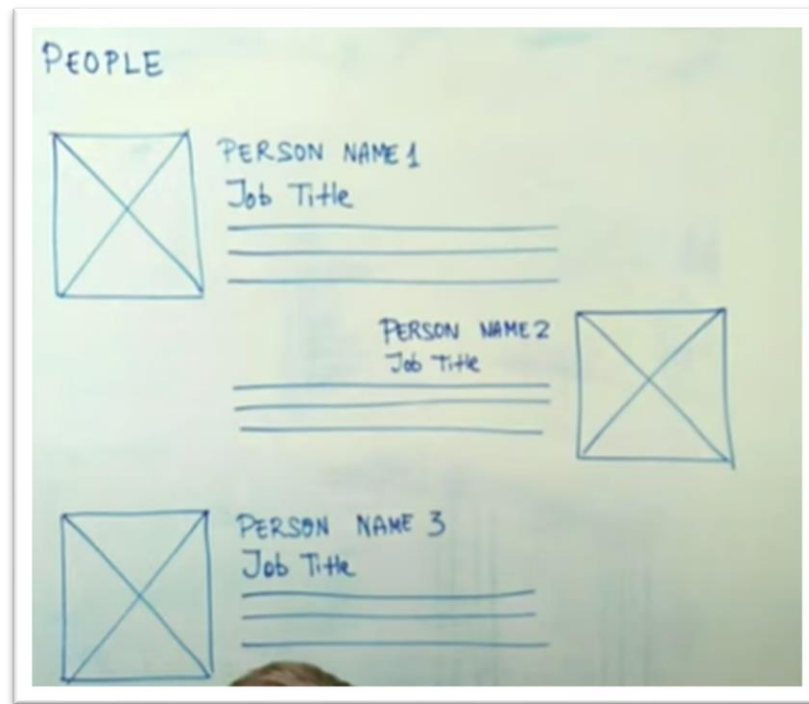


Рис. 84. Скриншот рукописного рисунка №2 из видео демонстрации сервиса Sketch2Code

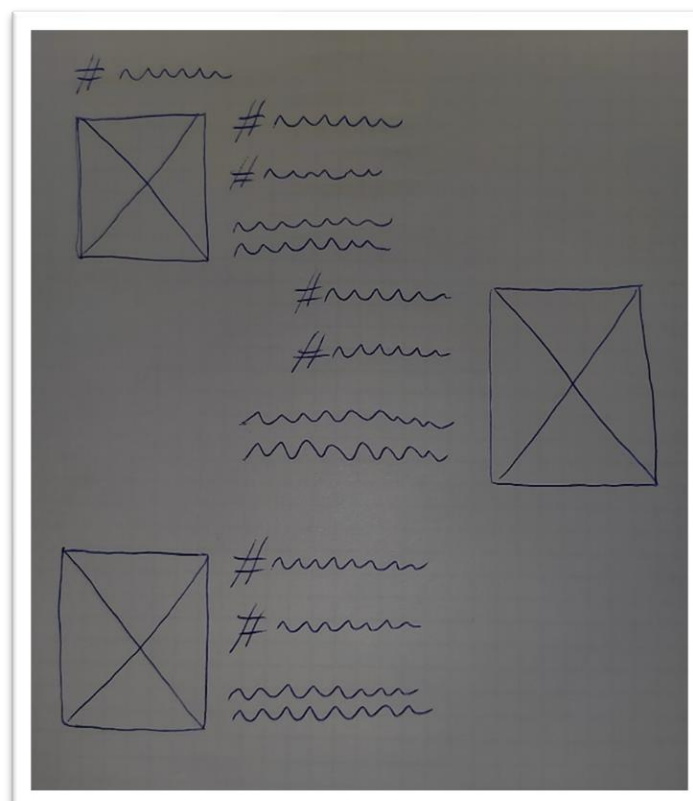


Рис. 85. Перерисованный рисунок в соответствии с нашим стандартом объектов

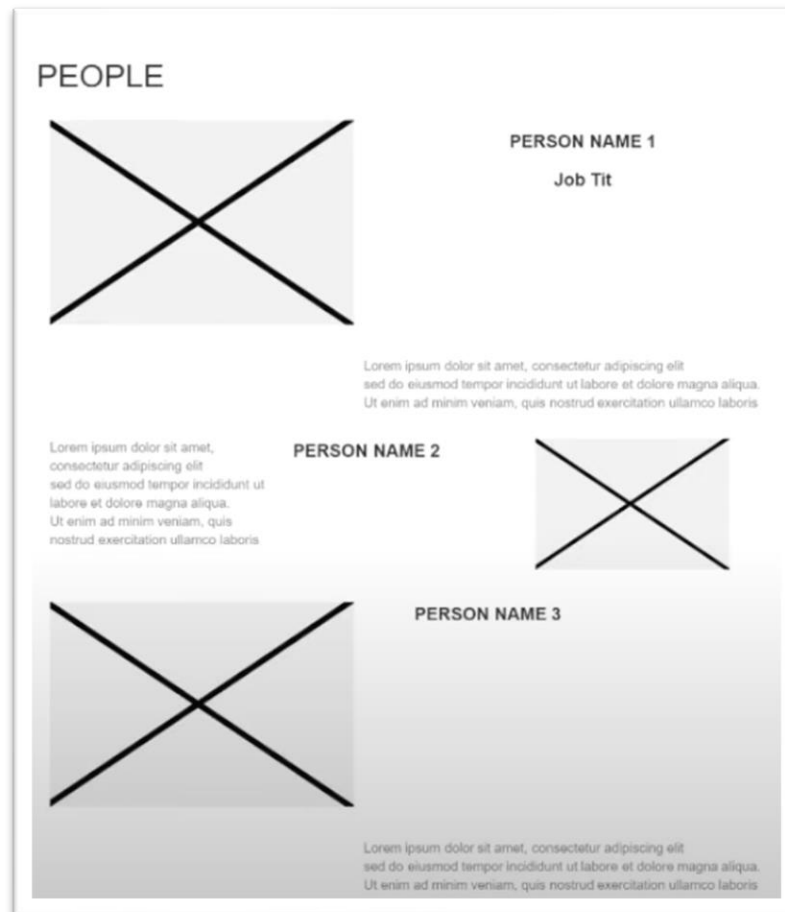


Рис. 86. Скриншот результата формирования HTML-шаблона стороннего сервиса №2

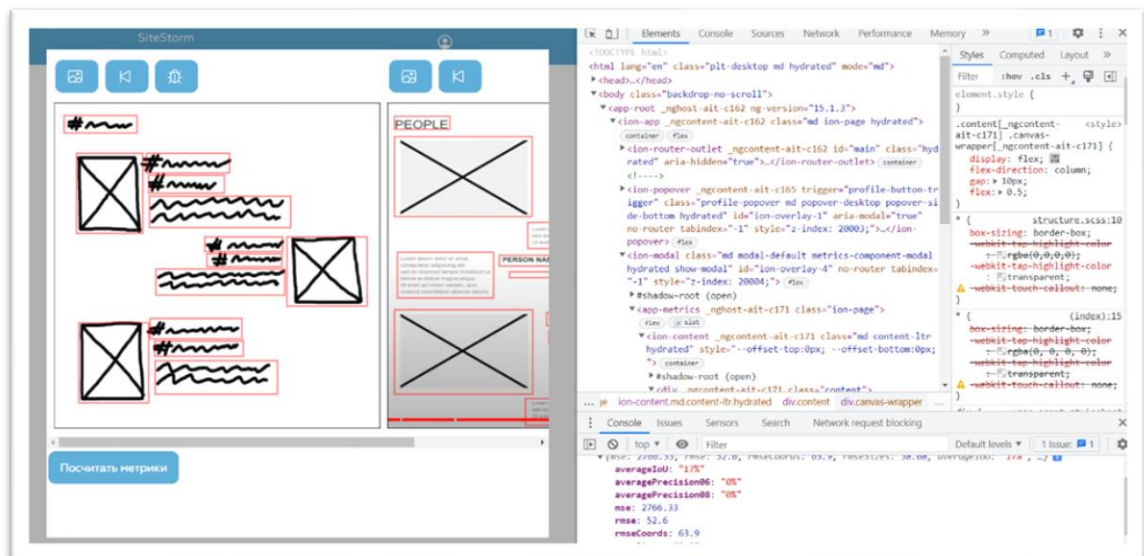


Рис. 87. Разметка исходных и распознанных объектов №2

После чего можно получить результаты метрик (табл.21-22).

Таблица 21

Результаты метрик оценки HTML-шаблона изображения №2 Sketch2Code

№	Название метрики	Результат
1	MSE	2766.33
2	RMSE	52.6
3	AverageIoU	17%

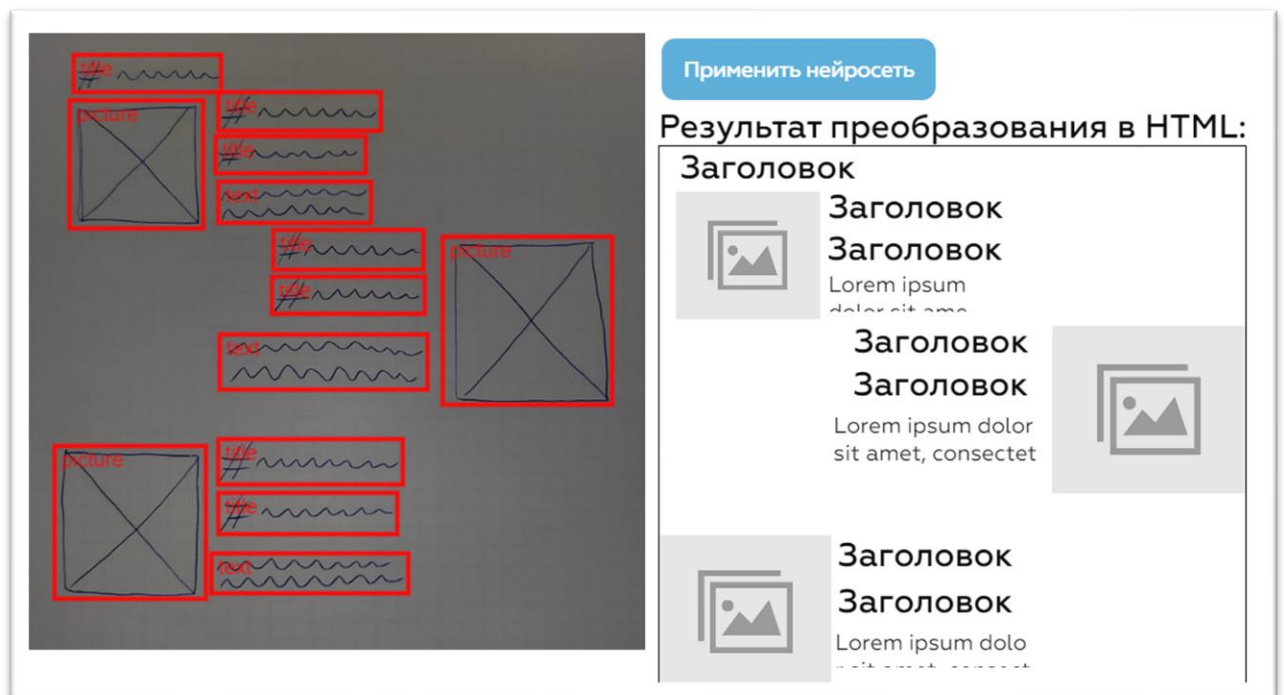


Рис. 88. Результат преобразования макета стороннего сервиса №2 в HTML
нашим алгоритмом

Таблица 22

Результаты метрик оценки HTML-шаблона изображения №2 нашим
алгоритмом

№	Название метрики	Результат
1	MSE	19.1
2	RMSE	4.37
3	AverageIoU	79%

Результат работы нашего алгоритма для изображения №2 – в среднем для каждого объекта отклонение в размерах и положении составляет 4.37

единицы. В среднем исходные объекты совпадают по площади с объектами на шаблоне на 79%.

Приведем гистограммы для оценки результатов метрик между нашим алгоритмом и сервисом sketch2Code для изображения №2 (рис.89-90).

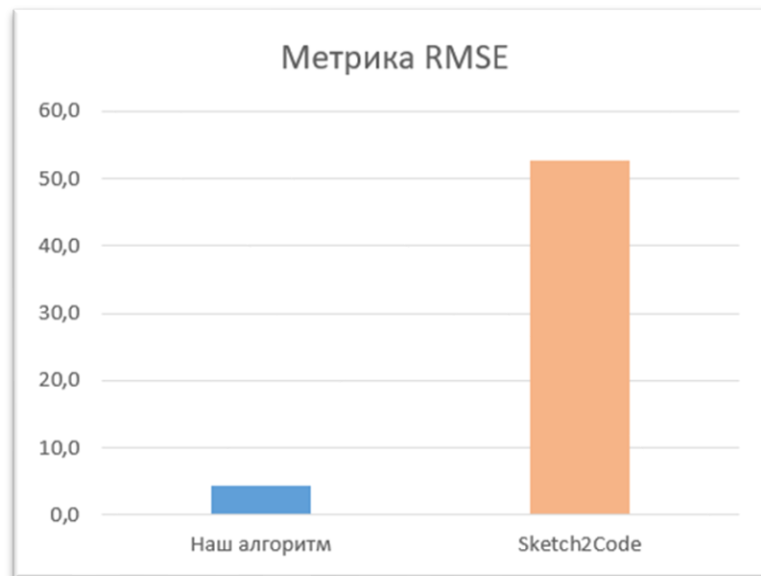


Рис. 89. Сравнение результатов метрики RMSE для изображения №2

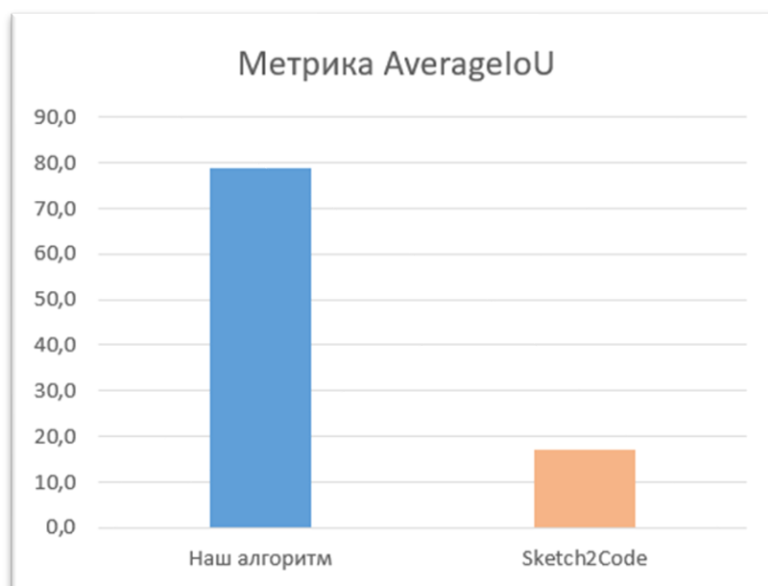


Рис. 90. Сравнение результатов метрики AverageIoU для изображения №2

Как видно из проведенных экспериментов – разработанный в рамках работы алгоритм выбирает размеры объектов и располагает их значительно

лучше, чем алгоритм, использующийся в сервисе Sketch2Code, при этом выходной HTML-код разработанного алгоритма является адаптивным и может подстраиваться под разрешения мобильных и планшетных устройств.

Также оценим работу алгоритма на собственных примерах (рис.91-96, табл.23-27):



Рис. 91. Результат преобразования макета собственного примера №1 в HTML

Таблица 23

Результаты метрик оценки HTML-шаблона собственного примера №1 нашим алгоритмом

№	Название метрики	Результат
1	MSE	249.71
2	RMSE	15.8
3	AverageIoU	77%

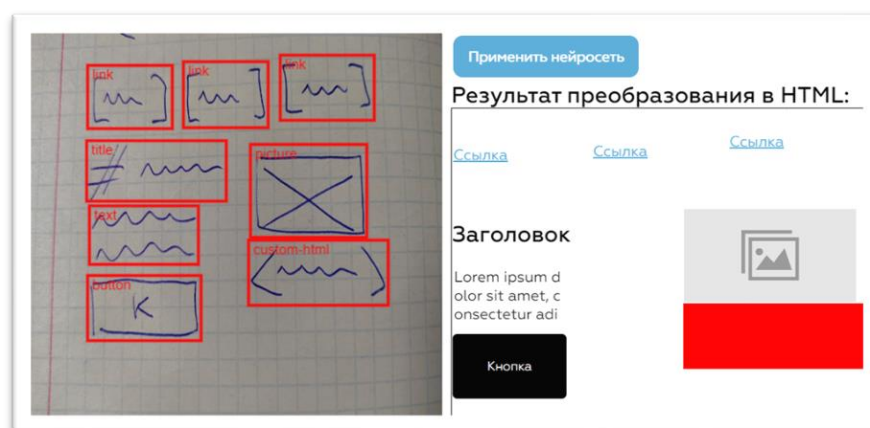


Рис. 92. Результат преобразования макета собственного примера №2 в HTML

Таблица 24

Результаты метрик оценки HTML-шаблона собственного примера №2 нашим алгоритмом

№	Название метрики	Результат
1	MSE	169.59
2	RMSE	13.02
3	AverageIoU	77%

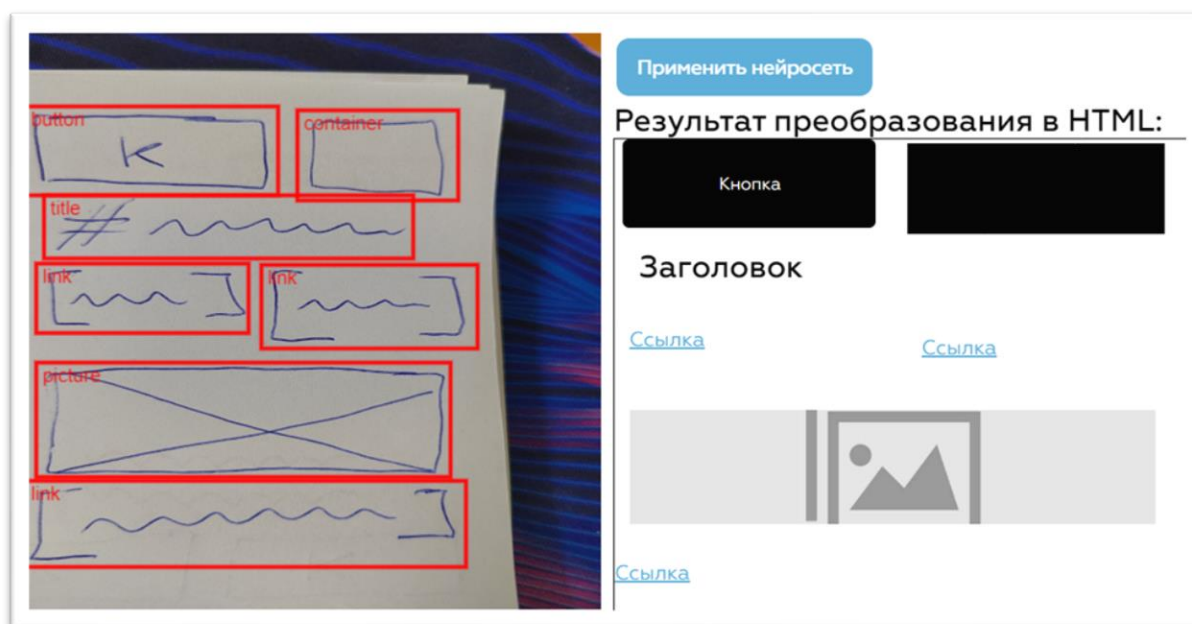


Рис. 93. Результат преобразования макета собственного примера №3 в HTML

Таблица 25

Результаты метрик оценки HTML-шаблона собственного примера №3 нашим алгоритмом

№	Название метрики	Результат
1	MSE	186.33
2	RMSE	13.65
3	AverageIoU	89%

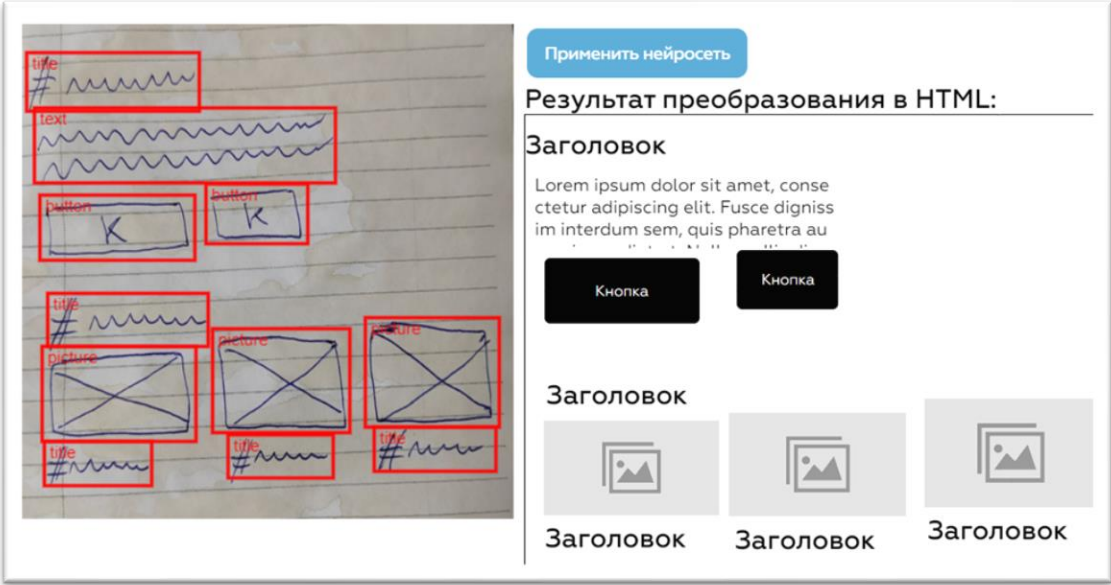


Рис. 94. Результат преобразования макета собственного примера №4 в HTML

Таблица 26

Результаты метрик оценки HTML-шаблона собственного примера №4 нашим алгоритмом

№	Название метрики	Результат
1	MSE	112.43
2	RMSE	10.6
3	AverageIoU	76%

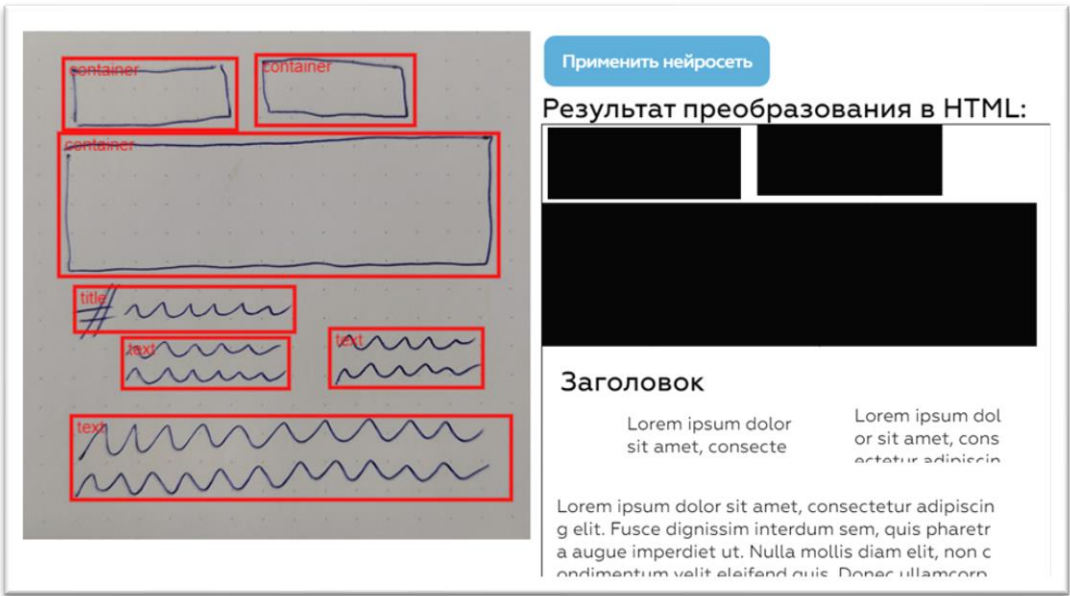


Рис. 95. Результат преобразования макета собственного примера №5 в HTML

При изменении ширины рабочего пространства менее 350-и пикселей – шаблон будет перестраиваться в одну колонку, как показано на рис.96.

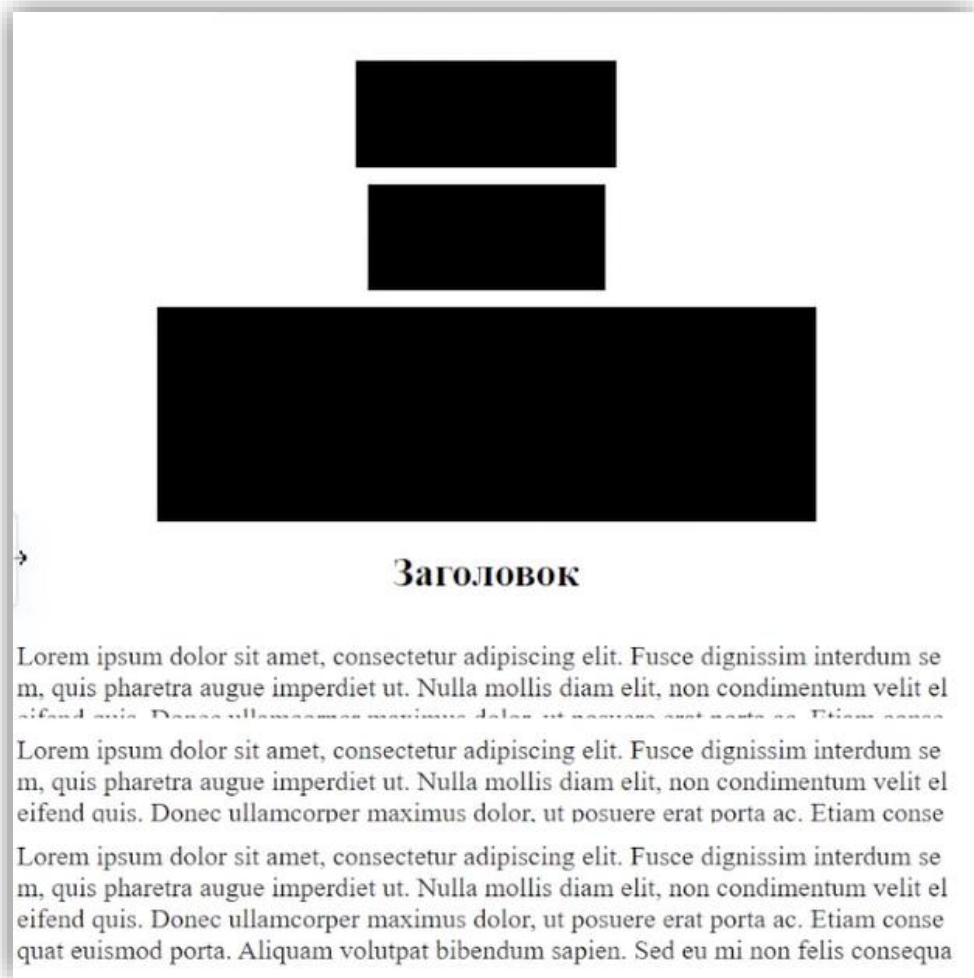


Рис. 96. Изменение шаблона при уменьшении ширины рабочего пространства

Таблица 27

Результаты метрик оценки HTML-шаблона собственного примера №5 нашим алгоритмом

№	Название метрики	Результат
1	MSE	31.03
2	RMSE	5.57
3	AverageIoU	89%

На основе собственных примеров можно заметить, что ни один объект в рамках тестирования не потерялся, порядок объектов и их положение в HTML

строго соответствуют рукописным рисункам, в отличие от решения Sketch2Code, где объекты могут идти в разном порядке на итоговом HTML-шаблоне. В среднем, получившийся алгоритм для генерации HTML-шаблонов по рукописным рисункам-макетам, на основе собственных примеров по метрике AverageIoU выдает результат в 81.6%.

Полезно оценить также и практическую значимость разработанного алгоритма на реальной бизнес-задаче. Для этого проведем эксперимент – на основе готового дизайна бизнес-задачи (рис.97) напишем HTML-код вручную, а также используя ПО для генерации HTML-кода по рукописным рисункам-макетам, сравним фактически затраченное время на оба случая.

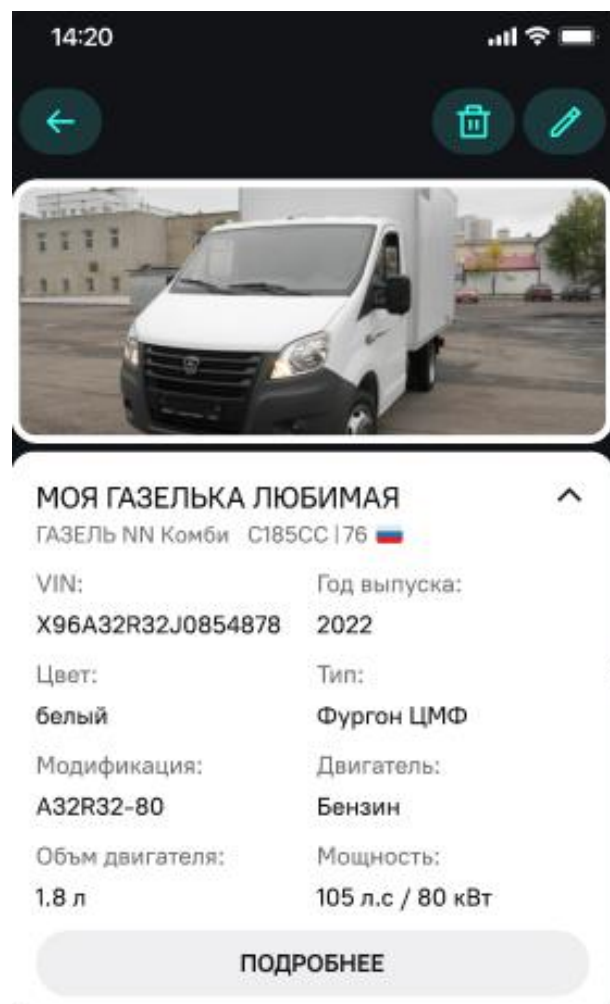


Рис. 97. Дизайн реальной бизнес-задачи

Так как ПО пока ограничено в наборе классов, то для тех классов, которых нет – рисуем на их месте объект класса «container» (рис.98).

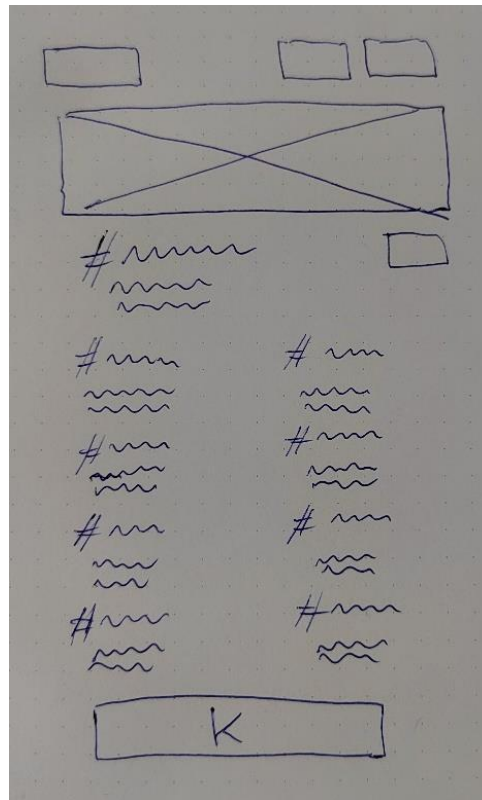


Рис. 98. Рукописный макет реальной бизнес-задачи

Очевидно, что после работы алгоритма (рис.99-100) преобразования HTML-макета в готовый код – нужны ручные правки для полного соответствия изначальному дизайну.

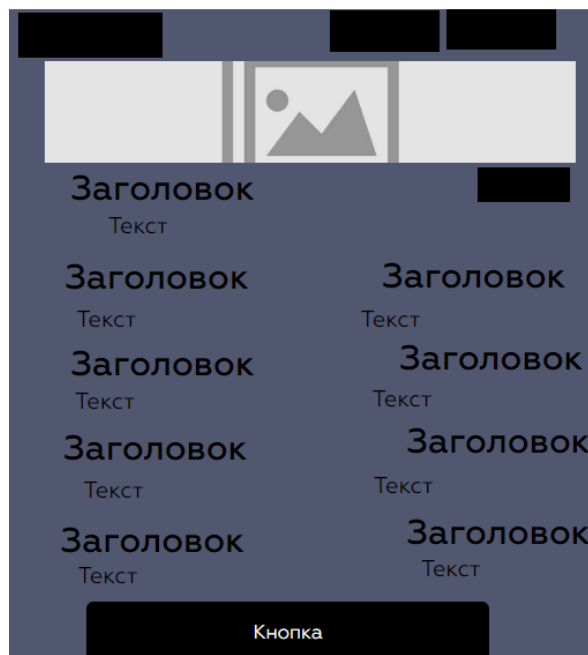


Рис. 99. Результат преобразования рукописного макета в HTML на реальной бизнес-задаче

```

1 <div class="neural-div-container">
2   <div class="row" style="position: relative; padding-right: 3.36788%;">
3     <div class="column" style="position: relative; flex-direction: column; padding-top: 0.729927%;">
4       <div style="background-color: black; width: 100%; height: 100%; min-height: 31px; min-width: 100px;"
5         id="object0"></div>
6     </div>
7     <div class="column" style="position: relative; flex-direction: column; padding-top: 0.243309%; padding-bottom: 0.973236%; p
8       <div style="background-color: black; width: 100%; height: 100%; min-height: 29px; min-width: 76px;"
9         id="object1"></div>
10    </div>
11    <div class="column" style="position: relative; flex-direction: column; padding-bottom: 1.45985%; padding-left: 1.29534%;">
12      <div style="background-color: black; width: 100%; height: 100%; min-height: 28px; min-width: 76px;"
13        id="object2"></div>
14    </div>
15  </div>

```

Рис. 100. Фрагмент сгенерированного HTML-кода для реальной бизнес-задачи

После ручных правок сгенерированного HTML, а также добавления свойств CSS-стилей, готовая HTML сетка выглядит, как показано на рисунке (рис.101).

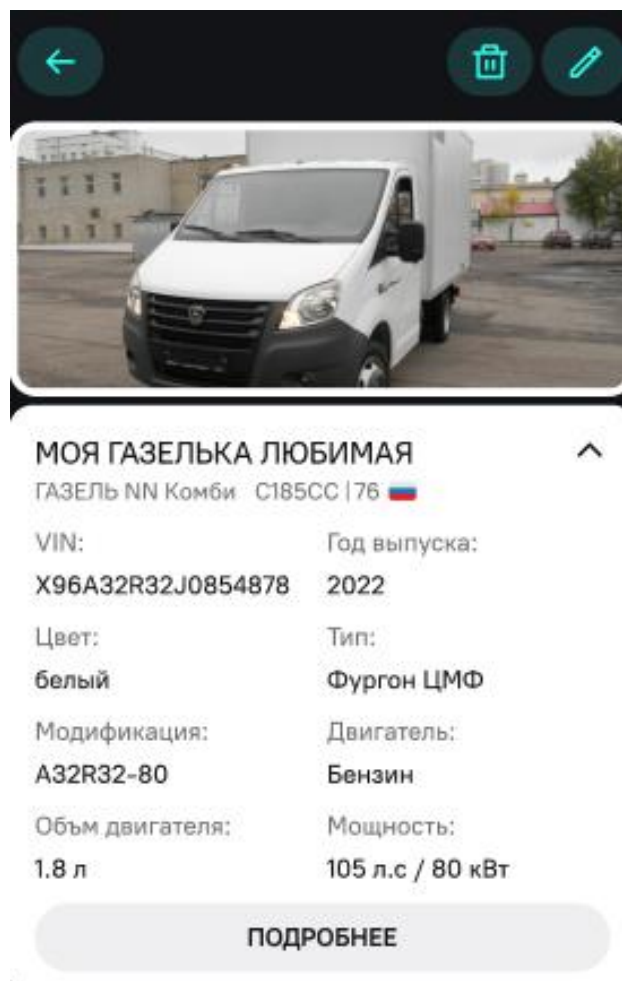


Рис. 101. Отображение в браузере готового HTML

Таким образом, в результате эксперимента были зафиксированы фактически затраченное время на два случая: ручное написание HTML-кода; генерация HTML-кода на основе рукописного-рисунка и дальнейшие ручные правки для соответствия изначальному дизайну, затраченное время на оба случая зафиксировано в табл.28.

Таблица 28

Фактически затраченное время верстки HTML на основе реальной бизнес-задачи

Способ формирования HTML	Затраченное время (мин.)
Ручной	20
С помощью генератора HTML-кода на основе рукописного рисунка с ручными правками	16

На основе даже такого простейшего примера реальной бизнес-задачи, сэкономленное время составило 4 минуты, а это составляет 20% выигранного времени относительно ручного способа создания кода.

Отсюда можно подсчитать практическую значимость разработанного ПО. Допустим, если каждый рабочий день сотрудник тратит 2ч на верстку HTML, а стоимость одного человека-часа составляет 3500 рублей, при этом всего рабочих дней в году 250, то за год данный сотрудник экономит около 100 часов своего рабочего времени, используя генератор HTML-кода, а это составляет 350.000 рублей в год. А если сотрудников больше, то и совокупная экономия времени и средств компании будет больше.

4.3. Результаты экспериментальной проверки модулей и классов программного обеспечения генератора HTML-шаблонов по рукописным рисункам-макетам

Для проверки программного обеспечения использовались два основных метода: функциональное тестирование и системное тестирование. Системное

тестирование включало в себя метод ручного контроля. Обычно этот метод применяется на ранних этапах разработки, так как позволяет обнаружить от 30 до 70% ошибок логического проектирования и кодирования. Исходными данными для таких проверок служили: техническое задание, спецификации, структурные и функциональные схемы программного продукта, а также схемы отдельных компонентов.

Один человек проводил проверку исходного текста программы, осуществляя поиск возможных ошибок в соответствии со списком наиболее часто встречающихся ошибок.

Для тестирования модулей был выбран метод функционального тестирования. Исходными данными для этого метода служили программные модули, спецификации компонентов, а также правильные и неправильные данные для модулей.

В ходе проверки программной документации методом ручного контроля были исправлены обнаруженные грамматические и пунктуационные ошибки.

Результаты тестирования компонентов ПО представлены в табл.29.

Таблица 29

Тестирование компонентов ПО

Дата	Тестируемый модуль	Кто проводил тестирование	Способ тестирования	Результат тестирования
1	2	3	4	5
23.04.2024	mean-squared-error.ts	Разработчик	Функциональное тестирование	Успех
23.04.2024	intersection-over-union.ts	Разработчик	Функциональное тестирование	Успех
23.04.2024	metrics/index.ts	Разработчик	Функциональное тестирование	Успех
23.04.2024	absolute-positioning-strategy.ts	Разработчик	Функциональное тестирование	Успех
24.04.2024	rows-and-columns-strategy.ts	Разработчик	Функциональное тестирование	Неудача, строки были перепутаны с колонками. Исправлено
24.04.2024	html-parser.ts	Разработчик	Функциональное тестирование	Успех

Продолжение табл.29

1	2	3	4	5
24.04.2024	neural.service.ts	Разработчик	Функциональное тестирование	Успех
24.04.2024	http-interceptor.service.ts	Разработчик	Функциональное тестирование	Неудача, неверно был указан порт локального сервера. Исправлено
24.04.2024	engine.ts	Разработчик	Функциональное тестирование	Успех
25.04.2024	app.module.ts	Разработчик	Функциональное тестирование	Успех
25.04.2024	app.component.ts	Разработчик	Функциональное тестирование	Успех
25.04.2024	render-app-routing.module.ts	Разработчик	Функциональное тестирование	Успех
25.04.2024	app-config.service.ts	Разработчик	Функциональное тестирование	Успех
25.04.2024	utils.service.ts	Разработчик	Функциональное тестирование	Успех
25.04.2024	providers/engine/engine.ts	Разработчик	Функциональное тестирование	Успех
25.04.2024	providers/neural/neural.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	main.module.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	main.page.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	edit-site.component.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	page-editor.component.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	ui-tree-element-editor.component.ts	Разработчик	Функциональное тестирование	Неудача, элемент перекрывался на странице другими элементами. Исправлено
26.04.2024	ui-tree.component.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	components.module.ts	Разработчик	Функциональное тестирование	Неудача, не все компоненты были загружены. Исправлено

Продолжение табл.29

1	2	3	4	5
26.04.2024	neural-parser-to-html.component.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	test-predicted.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	header.component.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	editor-item.component.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	editor-components/index.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	buttons-editor.component.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	features-editor.component.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	forms-editor.component.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	link-editor.component.ts	Разработчик	Функциональное тестирование	Неудача, у ссылки был неправильно указан атрибут. Исправлено
26.04.2024	slider-editor.component.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	text-editor.component.ts	Разработчик	Функциональное тестирование	Успех
26.04.2024	title-editor.component.ts	Разработчик	Функциональное тестирование	Успех
27.04.2024	video-editor.component.ts	Разработчик	Функциональное тестирование	Успех
27.04.2024	dropdown-edit-panel.component.ts	Разработчик	Функциональное тестирование	Успех
27.04.2024	block-appender.component.ts	Разработчик	Функциональное тестирование	Успех

Результаты тестирования программного обеспечения на выполнение требований к функциональным характеристикам представлены в табл.30.

Тестирование функциональных характеристик

Дата	Тестируемое требование из ТЗ	Кто проводил тестирование	Способ тестирования	Результат тестирования
25.04.2024	Скорость обработки изображения нейросетью не более 10-и секунд на изображение	Разработчик	Системное тестирование	Успех
25.04.2024	Детектирование объектов классов «контейнер», «кнопка», «изображение», «заголовок», «текст», «ссылка», «пользовательский HTML»	Разработчик	Системное тестирование	Неудача, ошибка при детектировании кнопок. Исправлено
25.04.2024	Формирование HTML-сетки с иерархической структурой вида строки-столбцы	Разработчик	Системное тестирование	Неудача, генерация избыточного HTML-кода. Исправлено
26.04.2024	Возможность выбора заготовленных шаблонов по категориям	Разработчик	Системное тестирование	Успех
26.04.2024	Возможность добавления и отображения новых блоков	Разработчик	Системное тестирование	Успех
27.04.2024	Инструменты для редактирования типовых элементов, редактирование контента и стилевых свойств	Разработчик	Системное тестирование	Неудача, инструменты не привязывались к HTML-коду. Исправлено

4.4. Перспективы применения алгоритма генерирования HTML-сетки

Предложенный алгоритм генерирования HTML-сетки можно использовать для прототипирования сайтов/приложений, что ускорит процесс создания HTML-макетов и улучшит взаимодействие между дизайнерами и

разработчиками. Формируемый код совместим с современными версиями браузеров, формируемая HTML-сетка является адаптивной и при изменении размера окна будет подстраиваться под эти размеры. При более плотной интеграции с инструментами редактирования HTML-кода – можно значительно сократить время на создание интерфейсов для различных приложений (поддерживающих HTML), а также избавиться от необходимости ручной правки получившегося HTML-кода.

4.5. Выводы по результатам экспериментального тестирования

Разработанный алгоритм для генерации HTML-шаблона на основе рукописного рисунка-макета обладает всеми предусмотренными ранее требованиями - шаблон строится сразу же для адаптивных разрешений экранов – т.е. получившийся макет можно запускать с экранов компьютера, телефона, планшета. Адаптивность выражается в том, что при уменьшении размеров экрана – пропорционально уменьшаются размеры объектов и отступы между ними, при ширине экрана менее 350 пикселей все объекты начинают идти друг за другом по вертикали.

Также получившийся шаблон можно будет совместно использовать с библиотекой Materialize или же Bootstrap, в том числе, разработанный алгоритм располагает объектами точнее, нежели существующее техническое решение «Sketch2Code». Стоит отметить, что созданное в рамках исследования программное обеспечение, в отличие от конкурентов, позволяет в интерфейсе отредактировать получившийся шаблон под конкретные требования пользователя. Подтверждением успешного использования спроектированных алгоритмов и разработанного программного обеспечения служит «Акт о внедрении» (Прил.5).

Заключение

Результатом выполнения выпускной квалификационной работы является разработанное программное обеспечение генерирования HTML-шаблонов по рукописным рисункам-макетам. В работе, первую очередь, были описаны объект и предмет исследования. Была изучена модель сверточной нейронной сети для детектирования объектов на изображении рукописного рисунка-макета.

В ходе работы было выполнено подробное описание математического и алгоритмического аппарата для успешного выполнения поставленных целей и задач. Для распознавания объектов с рукописных рисунков-макетов использовалась архитектура сверточной нейронной сети YOLOv3, для помощи в подготовке тренировочного набора изображений для используемой нейросети был спроектирован и реализован специальный алгоритм.

В результате обучения нейронной сети на тренировочном наборе изображений – была достигнута точность распознавания объектов на рукописных рисунках-макетах по метрике mAP50 равная 93%, при обучении использовалось 7 классов объектов. Архитектура программного обеспечения предусмотрена таким образом, что при потребности можно добавить и обучить новые классы объектов.

Также в рамках работы был разработан и реализован алгоритм генерирования HTML-шаблона на основе рукописного рисунка-макета, данный алгоритм применяется после этапа детектирования объектов. На основе метрик MSE, RMSE, и AverageIoU разработанный алгоритм показывает результат лучше, чем аналогичное решение, реализованное в программном обеспечении от компании Microsoft с названием «Sketch2Code». Мало того, что алгоритм лучше располагает объекты по размеру и положению, чем конкурирующее решение, но и при изменении размеров рабочего окна – гибко меняет сетку под новые размеры, в том числе под планшетные и мобильные устройства.

На основе собственных примеров, разработанный алгоритм в среднем по метрике AverageIoU показывает результат в 81.6%, что является оптимальным результатом.

По итогу выполнения работы были освоены следующие компетенции:

- способен осуществлять критический анализ проблемных ситуаций на основе системного подхода, вырабатывать стратегию действий;
- способен управлять проектом на всех этапах его жизненного цикла;
- способен организовывать и руководить работой команды, вырабатывая командную стратегию для достижения поставленной цели;
- способен применять современные коммуникативные технологии, в том числе на иностранном(ых) языке(ах), для академического и профессионального взаимодействия;
- способен анализировать и учитывать разнообразие культур в процессе межкультурного взаимодействия;
- способен определять и реализовывать приоритеты собственной деятельности и способы ее совершенствования на основе самооценки;
- способен самостоятельно приобретать, развивать и применять математические, естественнонаучные, социально-экономические и профессиональные знания для решения нестандартных задач, в том числе, в новой или незнакомой среде и в междисциплинарном контексте;
- способен разрабатывать оригинальные алгоритмы и программные средства, в том числе с использованием современных интеллектуальных технологий, для решения профессиональных задач;
- способен анализировать профессиональную информацию выделять в ней главное, структурировать, оформлять и представлять в виде

аналитических обзоров с обоснованными выводами и рекомендациями;

- способен применять на практике новые научные принципы и методы исследований;
- способен разрабатывать и модернизировать программное и аппаратное обеспечение информационных и автоматизированных систем;
- способен самостоятельно приобретать с помощью информационных технологий и использовать в практической деятельности новые знания и умения, в том числе в новых областях знаний, непосредственно не связанных со сферой деятельности;
- способен применять при решении профессиональных задач методы и средства получения, хранения, переработки и трансляции информации посредством современных компьютерных технологий, в том числе, в глобальных компьютерных сетях;
- способен осуществлять эффективное управление разработкой программных средств и проектов;
- управление программно-техническими, технологическими и человеческими ресурсами.

Список литературы

1. Бекирова Э.А., Халилова З.Э. Основные этапы создания web-приложений // Информационно-компьютерные технологии в экономике, образовании и социальной сфере, Том. 1, №. 23, 2019. стр. 84-91.
2. Язык HTML: что это такое и как он работает // Skillbox. 2024. URL: https://skillbox.ru/media/code/chto_takoe_html/
3. Кирдюк Д.И. Адаптивная верстка веб-страниц сайта минимальными средствами HTML и CSS // Молодость. Интеллект. Инициатива, 2018. стр. 262-263.
4. Салимянов И.Т. Создание веб-сайтов с использованием bootstrap. Казань: Общество с ограниченной ответственностью "Редакционно-издательский центр "Школа" (Казань), 2019.
5. Колмыкова С.В. Основы верстки web-страниц в программном продукте notepad++ при подготовке специалистов среднего звена в области информационных технологий // Современные научные исследования: теория и практика, 2019. стр. 36-44.
6. Гагарин А.Г., Рогачев А.Ф., Плещенко Т.В. Применение языка гипертекстовой разметки для разработки интернет-ресурсов, Волгоград, 2013. 144 стр.
7. Быкова К.И., Дубинин Е.С., Кузьмичева Е.А. Возможные сложности при верстке сайтов с использованием CSS и пути их решения // Информационные технологии в образовательном процессе вуза и школы, 2023. стр. 71-77.
8. Мальцев Д.В., Тазетдинов Б.И. Актуальные проблемы науки и образования в условиях современных вызовов // Об устаревших тегах HTML5. Москва. 2022.
9. Баев Д.Э., Суров А.Д., Григорьев А.В. Современные информационные технологии в образовании и научных исследованиях (СИТОНИ-2021) // Генерация программного кода. Донецк. 2021. стр. 397-412.
10. Chat GPT-4 Нейросеть GPT-4 что это такое? // ИИ, Фишки и польза. URL: <https://vc.ru/u/1628180-fishki-i-polza-ii/650619-chat-gpt-4-neyroset-gpt-4-chto-eto-takoe> (дата обращения: 12.04.2024).

11. Компьютерное зрение - Викиконспекты URL: https://neerc.ifmo.ru/wiki/index.php?title=%D0%9A%D0%BE%D0%BC%D0%BF%D1%8C%D1%8E%D1%82%D0%B5%D1%80%D0%BD%D0%BE%D0%B5_%D0%B7%D1%80%D0%B5%D0%BD%D0%B8%D0%B5 (дата обращения: 06.02.2024).
12. Наглядно о том, как работает свёрточная нейронная сеть URL: <https://habr.com/ru/companies/skillfactory/articles/565232/> (дата обращения: 06.02.2024).
13. Компьютерное зрение [Электронный ресурс] // exponenta: [сайт]. URL: <https://exponenta.ru/comp-vision#:~:text=%D0%9A%D0%BB%D0%B0%D1%81%D1%81%D0%B8%D1%87%D0%B5%D1%81%D0%BA%D0%B8%D0%B5%20%D0%BC%D0%B5%D1%82%D0%BE%D0%B4%D1%8B%20%D0%BA%D0%BE%D0%BC%D0%BF%D1%8C%D1%8E%D1%82%D0%B5%D1%80%D0%BD%D0%BE%D0%B3%D0%BE%20%D0%B7%D1%80%D0%B5%20%D0%BD%D0%B8%D0%B5> (дата обращения: 28.05.2024).
14. Верхов К.А. Новые информационные технологии в научных исследованиях: материалы XXVI Всероссийской научно-технической конференции студентов, молодых ученых и специалистов // Обнаружение объектов на изображении с помощью метода Regional Proposal Network. Рязань. 2021. стр. 213-214.
15. Шнайдер А.С. Детектирование объектов на изображениях на примере модели YOLO // Труды молодых ученых алтайского государственного университета, №. 15, 2018. стр. 358-361.
16. Скрипачев В.О., Гуйда М.В., Гуйда Н.В., Жуков А.О. Исследование сверточных нейронных сетей для обнаружения объектов на аэрокосмических снимках // International Journal of Open Information Technologies, №. 7, 2022.
17. Чжу И., Новикова Т.П., Евдокимова С.А. Моделирование информационных систем и технологий // Исследование остаточной нейронной сети RESNET. Воронеж. 2022.
18. Дильмухаметова А.Р., Беляева М.Б. Архитектура RESNET // Modern science, Том. 2, №. 12, 2021. стр. 356-359.
19. Цышулин А.К., Бобровский А.И., Морозов А.В., Павлов В.А., Галеева М.А. Применение свёрточных нейронных сетей для автоматической селекции малоразмерных искусственных космических объектов на

оптических изображениях звёздного неба // Оптический журнал, Том. 86, №. 10, 2019. стр. 30-38.

20. Леус А.В., Ефремов В.А. Применение методов компьютерного зрения для анализа изображений, собранных с фотоловушек в рамках программно-аппаратного комплекса мониторинга состояния окружающей среды на особо охраняемых природных территориях // Труды мордовского государственного природного заповедника им. П.Г. Смидовича, №. 28, 2021. стр. 121-129.
21. Федосеев А.А., Гаязова Е.Э. Определение объектов заданных классов на изображении // Ученые заметки ТОГУ, Том. 11, №. 2, 2020. стр. 31-37.
22. mAP (mean Average Precision) в детекции объектов URL: <https://ml.i-neti.ru/map-mean-average-precision/> (дата обращения: 06.02.2024).
23. Чернова Е.А., Соснина О.А. Модель вёрстки Flexbox // КОГРАФ - 2020, 2020. стр. 176-178.
24. Волков А.С., Бочаров Д.М. Современные технологии вёрстки и frontend // Материалы VII Международной научно-технической конференции. Под общей редакцией В.Н. Павлыша. Донецк, 2021, 2021. стр. 515-520.
25. Клеменьтев А.А. Методы адаптивной верстки сайтов // сборник статей Международной научно-практической конференции в 2 частях, Том. 1, №. 1, 2021. стр. 125-127.
26. Чёрный ящик URL: https://ru.wikipedia.org/wiki/%D0%A7%D1%91%D1%80%D0%BD%D1%8B%D0%B9_%D1%8F%D1%89%D0%B8%D0%BA (дата обращения: 06.02.2024).
27. Пономарев А.Б., Пикулева Э.А. Методология научных исследований, Пермь, 2014.
28. Жизненный цикл программного обеспечения URL: <https://qaevolution.ru/zhiznennyj-cikl-programmnogo-obespecheniya/> (дата обращения: 06.02.2024).
29. Структурный подход к проектированию ИС URL: <https://mashuk.ru/shkolniku/346-strukturnyj-podkhod-k-proektirovaniyu-is> (дата обращения: 06.02.2024).
30. Концепции объектно-ориентированного программирования JAVA URL: <https://javarush.com/groups/posts/1265-koncepcii-obhhektno-orientirovannogo-programmirovaniya-java> (дата обращения: 06.02.2024).

31. Рыжов Н.Г., Анисимов В.И., Васильев С.А. Информационные технологии и математическое моделирование систем 2019 // Обзор современных инструментов для разработки клиентских приложений. г. Санкт-Петербург. 2019.
32. Сыротюк С.Д., Гавриленков В.И. Разработка нейросети на языке программирования Python // Педагогический форум, Vol. 1, No. 11, 2023. стр. 32-34.
33. Бадыков И.В., Мышкина И.Ю., Грудцына Ю.Л. Распознавание состояний человека с использованием библиотеки машинного обучения Tensorflow // Научно-технический вестник Поволжья, №. 5, 2019. стр. 12-14.
34. Лягина К.И. Разработка диаграммы вариантов использования системы автоматизированного документооборота отдела международных связей // Разработка диаграммы вариантов использования системы автоматизированного документооборота отдела международных связей. Пенза. 2016.
35. Урок 2. Разновидности UML диаграмм URL: <https://forum.itvdn.com/t/urok-2-raznovidnosti-uml-diagramm/3315> (дата обращения: 06.02.2024).
36. State & Transition Diagram — что это и как применять URL: <https://habr.com/ru/articles/548192/> (дата обращения: 06.02.2024).
37. Разновидности диаграмм UML URL: <https://www.lucidchart.com/blog/ru/types-of-UML-diagrams> (дата обращения: 06.02.2024).
38. Простое руководство по диаграммы компонентов URL: <https://creately.com/blog/ru/uncategorized-ru/%D1%83%D1%87%D0%B5%D0%B1%D0%BD%D0%BE%D0%B5-%D0%BF%D0%BE%D1%81%D0%BE%D0%B1%D0%B8%D0%B5-%D0%BF%D0%BE-%D0%BA%D0%BE%D0%BC%D0%BF%D0%BE%D0%BD%D0%B5%D0%BD%D1%82%D0%BD%D0%BE%D0%B9-%D0%B4%D0%B8%D0%B0%D0%B3/> (дата обращения: 02.05.2024).
39. Андриевская Н.К., Секирин А.И., Ченгарь О.В. Разработка архитектурной модели системы управления информационными ресурсами организаций // Программная инженерия: методы и технологии разработки информационно-вычислительных систем (ПИИВС-2020). 2020. стр. 46-54.
40. Распознавание образов с помощью искусственного интеллекта URL: <https://habr.com/ru/articles/709432/> (дата обращения: 06.02.2024).

41. Тимошкин М.С., Миронов А.Н., Леонтьев А.С. Сравнение YOLO v5 и Faster R-CNN для обнаружения людей на изображении в потоковом режиме // Международный научно-исследовательский журнал, №. 6, 2022.

УСТАВ ПРОЕКТА

Программное обеспечение генератора HTML-шаблонов по рукописным рисункам-макетам

Общие положения

Данный документ является соглашением между основными участниками проекта, определяющим цели проекта, рамки проекта, обязанности участников проекта, основные этапы выполнения работ по проекту. Соблюдение положения устава является обязательным для всех участников проекта.

Данный документ может меняться в ходе выполнения проекта. Внесение изменений и дополнений в устав проекта инициируется руководителем проекта и утверждается заказчиком (решением управляющего комитета). Настоящий устав действует, начиная с момента подписания договора на проектные работы и до окончания срока реализации проекта (до подписания акта сдачи-приемки всех работ по проекту).

Требования удовлетворяющие потребности, пожелания и ожидания заказчика и других участников проекта

Программный продукт должен обеспечить возможность генерации HTML-шаблонов по рукописным рисункам-макетам.

Пользовательский интерфейс должен быть интуитивно понятным, должна быть возможность выгрузки HTML-кода для его последующего применения или редактирования. Должны быть предусмотрены инструменты для редактирования типовых HTML-элементов.

Производственная необходимость

Разрабатываемое техническое решение можно применить в разных ситуациях: при разработке веб-интерфейсов, при разработке интерфейсов для мобильных приложений при использовании фреймворков Ionic или React Native, при прототипировании интерфейсов для приложений, использующих HTML для отображения пользовательского интерфейса. У компании ООО «АСТРОСОФТ ПРО» стоит задача для внутренних целей компании внедрить программное обеспечение для генерирования HTML-шаблонов по рукописным рисункам-макетам, чтобы затрачивать меньшее количество времени при написании HTML-кода для интерфейсов различных приложений. В настоящее время существует мало программных решений (в том числе ни одного известного отечественного), способных решить данную задачу, это обусловлено тем, что сама по себе задача генерации HTML-шаблонов является алгоритмически сложной и имеет нюансы с обеспечением высокой производительности обработки изображений с детектированием объектов, при этом большинство известных решений даёт большую ошибку в расположении элементов в финальном HTML-шаблоне.

Требования к продукту, который является предметом проекта

Требования к функциональным характеристикам:

- выбор архитектуры нейронной сети, которая позволит добиться высокой скорости обработки изображений (не более 10-ти секунд на изображение);
- реализованная архитектура нейронной сети должна обеспечить возможность детектирования объектов с рукописных рисунков-макетов, как минимум, необходимо детектировать следующие классы — «контейнер», «кнопка», «изображение», «заголовок», «текст», «ссылка», «пользовательский HTML»;
- из объектов, которые распознаны нейросетью необходимо формировать HTML-сетку, сетка должна формировать строки и

столбцы (в виде иерархической структуры, используя тэг «div» и прикрепленный к нему атрибут класса «row» или «column»), чтобы можно было в случае необходимости интегрировать сетку с CSS-фреймворками Bootstrap или Materialize;

- в пользовательском интерфейсе клиентского приложения предусмотреть возможность выбора заготовленных шаблонов по категориям;
- в пользовательском интерфейсе предусмотреть возможность добавления новых блоков, а также их отображения, блоком считать фрагмент HTML-кода, на странице может быть множество блоков;
- элементы внутри блока должны быть редактируемы, то есть нужны инструменты для редактирования типовых элементов – редактированию подлежит текст, стилевые свойства элемента (цвет, цвет фона, отступы, ширина, высота, поворот, скругление краев).

Цель проекта

Спроектировать и реализовать программное обеспечение для генерации HTML-шаблонов по рукописным рисункам-макетам.

Критерии оценки успешной реализации проекта

- точность распознавания классов нарисованных от руки элементов не менее 85% (по метрике mAP);
- время обработки изображения и выдачи HTML и CSS кода – не более 10-ти секунд (табл.П1.1).

Расписание контрольных событий

Этап	Срок
1 этап: Анализ предметной области, постановка задачи (разработка требований)	ноябрь-декабрь 2022
2 этап: Выбор и описание подхода и инструментальных средств разработки программного обеспечения	ноябрь-декабрь 2022
3 этап: Выбор и описание математического аппарата, используемого в проекте	январь-март 2023
4 этап: Проектирование программного обеспечения: разработка логической модели и физической модели	апрель-сентябрь 2023
5 этап: Разработка программного обеспечения	октябрь-март 2024
6 этап: Экспериментальная проверка программного обеспечения, оформление результатов проекта	апрель-май 2024

Информация об участниках проекта

Таблица П1.2

Заказчик проекта

Организация	ООО «Астрософт ПРО»
Полномочия	• формирование общих требований к результату проекта; • назначение руководителя проекта; • контроль хода выполнения работ по проекту; • приемка результатов проекта.
Ответственность	• за своевременное обеспечение проекта ресурсами.

Руководитель (менеджер) проекта

Таблица П1.3

Организация	ФГБОУ ВО «Череповецкий государственный университет»
ФИО, должность	Ершов Евгений Валентинович, заведующий кафедрой МПО ЭВМ
Контактная информация	Тел: +7(8202) 51-90-69
Полномочия	• формирование команды проекта, распределение ответственности, организация взаимодействия между участниками проектной команды и заказчиком; • распределение ресурсов по стадиям (этапов) проекта; • управление рисками проекта, управление процессом решения проблем; • участие в разрешении противоречий в проектных решениях;

	• планирование, организация и контроль выполнения работ по достижению целей проекта с требуемыми затратами, качеством и в заданный срок; • достижение запланированных результатов; • оценка результативности и разработка отчетных документов по проекту.
Ответственность	• за качественное и своевременное исполнение работ по проекту; достижение запланированных результатов.

Разработчик проекта

Таблица П1.4

Организация	ФГБОУ ВО «Череповецкий государственный университет»
ФИО, должность	Квашенников Михаил Сергеевич, студент группы 1ПИМ-03-1оп-22 направления подготовки (специальности) 09.04.04
Контактная информация	Тел: +79215402003
Полномочия	• разработка, проектирование программного продукта согласно требованиям технического задания; • создание дорожной карты для планирования и организации выполнения работ по достижению целей; • коммуникация с заказчиком, получение обратной связи по реализации проекта.

Ответственность	• за качественное и своевременное выполнение работ по проекту; достижение запланированных результатов.
-----------------	--

Функциональные организации и их участие

Проект реализуется на базе ФГБОУ ВО «Череповецкий государственный университет».

Заказчиком проекта является ООО «АСТРОСОФТ ПРО» - разработка и поддержка решений, которые помогают компаниям по всей стране решать сложные интеллектуальные и технические задачи в области машинного зрения, iot-платформ и работе с большими данными.

Бюджет проекта

Проект реализуется в рамках освоения магистерской программы «Разработка программно-информационных систем» по направлению 09.04.04 Программная инженерия.

В рамках реализации данного проекта отсутствует необходимость в приобретении программных продуктов специального назначения, так как все требования для разработки можно реализовать на свободно распространяемом программном обеспечении.

Для разработки программного обеспечения необходим ноутбук или стационарный компьютер с наличием дискретной видеокарты со средними характеристиками, к примеру MSI GL65 9SCK (70000 рублей).

Программное решение предполагает вложение финансовых средств, поэтому возникает вопрос рационального их применения и выбора наиболее эффективного решения задачи.

Трудовзатраты на разработку и отладку программы

Расчет экономической эффективности и срока окупаемости проектируемой программы для ЭВМ начинается с расчета трудовых затрат.

Состав разработчиков: один fullstack-программист.

В себестоимость разработки входят: основная заработная плата разработчиков, дополнительная заработная плата, единый социальный налог, затраты на использование машинного времени, затраты на носители информации, затраты на текущий и профилактический ремонт вычислительной техники, прочие эксплуатационные расходы.

В итоге формула для вычисления себестоимости программного продукта выглядит следующим образом:

$$C_{пп} = Z_{тр} + Z_{м.вр} + Z_{н.и} + Z_{рем} + Z_{пр}, \quad (П1.1)$$

где $C_{пп}$ – себестоимость программного продукта, руб.;

$Z_{тр}$ – затраты на оплату труда, руб.;

$Z_{м.вр}$ – затраты на использование машинного времени, руб.;

$Z_{н.и}$ – затраты на носители информации, руб.;

$Z_{рем}$ – затраты на текущий и профилактический ремонт вычислительной техники, руб.;

$Z_{пр}$ – прочие эксплуатационных расходов, руб.

Требуется рассчитать затраты на оплату труда $Z_{тр}$.

Над созданием программного продукта работал 1 программист-выпускника и руководитель ВКР, которые работали с зарплатой по окладу, вычисляемой по формуле:

$$Z_{тр} = (Z_{общ} + Отч) * T_n, \quad (П1.2)$$

где $Z_{\text{общ}}$ – общая зарплата работника за час, руб.;

Отч – отчисления с зарплаты, %;

T_n – время написания программы, ч.

$$Z_{\text{пр}} = C_{T_{\text{пр}}}, \quad (\text{П1.3})$$

Дополнительная зарплата рассчитывается по формуле:

$$Z_{\text{доп}} = \frac{Z_{\text{пр}} * N_{\text{доп}}}{100\%}, \quad (\text{П1.4})$$

где $Z_{\text{пр}}$ – заработная плата работника за час, руб.;

$N_{\text{доп}}$ – норма отчислений на дополнительную зарплату (10%).

В итоге общая зарплата формируется на основе их:

$$Z_{\text{общ}} = Z_{\text{пр}} + Z_{\text{доп}}. \quad (\text{П1.5})$$

Отчисления «Отч на соцстрах», фонд занятости и пенсионный фонд вычисляются по следующей формуле:

$$\text{Отч} = O_{\text{сс}} + O_{\text{фз}} + O_{\text{пф}}, \quad (\text{П1.6})$$

где $O_{\text{сс}}$ – отчисления на соцстрах (0,5% от $Z_{\text{общ}}$);

$O_{\text{фз}}$ – отчисления в фонд занятости (0,5% от $Z_{\text{общ}}$);

$O_{\text{пф}}$ – отчисления в пенсионный фонд (2% от $Z_{\text{общ}}$).

Далее необходимо рассчитать заработную плату программиста-выпускника с окладом 49000 руб., который занимался разработкой с 1 февраля 2023 года до 15 июня 2023 года по 6 часов в рабочие дни недели. Расчеты производятся по формулам (2) – (6):

$$З_{\text{доп}} = 49000 * 10\% = 4900 \text{ (руб./мес.)}$$

$$З_{\text{общ}} = 49000 + 4900 = 53900 \text{ (руб./мес.)}$$

$$О_{\text{сс}} = 53900 * 0,5\% = 269,5 \text{ (руб.)}$$

$$О_{\text{фз}} = 53900 * 0,5\% = 269,5 \text{ (руб.)}$$

$$О_{\text{пф}} = 53900 * 2\% = 1078 \text{ (руб.)}$$

$$О_{\text{тч}} = 269,5 + 269,5 + 1078 = 1617 \text{ (руб.)}$$

$$З_{\text{тр}} = (53900 + 1617) * 4,5 = 249826,5 \text{ (руб.)}$$

После этого необходимо рассчитать заработную плату руководителя выпускной квалификационной работы с окладом 31226 руб., который занимался разработкой с 1 февраля 2023 года до 15 июня 2023 года. Расчеты производятся по формулам (2) – (6):

$$З_{\text{доп}} = 31226 * 10\% = 3122,6 \text{ (руб./мес.)}$$

$$З_{\text{общ}} = 31226 + 3122,6 = 34348,6 \text{ (руб./мес.)}$$

$$О_{\text{сс}} = 34348,6 * 0,5\% = 171,74 \text{ (руб.)}$$

$$О_{\text{фз}} = 34348,6 * 0,5\% = 171,74 \text{ (руб.)}$$

$$О_{\text{пф}} = 34348,6 * 2\% = 686,97 \text{ (руб.)}$$

$$О_{\text{тч}} = 171,74 + 171,74 + 697,4 = 1030,45 \text{ (руб.)}$$

$$З_{\text{тр}} = (34348,6 + 1030,45) * 4,5 = 159205,73 \text{ (руб.)}$$

Таким образом, затраты на заработную плату составляют 409032,23 рубля.

Все данные по заработной плате сводятся в табл.П1.5.

Таблица П1.5

Данные по заработной плате

Должность разработчика	Время работы, мес.	С _т , руб.	З _{доп} , руб.	З _{общ} , руб.	Отч, руб.	З _{тр} , руб.
Программист-выпускник	10	49000	4900	53900	1617	249826,5
Руководитель ВКР	10	31226	3122,6	34348,6	1030,45	159205,73
Итого:		80226	8022,6	88248,6	2647,45	409032,23

Затраты на использование машинного времени вычисляются по формуле:

$$З_{м.вр} = C_{м.вр} * Вр_{в.т}, \quad (П1.7)$$

где $З_{м.вр}$ – затраты на использование машинного времени, руб.;

$C_{м.вр}$ – стоимость одного часа машинного времени, руб./ч;

$Вр_{в.т}$ – время использования вычислительной техники, ч.

Стоимость одного часа машинного времени рассчитывается по формуле:

$$C_{м.вр} = \frac{Ц_k}{C_{сл.к} * K_{р.д} * Вр_c} + C_{т_э} * M_{вс}, \quad (П1.8)$$

где $C_{м.вр}$ – стоимость одного часа машинного времени, руб./ч;

$Ц_k$ – покупная цена компьютера, руб.;

$C_{сл.к}$ – срок службы компьютера, год;

$K_{р.д}$ – количество рабочих дней в году;

$Вр_c$ – время работы компьютера в течение суток, ч;

$C_{т_э}$ – стоимость одного кВт*ч электроэнергии, руб.;

$M_{вс}$ – мощность вычислительной системы, кВт.

Время использования вычислительной техники рассчитывается по следующей формуле:

$$Вр_{в.т.} = K_{д.р.} * Вр_c, \quad (П1.9)$$

где $Вр_{в.т.}$ – время использования вычислительной техники, ч;

$K_{д.р.}$ – количество дней разработки ПО.

Затраты на носители информации принимаются в размере 2% от цены вычислительной техники.

Затраты на текущий и профилактический ремонт принимаются в размере 4% от цены вычислительной техники.

Прочие эксплуатационные расходы включают в себя затраты на освещение, отопление, охрану, уборку и текущий ремонт помещений. Они принимаются в размере 10% от стоимости помещения (или его аренды), где происходит разработка программного продукта.

Время работы компьютера в течение суток составляет 6 часов, а количество дней на разработку ПО составляет 90 рабочих дней за 4,5 месяца с февраля по июнь.

Таким образом, по формуле (9) получаем, что время работы вычислительной техники:

$$V_{p.v.t.} = 90 * 6 = 540 \text{ (ч.)}$$

Покупная цена персонального компьютера разработчика составляет 70000 руб. и имеет срок службы 4 года, количество рабочих дней в году составляет 247 дней, в течение суток он будет работать 6 часов. Его мощность составляет 0,311 кВт, тогда как стоимость электроэнергии составляет 3 рубля за 1 кВт*ч.

Тогда по формуле (8) можно вычислить стоимость одного часа машинного времени:

$$C_{m.vp} = \frac{70000}{4 * 247 * 6} + 3 * 0,311 \approx 12,7 \text{ (руб./ч)}$$

По формуле (7) вычислим затраты на использование машинного времени:

$$Z_{m.vp} = 12,7 * 540 = 6858 \text{ (руб.)}$$

Затраты на носители информации:

$$70000 * 2\% = 1400 \text{ (руб.)}$$

Затраты на текущий и профилактический ремонт:

$$70000 * 4\% = 2800 \text{ (руб.)}$$

Прочие эксплуатационные расходы:

$$11000 * 0,1 = 1100 \text{ (руб.)}$$

Таким образом по формуле (1) рассчитаем себестоимость разработки ПО:

$$C_{пп} = 409032,23 + 6858 + 1400 + 2800 + 1100 = 421190,23 \text{ (руб.)}$$

Расчет цены программного продукта

Для определения минимальной цены, ниже которой разработчику будет невыгодно продавать программный продукт, используется следующая формула:

$$Ц_{п.п} = C_{п.п} * (1 + H_{пр}), \quad (П1.10)$$

где $Ц_{п.п}$ – цена программного продукта, руб.;

$C_{п.п}$ – себестоимость программного продукта, руб.;

$H_{пр}$ – норматив прибыли (20 %, в формуле $H_{пр} = 0,2$).

Тогда по этой формуле получаем стоимость разрабатываемой системы для пользователя:

$$Ц_{п.п} = 421190,23 * (1 + 0,2) = 509028,276 \text{ (руб.)}$$

Расчет экономической эффективности

В расчет экономической эффективности входит определение эксплуатационных расходов и капитальных затрат потребителя, годовой экономии эксплуатационных расходов у одного потребителя, срока окупаемости программного продукта, годового экономического эффекта.

Расходы потребителя, связанные с эксплуатацией программы, определяются по следующей формуле:

$$P_{э.п.} = V_{р.п.} * C_{м.вр} + \frac{Ц_{п.п}}{C_{сл}}, \quad (П1.11)$$

где $P_{э.п.}$ – эксплуатационные расходы потребителя, руб.;

$V_{р.п.}$ – объем машинного времени в течение года, необходимый для решения данной задачи с использованием программы, ч;

$C_{м.вр}$ – стоимость одного часа машинного времени, руб./ч;

$C_{п.п}$ – цена программного продукта, руб.;

$C_{сл}$ – срок службы программного продукта, год.

Если у пользователей будет компьютер стоимостью 30000 рублей, который обладает мощностью 0,224 кВт, тогда как стоимость электроэнергии составляет 3 рубля за кВт*ч, то по формуле (8) можно вычислить стоимость одного часа машинного времени у пользователя:

$$C_{м.вр} = \frac{30000}{2 \cdot 247 \cdot 1} + 3 \cdot 0,224 = 61,3 \text{ (руб./ч)}$$

Пусть программа будет работать 247 дней по 1 часу в день и иметь срок службы в два года. Тогда по формуле (11) можно вычислить расходы потребителя при использовании разработанного ПО:

$$P_{э.п} = 247 \cdot 61,3 + \frac{740420,076}{2} = 385351,138 \text{ (руб.)}$$

Если на момент внедрения программного продукта у потребителя все работы выполнялись вручную, тогда капитальные затраты рассчитываются по формуле:

$$P_{кап} = \frac{P_{п.п.} \cdot K_{ЭВМ}}{\Phi_{вр}} + C_{п.п.}, \quad (П1.12)$$

где $P_{кап}$ – капитальные расходы потребителя, руб.;

$\Phi_{вр}$ – полезный годовой фонд времени работы вычислительной техники, принимается условно 2000 ч в год;

$K_{ЭВМ}$ – капитальные затраты на вычислительную технику, для которой предназначена программа, руб.

Капитальные затраты на вычислительную технику рассчитываются по формуле:

$$K_{ЭВМ} = C_{ЭВМ} + P_{п.п.}, \quad (П1.13)$$

где $C_{ЭВМ}$ – цена вычислительной техники, руб.;

$P_{п.п}$ – прочие расходы потребителя, связанные с помещением (отопление, освещение, уборка и т.д.), принимаются в размере 10 % от стоимости помещения потребителя (или его аренды), руб.

По формуле (13) можно вычислить капитальные затраты пользователя на вычислительную технику:

$$K_{ЭВМ} = 30000 + 2000 = 32000 \text{ (руб.)}$$

По формуле (12) можно узнать все капитальные расходы потребителя:

$$P_{\text{кап}} = \frac{247 \cdot 30000}{2000} + 740420,076 = 744125,076 \text{ (руб.)}$$

Для расчета годовой экономии эксплуатационных расходов потребителя вычисляются эксплуатационные затраты потребителя при решении задачи вручную применяют формулу:

$$P_{\text{э.руч}} = 1,21 \cdot \PhiЗП \cdot 12, \quad (\text{П1.14})$$

где $P_{\text{э.руч}}$ – эксплуатационные расходы потребителя при решении задачи вручную, руб.;

$\PhiЗП$ – фонд заработной платы персонала, обслуживающего решение задачи вручную, руб.;

12 – количество месяцев в году;

1,21 – поправочный коэффициент.

$$P_{\text{э.руч}} = 1,21 \cdot 72000 \cdot 12 = 1045440 \text{ (руб.)}$$

Тогда годовая экономия эксплуатационных расходов у одного потребителя рассчитывается по формуле:

$$\mathcal{E} = P_{\text{э.руч}} - P_{\text{э.п.}}, \quad (\text{П1.15})$$

Срок окупаемости программного продукта рассчитывается по формуле:

$$T_{\text{ок}} = \frac{P_{\text{кап}}}{\mathcal{E}}, \quad (\text{П1.16})$$

Годовой экономический эффект, получаемый одним потребителем, рассчитывается по формуле:

$$\mathcal{E}\mathcal{E} = \mathcal{E} - E_{\text{н}} * P_{\text{кап}}, \quad (\text{П1.17})$$

где $E_{\text{н}}$ – нормативный коэффициент эффективности дополнительных капитальных вложений, равный 0,15.

Годовая экономия эксплуатационных расходов у одного потребителя составляет:

$$\mathcal{E} = 1045440 - 744125,076 = 301314,924 \text{ (руб.)}$$

Срок окупаемости ИС составляет:

$$T_{\text{ок}} = \frac{744125,076}{301314,924} = 2,4 \text{ (года)}$$

Годовой экономический эффект составляет:

$$\mathcal{E}\mathcal{E} = 301314,924 - 0,15 * 744125,076 = 189626,2076 \text{ (руб.)}$$

Основные риски проекта

Риски в рамках реализуемого проекта представлены в табл.П1.6.

Таблица П1.6.

Риски проекта

Риски проекта	Мероприятия по управлению рисками (по предотвращению риска / по реагированию при возникновении риска)
Недостаток тестового набора изображений для обучения нейросети	Сгенерировать дополнительно тестовые изображения с помощью утилитарной программы, нарисовать макеты от руки самостоятельно;

Недостаток вычислительных мощностей для обучения спроектированной нейросети	Запросить у компании-заказчика дополнительное оборудование
---	--

Реальная бизнес-ситуация, служащая обоснованием проекта с данными о прибыли на инвестиции

Проект не является коммерческим, доходы по проекту в период его реализации не предусмотрены.

Текст программы

```

import { Component, OnInit, OnDestroy,
AfterViewInit, EnvironmentInjector } from
'@angular/core';
import { registerLocaleData } from
'@angular/common';
import localeRu from
'@angular/common/locales/ru';
import { Platform } from '@ionic/angular';
import { TranslateService } from '@ngx-
translate/core';
import { Store, select } from
'@ngrx/store';
import { Subject, SubscriptionLike } from
'rxjs';
import { take, takeUntil } from
'rxjs/operators';
import dayjs from 'dayjs';
import utc from 'dayjs/plugin/utc';
import timezone from
'dayjs/plugin/timezone';
import customParseFormat from
'dayjs/plugin/customParseFormat';
import 'dayjs/locale/ru';

import packageInfo from 'package.json';

import { AppConfigService } from
'src/app/services/app-config/app-
config.service';
import { UtilsService } from
'src/app/services/utils/utils.service';

import {
  metaToggleMenu,
  clearAppCache,
  metaSetVersion,
} from './store/actions';
import { IUserState } from
'src/app/store/state';
import {
  selectUser,
  selectVersion,
} from 'src/app/store/selectors';

@Component({
  selector: 'app-root',
  templateUrl: 'app.component.html',
  styleUrls: ['app.component.scss'],
})
export class AppComponent implements
OnInit, OnDestroy, AfterViewInit {
  unsubscribe$: Subject<void>;
  clearCacheSubs: SubscriptionLike;
  user: any = {};

  isMenuCompact = false;
  isMenuEnabled = false;

  constructor(
    private env: AppConfigService,
    private platform: Platform,
    private translateService:
TranslateService,
    private store: Store<{ user:
IUserState; }>,
    private utils: UtilsService,
    private environmentInjector:
EnvironmentInjector
  ) {
    this.unsubscribe$ = new
Subject<void>();

    // инициализация языка

    this.translateService.setDefaultLang('ru');
    registerLocaleData(localeRu, 'ru');
    this.translateService.use('ru');

    // инициализация библиотеки dayjs
    dayjs.extend(utc);
    dayjs.extend(timezone);
    dayjs.extend(customParseFormat);
    dayjs.locale('ru');
  }

  ngOnInit() {
    this.platform.ready().then(async () =>
{
      this.store
        .pipe(select(selectUser),
takeUntil(this.unsubscribe$))
        .subscribe((user: IUserState) => {
          this.user = user;
        });

      this.store.pipe(select(selectVersion),
take(1)).subscribe((version: string) => {
        // если версия приложения
отличается от версии в сторе, то сбрасываем
кэш
        if (packageInfo.version !==
version) {
          this.utils.clearStorage();
        }

        this.store.dispatch(clearAppCache());
      });
    });
  }
}

```

Рис.П2.1. Текст программы модуля «app.component.ts»

П2.1. Продолжение

```

this.store.dispatch(metaSetVersion({
  version: packageInfo.version }));
});
});
}

toggleMenu() {
  this.isMenuCompact =
!this.isMenuCompact;
  this.store.dispatch(metaToggleMenu({
    isMenuCompact: this.isMenuCompact }));
}

import { IObject } from
"src/app/services/neural/neural.service";

export interface
IConvertedNeuralDataToHtmlResult {
  layout: () => HTMLElement,
  objects: IObject[],
  editorObjects: any[]
}

export abstract class HtmlParser<T> {
  abstract parseNeuralData(data: any[]): {
    parseResult: T, objects: IObject[] };
  abstract convertNeuralDataToHTML(
    parserResult: T,
    container: HTMLElement,
    objects: IObject[],
    editorObjects: any[]
  ): IConvertedNeuralDataToHtmlResult;
};

```

Рис.П2.2. Текст программы модуля «html-parser.ts»

```

paddingRight?: number
}

export class RowsAndColumnsStrategy extends
HtmlParser<INeuralParserResult> {
  private elementsFactory:
HTMLElementFactory = new
HTMLElementFactory();
  private parseDataIndex: number = -1;
  private htmlLayout: HTMLLayout = new
HTMLLayout(null, null);
  private mainContainerWidthAndHeight =
null;

  calculateOverlap(obj1: IObject, obj2:
IObject, field: 'x' | 'y'): number {
    const start = Math.max(obj1[field][0],
obj2[field][0]);
    const end = Math.min(obj1[field][1],
obj2[field][1]);
    const overlap = Math.max(0, end -
start);
    const total = Math.min(obj1[field][1] -
obj1[field][0], obj2[field][1] -
obj2[field][0]);
    return overlap / total;
  }
}

```

```

import { CLASSES } from
"src/app/services/neural/constants";
import { IObject } from
"src/app/services/neural/neural.service";
import { HtmlParser } from "../html-parser";
import { findMinAndMaxCoords } from
"./metrics";
import { HTMLElementFactory } from
"src/core/html-element-factory";
import { ENGINE_ELEMENTS_TYPES } from
"src/app/pages/constants";
import { HTMLLayout } from
"src/core/layout";

interface INeuralParserResult {
  rows?: INeuralParserResult[],
  columns?: INeuralParserResult[],
  object?: IObject,
  width?: number,
  height?: number,
  minY?: number,
  maxY?: number,
  minX?: number,
  maxX?: number,
  paddingTop?: number,
  paddingBottom?: number,
  paddingLeft?: number,

```

Рис.П2.3. Текст программы модуля «rows-and-columns-strategy.ts»

П2.3. Продолжение

```

splitObjectsByRowsAndColumns(objects:
IObject[], result: INeuralParserResult) {
    const rows =
this.splitObjectsByRows(objects);

    if (rows.length === 1 && objects.length
=== 1) {
        result['object'] = rows[0][0];
        return;
    }

    result['rows'] = [...rows].map((row) =>
{ return {} });

    rows.forEach((row, rowIndex) => {
        const columns =
this.splitObjectsByColumns(row);

        if (columns.length === 1) {
            result['rows'][rowIndex] = {
object: columns[0][0] };
            return;
        }

        result['rows'][rowIndex]['columns'] =
[...columns].map((column) => { return {}
});

        columns.forEach((column, columnIndex)
=> {

this.splitObjectsByRowsAndColumns(column,
result['rows'][rowIndex]['columns'][columnI
ndex]);
        });
    });
}

splitObjectsByRows(objects: IObject[]):
IObject[][] {
    const rows: Map<number, IObject>[] =
[];

    // сначала сортируем объекты по высоте
objects.sort((a, b) => a.y[0] > b.y[0]
? 1 : -1);

    objects.forEach((currentObject) => {
        let objectHasIntercept = false;
        let currentRowIndex = -1;

        objects.forEach((object) => {
            if (currentObject.id !== object.id)
{
                // если объект имеет пересечение
с другим объектом
                if
(this.calculateOverlap(currentObject,
object, 'y') > 0.2) {
                    objectHasIntercept = true;
                }
            }
        });

        if (objectHasIntercept) {
            return;
        }

        if (currentRowIndex === -1) {
            rows.push(new Map());
            currentRowIndex = rows.length
- 1;
        }

        rows[currentRowIndex].set(currentObject.id,
currentObject);

        rows[currentRowIndex].set(object.id,
object);
    });

    return rows.map((row) =>
[...row].map(([name, value]) => value));
}

splitObjectsByColumns(objects:
IObject[]): IObject[][] {
    const columns: Map<number, IObject>[] =
[];

    // сначала сортируем объекты по ширине
objects.sort((a, b) => a.x[0] > b.x[0]
? 1 : -1);

    objects.forEach((currentObject) => {
        let objectHasIntercept = false;

        objects.forEach((object) => {
            if (currentObject.id !== object.id)
{
                // если объект имеет пересечение
с другим объектом
                if
(this.calculateOverlap(currentObject,
object, 'x') > 0.2) {
                    objectHasIntercept = true;
                }
            }
        });

        if (objectHasIntercept) {
            return;
        }

        if (currentColumnIndex === -1) {
            columns.push(new Map());
            currentColumnIndex = columns.length
- 1;
        }

        columns[currentColumnIndex].set(currentObject.id,
currentObject);

        columns[currentColumnIndex].set(object.id,
object);
    });

    return columns.map((column) =>
[...column].map(([name, value]) => value));
}

```

```

currentRowIndex =
rows.findIndex((row) =>
row.has(currentObject.id));

    if (currentRowIndex === -1) {
        rows.push(new Map());
        currentRowIndex = rows.length
- 1;
    }

    rows[currentRowIndex].set(currentObject.id,
currentObject);

    rows[currentRowIndex].set(object.id,
object);
}

    if (!objectHasIntercept) {
        rows.push(new Map());
        rows[rows.length -
1].set(currentObject.id, currentObject);
    }

    return rows.map((row) =>
[...row].map(([name, value]) => value));
}

splitObjectsByColumns(objects:
IObject[]): IObject[][] {
    const columns: Map<number, IObject>[] =
[];

    // сначала сортируем объекты по ширине
objects.sort((a, b) => a.x[0] > b.x[0]
? 1 : -1);

    objects.forEach((currentObject) => {
        let objectHasIntercept = false;

        objects.forEach((object) => {
            if (currentObject.id !== object.id)
{
                // если объект имеет пересечение
с другим объектом
                if
(this.calculateOverlap(currentObject,
object, 'x') > 0.2) {
                    objectHasIntercept = true;
                }
            }
        });

        if (objectHasIntercept) {
            return;
        }

        if (currentColumnIndex === -1) {
            columns.push(new Map());
            currentColumnIndex = columns.length
- 1;
        }

        columns[currentColumnIndex].set(currentObject.id,
currentObject);

        columns[currentColumnIndex].set(object.id,
object);
    });

    return columns.map((column) =>
[...column].map(([name, value]) => value));
}

```

П2.3. Продолжение

```

columns[currentColumnIndex].set(object.id,
object);
    } else {
        columns.push(new Map());
        columns[columns.length -
1].set(currentObject.id, currentObject);
        columns[columns.length -
1].set(object.id, object);
    }
}
});

if (!objectHasIntercept) {
    columns.push(new
Map().set(currentObject.id,
currentObject));
}
});

return columns.map((row) =>
[...row].map(([name, value]) => value));
}

/** Метод парсит данные, приходящие с
нейросети
* в формат: rows[], columns[]
*/
parseNeuralData(data: any): {
parseResult: INeuralParserResult; objects:
IObject[]; } {
    const objects = data.map((object,
index) => {
        return {
            ...object,
            id: index
        };
    });

    let result: INeuralParserResult = {};

this.splitObjectsByRowsAndColumns(objects,
result);

    return {
        parseResult: result,
        objects: objects
    };
}

/** Рекурсивная функция подсчёта ширины
* и высоты для получившейся структуры
DSL */

calculateWidthAndHeightForGridAndObjects(pa
rserResult: INeuralParserResult, objects:
IObject[] = []) {
    const accumCountOfObjects =
(rowOrColumn: INeuralParserResult,

```

```

accumObjects: IObject[] = []): IObject[] =>
{
    if (rowOrColumn.object) {
        accumObjects.push(rowOrColumn.object);
    }

    rowOrColumn.rows?.forEach((item) => {
        accumObjects =
accumCountOfObjects(item, accumObjects);
    });

    rowOrColumn.columns?.forEach((item)
=> {
        accumObjects =
accumCountOfObjects(item, accumObjects);
    });

    return accumObjects;
});

const
calculateMaxHeightAndWidthOfObjects =
(objects: IObject[]): {
    width: number,
    height: number,
    maxY: number,
    minY: number,
    maxX: number,
    minX: number
} => {
    let maxWidth = -9999, maxHeight = -
9999,
        maxY = -9999, minY = 9999,
        maxX = -9999, minX = 9999;

    objects.forEach((object) => {
        if (object.x[1] - object.x[0] >=
maxWidth) {
            maxWidth = object.x[1] -
object.x[0];
        }

        if (object.y[1] - object.y[0] >=
maxHeight) {
            maxHeight = object.y[1] -
object.y[0];
        }

        if (object.y[1] >= maxY) {
            maxY = object.y[1];
        }

        if (object.y[0] <= minY) {
            minY = object.y[0];
        }

        if (object.x[1] >= maxX) {
            maxX = object.x[1];
        }
    });
}

```

П2.3. Продолжение

```

    if (object.x[0] <= minX) {
      minX = object.x[0];
    }
  });

  return {
    width: maxWidth,
    height: maxHeight,
    minY,
    maxY,
    minX,
    maxX
  };
};

if (objects.length) {
  // для самого родительского
  контейнера значение width и height в
  пикселях
  this.mainContainerWidthAndHeight =
  findMinAndMaxCoords(objects.map((object) =>
  [object.x[0], object.y[0], object.x[1] -
  object.x[0], object.y[1] - object.y[0]]));
  parserResult.width =
  this.mainContainerWidthAndHeight.maxX -
  this.mainContainerWidthAndHeight.minX;
  parserResult.height =
  this.mainContainerWidthAndHeight.maxY -
  this.mainContainerWidthAndHeight.minY;
}

let sumOfRowsHeight = 0,
sumOfColumnsWidth = 0, sumOfColumnsHeight =
0, sumOfRowsWidth = 0,
  columnsMinY = 9999, columnsMaxY = -
9999, columnsMinX = 9999, columnsMaxX = -
9999,
  rowsMinY = 9999, rowsMaxY = -9999,
rowsMinX = 9999, rowsMaxX = -9999;
  parserResult?.rows?.forEach((row) => {
    const objects =
    accumCountOfObjects(row);
    const maxWidthAndHeight =
    calculateMaxHeightAndWidthOfObjects(objects
  );
    sumOfRowsHeight +=
    maxWidthAndHeight.height;
    sumOfRowsWidth +=
    maxWidthAndHeight.width;
    rowsMinY = maxWidthAndHeight.minY <=
    rowsMinY ? maxWidthAndHeight.minY :
    rowsMinY;
    rowsMaxY = maxWidthAndHeight.maxY >=
    rowsMaxY ? maxWidthAndHeight.maxY :
    rowsMaxY;
    rowsMinX = maxWidthAndHeight.minX <=
    rowsMinX ? maxWidthAndHeight.minX :
    rowsMinX;

```

```

    rowsMaxX = maxWidthAndHeight.maxX >=
    rowsMaxX ? maxWidthAndHeight.maxX :
    rowsMaxX;

    row.width = maxWidthAndHeight.width;
    row.height =
    maxWidthAndHeight.height;
    row.minX = maxWidthAndHeight.minX;
    row.maxX = maxWidthAndHeight.maxX;
    row.minY = maxWidthAndHeight.minY;
    row.maxY = maxWidthAndHeight.maxY;
  });

  parserResult?.rows?.forEach((row,
  rowIndex) => {
    row.paddingTop = rowIndex > 0 ?
    (row.minY - parserResult.rows[rowIndex -
    1].maxY) /
    (this.mainContainerWidthAndHeight.maxY -
    this.mainContainerWidthAndHeight.minY) *
    100 : 0;

    row.paddingLeft = row.minX > rowsMinX
    ? (row.minX - rowsMinX) /
    (this.mainContainerWidthAndHeight.maxX -
    this.mainContainerWidthAndHeight.minX) *
    100 : 0;
    row.paddingRight = row.maxX <
    rowsMaxX ? (rowsMaxX - row.maxX) /
    (this.mainContainerWidthAndHeight.maxX -
    this.mainContainerWidthAndHeight.minX) *
    100 : 0;

    row.height = Number((row.height /
    sumOfRowsHeight).toFixed(2));
    row.width = Number((row.width /
    sumOfRowsWidth).toFixed(2));

    this.calculateWidthAndHeightForGridAndObjec
    ts(row);
  });

  parserResult?.columns?.forEach((column)
  => {
    const objects =
    accumCountOfObjects(column);
    const maxWidthAndHeight =
    calculateMaxHeightAndWidthOfObjects(objects
  );
    sumOfColumnsWidth +=
    maxWidthAndHeight.width;
    sumOfColumnsHeight +=
    maxWidthAndHeight.height;
    columnsMinY = maxWidthAndHeight.minY
    <= columnsMinY ? maxWidthAndHeight.minY :
    columnsMinY;
    columnsMaxY = maxWidthAndHeight.maxY
    >= columnsMaxY ? maxWidthAndHeight.maxY :
    columnsMaxY;

```

П2.3. Продолжение

```

        columnsMinX = maxWidthAndHeight.minX
<= columnsMinX ? maxWidthAndHeight.minX :
columnsMinX;
        columnsMaxX = maxWidthAndHeight.maxX
>= columnsMaxX ? maxWidthAndHeight.maxX :
columnsMaxX;

        column.width =
maxWidthAndHeight.width;
        column.height =
maxWidthAndHeight.height;
        column.minY = maxWidthAndHeight.minY;
        column.maxY = maxWidthAndHeight.maxY;
        column.minX = maxWidthAndHeight.minX;
        column.maxX = maxWidthAndHeight.maxX;
    });

    parserResult?.columns?.forEach((column,
colIndex) => {
        column.paddingTop = column.minY >
columnsMinY ? (column.minY - columnsMinY) /
(this.mainContainerWidthAndHeight.maxY -
this.mainContainerWidthAndHeight.minY) *
100 : 0;
        column.paddingBottom = column.maxY <
columnsMaxY ? (columnsMaxY - column.maxY) /
(this.mainContainerWidthAndHeight.maxY -
this.mainContainerWidthAndHeight.minY) *
100 : 0;

        column.paddingLeft = colIndex > 0 ?
(column.minX -
parserResult.columns[colIndex - 1].maxX) /
(this.mainContainerWidthAndHeight.maxX -
this.mainContainerWidthAndHeight.minX) *
100 : 0;

        column.width = Number((column.width /
sumOfColumnsWidth).toFixed(2));
        column.height = Number((column.height
/ sumOfColumnsHeight).toFixed(2));

this.calculateWidthAndHeightForGridAndObjec
ts(column);
    });
}

/** Конвертирование DSL данных в HTML-код
*/
convertNeuralDataToHTML(
    parserResult: INeuralParserResult,
    container: HTMLElement = null,
    objects: IObject[] = [],
    editorObjects: any[] = []
): { layout: () => HTMLElement; objects:
IObject[]; editorObjects: any[] } {
    if (!container) {
        this.parseDataIndex = -1;
        container =
this.htmlLayout.createElement();

```

```

this.convertNeuralDataToHTML(parserResult,
container, objects, editorObjects);
    } else {
        parserResult.rows?.forEach((row) => {
            const childContainer =
this.htmlLayout.createRow(container);

            if (row.paddingTop) {
                childContainer.style.paddingTop =
`${row.paddingTop}%`;
            }

            if (row.paddingLeft) {
                childContainer.style.paddingLeft
= `${row.paddingLeft}%`;
            }

            if (row.paddingRight) {
                childContainer.style.paddingRight
= `${row.paddingRight}%`;
            }
            this.convertNeuralDataToHTML(row,
childContainer, objects, editorObjects);
        });

    parserResult.columns?.forEach((column) => {
        const childContainer =
this.htmlLayout.createColumn(container);

        if (column.paddingTop) {
            childContainer.style.paddingTop =
`${column.paddingTop}%`;
        }

        if (column.paddingBottom) {
            childContainer.style.paddingBottom =
`${column.paddingBottom}%`;
        }

        if (column.paddingLeft) {
            childContainer.style.paddingLeft
= `${column.paddingLeft}%`;
        }

        this.convertNeuralDataToHTML(column,
childContainer, objects, editorObjects);
    });

    if (parserResult.object) {
        this.parseDataIndex++;

        const elementTemplate =
this.elementsFactory.create(parserResult.ob
ject.className as ENGINE_ELEMENTS_TYPES);
        if (elementTemplate) {
            const htmlElement =
elementTemplate.createElement();

```

П2.3. Продолжение

```

container.appendChild(htmlElement);

editorObjects.push(elementTemplate);
}

const renderedElement =
container.lastElementChild;
renderedElement.setAttribute('id',
'object' +
(this.parseDataIndex).toString());
(renderedElement as
HTMLElement).style.setProperty('min-
height', `${parserResult.object.y[1] -
parserResult.object.y[0]}px`);
(renderedElement as
HTMLElement).style.setProperty('min-width',
`${parserResult.object.x[1] -
parserResult.object.x[0]}px`);
if (
  parserResult.object.classname ===
CLASSES.picture ||
  parserResult.object.classname ===
CLASSES.slider ||
  parserResult.object.classname ===
CLASSES.video
) {
  (renderedElement as
HTMLElement).style.setProperty('height',
`${parserResult.object.y[1] -
parserResult.object.y[0]}px`);
  (renderedElement as
HTMLElement).style.setProperty('min-width',
`${parserResult.object.x[1] -
parserResult.object.x[0]}px`);
  // (renderedElement as
HTMLElement).style.setProperty('max-width',

```

```

`${parserResult.object.x[1] -
parserResult.object.x[0]}px`);
  // (renderedElement as
HTMLElement).style.setProperty('width',
`${parserResult.object.x[1] -
parserResult.object.x[0]}px`, 'important');
}

  if (parserResult.object.classname
=== CLASSES.text) {
    (renderedElement as
HTMLElement).style.setProperty('height',
`${parserResult.object.y[1] -
parserResult.object.y[0]}px`);
    // (renderedElement as
HTMLElement).style.setProperty('width',
`${parserResult.object.x[1] -
parserResult.object.x[0]}px`, 'important');
  }

parserResult.object.parentHtmlContainer =
container;

  objects.push(parserResult.object);
}

return {
  layout: () =>
this.htmlLayout.getElement(),
  objects: objects,
  editorObjects: editorObjects
};
}

```

```

import { ComponentRef, Injectable,
Renderer2, RendererFactory2,
ViewContainerRef } from '@angular/core';

import { UtilsService } from
'src/app/services/utils/utils.service';
import { HtmlParser,
IConvertedNeuralDataToHtmlResult } from
'./html-parser-strategies/html-parser';
import { RowsAndColumnsStrategy } from
'./html-parser-strategies/rows-and-columns-
strategy';
import { CLASSES, getDefaultElementData }
from './constants';
import { normalizeCoords,
normalizeCoordsByResolution } from
'./metrics';
import { BoundingBox, meanSquaredError,
rootMeanSquaredError } from
'./metrics/mean-squared-error';
import { averagePrecision,
intersectionOverUnion } from
'./metrics/intersection-over-union';

export interface IObject {
  id?: number,
  x: [number, number],
  y: [number, number],
  classname: string,
  objScore: number,
  htmlTop?: number,
  htmlLeft?: number,
  htmlWidth?: number,
  htmlHeight?: number,
  parentHtmlContainer?: HTMLElement
}

export interface IMetricsResult {
  mse: number,
  rmse: number,
  averageIoU: string,
}

@Injectable({
  providedIn: 'root',
})
export class NeuralService {
  private htmlParser: HtmlParser<any> = new
RowsAndColumnsStrategy();
  private renderer: Renderer2;

  constructor(
    private utils: UtilsService,
    private rendererFactory:
RendererFactory2
  ) {
    this.renderer =
rendererFactory.createRenderer(null, null);
  }

  /** Метод получает текущую стратегию для
формирования HTML из DSL */
  getHtmlParser(): HtmlParser<any> {
    return this.htmlParser;
  }

  /** Метод рендерит DSL в HTML и
отображает сразу же этот HTML */
  renderDSLToHTML(renderContainer:
ViewContainerRef, objects: IObject[]): {
    convertedHTMLData:
IConvertedNeuralDataToHtmlResult,
    objects: IObject[],
    componentRef: ComponentRef<any>
  } {
    for (let child of
renderContainer.element.nativeElement.child
ren) {
      this.renderer.removeChild(renderContainer.e
lement.nativeElement, child);
    }

    const neuralHtmlParser =
this.getHtmlParser();
    const parsedNeuralData =
neuralHtmlParser.parseNeuralData(objects);

    if (neuralHtmlParser instanceof
RowsAndColumnsStrategy) {
      neuralHtmlParser.calculateWidthAndHeightFor
GridAndObjects(parsedNeuralData.parseResult
, parsedNeuralData.objects);
    }

    const convertedHTMLData =
neuralHtmlParser.convertNeuralDataToHTML(pa
rsedNeuralData.parseResult, null, [], []);

    this.renderer.appendChild(renderContainer.e
lement.nativeElement,
convertedHTMLData.layout());
    this.utils.createDynamicComponent({
      props: {
        data:
convertedHTMLData.objects.map((object) => {
          return
getDefaultElementData(CLASSES[object.classn
ame]));
        }),
      styles: {
        backgroundColor: '#525770'
      }
    },
    container: renderContainer
  });

    return {

```

Рис.П2.4. Текст программы модуля «neural.service.ts»

П2.4. Продолжение

```

        convertedHTMLData: convertedHTMLData,
        objects: convertedHTMLData.objects,
        componentRef:
renderContainer.element.nativeElement
    };
}

/** Подсчет метрик по прямоугольникам */
calculateMetrics(
    predictedBoxesOriginalCoords:
BoundingBox[],
    actualBoxesOriginalCoords:
BoundingBox[],
    currentWindowWidth: number,
    currentWindowHeight: number
): IMetricsResult {
    const predictedBoxes: BoundingBox[] =
normalizeCoordsByResolution(normalizeCoords
(JSON.parse(JSON.stringify(predictedBoxesOriginalCoords)), false), [448, 448],
[currentWindowWidth, currentWindowHeight]);
    const actualBoxes: BoundingBox[] =
normalizeCoordsByResolution(normalizeCoords
(JSON.parse(JSON.stringify(actualBoxesOriginalCoords)), false), [448, 448],
[currentWindowWidth, currentWindowHeight]);
    const predictedBoxesRelative:
BoundingBox[] =
normalizeCoordsByResolution(normalizeCoords
(JSON.parse(JSON.stringify(predictedBoxesOriginalCoords)), true), [448, 448],
[currentWindowWidth, currentWindowHeight]);
    const actualBoxesRelative:
BoundingBox[] =
normalizeCoordsByResolution(normalizeCoords
(JSON.parse(JSON.stringify(actualBoxesOriginalCoords)), true), [448, 448],
[currentWindowWidth, currentWindowHeight]);

    return {
        mse: meanSquaredError(predictedBoxes,
actualBoxes),
        rmse:
rootMeanSquaredError(predictedBoxes,
actualBoxes),
        averageIoU:
intersectionOverUnion(predictedBoxesRelative, actualBoxesRelative),
    };
}

/** Метод подсчёта метрик для
получившегося результата конвертирования
HTML в DSL */

calculateMetricsByHTML(convertedHTMLData):
Promise<IMetricsResult> {

```

```

    return new Promise((resolve) => {
        setTimeout(() => {
            console.log('Objects: ',
convertedHTMLData.objects.map((object:
IObject, index) => {
                const neuralDivContainer =
document.getElementsByClassName('neural-
div-container')[0];
                const htmlElement =
document.getElementById(`object${index}`);
                const rect =
htmlElement.getBoundingClientRect();
                const rectNeural =
neuralDivContainer.getBoundingClientRect();
                object.htmlTop = rect.top -
rectNeural.top;
                object.htmlLeft = rect.left -
rectNeural.left;
                object.htmlWidth = rect.width;
                object.htmlHeight = rect.height;
                return object;
            }));

            const neuralDivContainer =
document.getElementsByClassName('neural-
div-container')[0];
            const rectNeural =
neuralDivContainer.getBoundingClientRect();
            const predictedBoxesOriginalCoords:
BoundingBox[] =
convertedHTMLData.objects.map((object) =>
[object.x[0], object.y[0], object.x[1] -
object.x[0], object.y[1] - object.y[0]]);
            const actualBoxesOriginalCoords:
BoundingBox[] =
convertedHTMLData.objects.map((object) =>
[object.htmlLeft, object.htmlTop,
object.htmlWidth, object.htmlHeight]);

            return resolve(
                this.calculateMetrics(
                    JSON.parse(JSON.stringify(predictedBoxesOriginalCoords)),
                    JSON.parse(JSON.stringify(actualBoxesOriginalCoords)),
                    rectNeural.width,
                    rectNeural.height
                )
            );
        }, 100);
    });
}

```

Спецификация

Настоящее приложение содержит спецификацию на разработанную программную документацию и программное обеспечение. Спецификация приведена в таблице (табл. ПЗ.1).

Таблица ПЗ.1

Спецификация на разработанную программную документацию

Обозначение	Назначение	Примечание
1	2	3
Документация		
Квашенников_ВКР.docx	Расчетно-пояснительная записка к магистерской диссертации на тему «Программное обеспечение генератора HTML-шаблонов по рукописным рисункам-макетам», приложения 2,4,5	
Компоненты		
app.module.ts	Основная компонента программы, где импортируются все модули	
app.component.ts	Основная компонента программы, выполняющая рендер HTML-страницы	
app.component.html	Компонента, в которой содержится HTML-код для отображения основного окна программы	
rows-and-columns-strategy.ts	Компонента, реализующая основной алгоритм для генерирования HTML-сетки с рукописных рисунков-макетов	
html-parser.ts	Компонента для преобразования результата генерирования в HTML-код для рендеринга на странице	

1	2	3
neural.service.ts	Компонента, объединяющая алгоритмы для генерации HTML-сетки	
block-appender.component.ts	Компонента для добавления новых блоков на страницу	
dropdown-edit-panel.component.ts	Компонента всплывающего окна для редактирования существующего шаблона	
editor-item.component.ts	Элемент редактирования объекта	
header.component.ts	Шапка HTML-страницы	
image-uploader.component.ts	Компонента для загрузки изображения на сервер	
image-uploader-input.component.ts	Компонента, отображающаяся при загрузке изображения с клиента	
input.component.ts	Компонента ввода данных	
metrics.component.ts	Компонента для вычисления показателей точности сгенерированной HTML-сетки	
neural-parser-to-html.component.ts	Компонента преобразования результата нейронной сети в фрагмент HTML-кода	
preloader.component.ts	Компонента, отображающаяся при загрузке каких-либо данных с сервера	
sites-breadcrumbs.component.ts	Компонента, реализующая «хлебные крошки» для перехода в предыдущие разделы	
ui-tree.component.ts	Компонента для отображения «дерева элементов»	
ui-tree-element-editor.component.ts	Компонента, используемая в «дереве элементов»	
version.component.ts	Компонента для отображения текущей версии программы	

Руководство пользователя

1. Описание программного обеспечения

Программный продукт предназначен для решения проблемы в компании ООО «АСТРОСОФТ ПРО», связанной с оптимизации временных затрат при верстке HTML-шаблонов во время разработки клиентской части приложений за счет генерирования HTML-шаблонов по рукописным рисункам-макетам.

2. Минимальные системные требования

Компьютер, на котором будет установлена система должен обладать следующими минимальными требованиями:

- объем оперативной памяти не менее 4 Гб;
- монитор с разрешением не менее 1280x720;
- операционная система семейства Windows (Windows 7, Windows 8, Windows 10, Windows 11);
- твердотельный накопитель объемом не менее 512 Гб;
- процессор Intel core I5, не ниже 6-го поколения, тактовая частота 2.1 ГГц.;
- установленная NodeJS и npm (менеджер пакетов);
- наличие установленного браузера современной версии (Google Chrome, Яндекс Браузер, Opera, Firefox).

3. Запуск программы

Для запуска программы необходимо иметь на компьютере установленный браузер современной версии. Программу можно запустить двумя сценариями: при сценарии, где программа развернута на внешнем доменом адресе; сценарий, при локальном запуске программы. В первом случае достаточно

ввести в поисковой строке браузера ссылку на сайт, где развернута программа.

Во втором случае запуск программы осуществляется следующим образом:

В первую очередь открывается терминал Windows (это может быть Windows Powershell) (рис.П4.1).

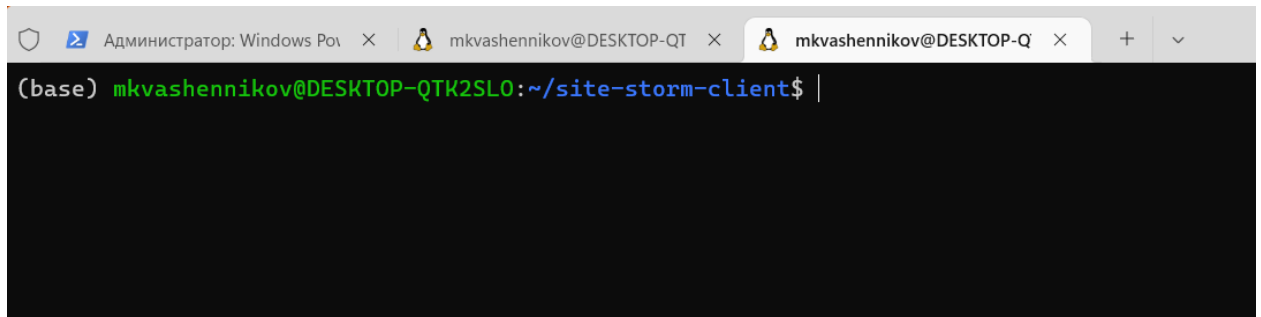


Рис. П4.1. Окно терминала Windows

В окне терминала необходимо открыть папку с программой командой ``cd <наименование папки>`` ввести следующий набор команд и выполнить их по отдельности, нажав клавишу «Enter»:

- «`npm run clean:project`»;
- «`npm run run:browser -- -p 9000`».

Если все выполнено правильно, то в таком случае можно будет в терминале в самом низу увидеть сообщение «Compiled successfully» (рис.П4.2).

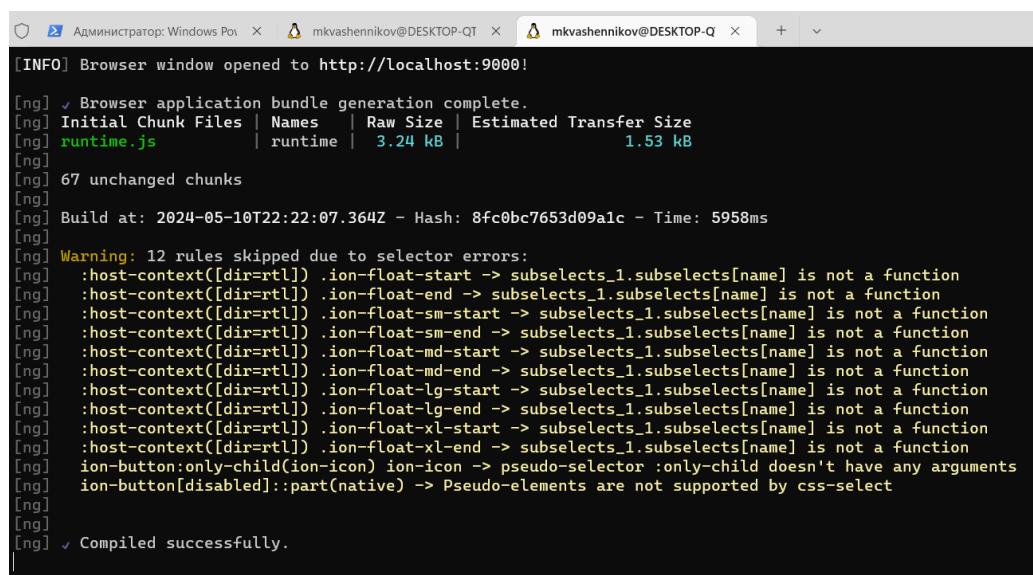


Рис. П4.2. Сообщение об успешном запуске программы

После этого необходимо в поисковой строке браузера ввести адрес «http://localhost:9000», где можно будет увидеть основное окно работы с программой (рис.П4.3).

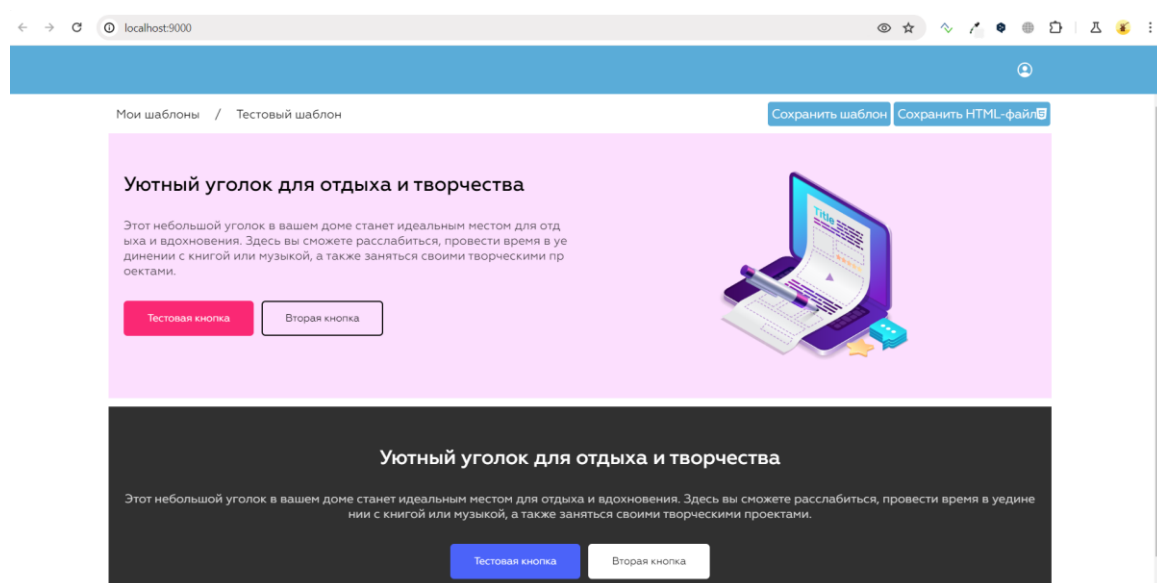


Рис. П4.3. Основное окно работы с программой

Акт о внедрении

Акт о внедрении представлен на рис.П5.1.

УТВЕРЖДАЮ
Руководитель проектов
ООО «АСТРОСОФТ ПРО»
Кондрашев С.А.
« 1 » 06 2024 г.

АКТ О ВНЕДРЕНИИ

Данным актом подтверждается, что результаты выпускной квалификационной работы «Программное обеспечение генератора HTML-шаблонов по рукописным рисункам-макетам», выполненной Квашенниковым Михаилом Сергеевичем, выпускником по направлению подготовки 09.04.04 «Программная инженерия (искусственный интеллект)» в федеральном государственном бюджетном образовательном учреждении высшего образования «Череповецкий государственный университет» внедрены в ООО «АСТРОСОФТ ПРО».

Дата внедрения: 30 мая 2024 г.

Были использованы следующие результаты исследования:

- Алгоритм подготовки тренировочных изображений для нейронной сети, детектирующей объекты на рукописных рисунках-макетах;
- алгоритм формирования HTML-сетки после этапа распознавания типовых объектов с рукописного рисунка-макета.

В ходе эксплуатации программы подтверждено, что она обладает всеми заявленными возможностями.

Руководитель проектов



Рис.П5.1. Акт о внедрении